

CONVOLVING THE CONVOLUTED:
THE LONG & WINDING ROAD TO ACOUSTIGRAMMETRY

by

JUSTIN ALLEY

B.A., University of California Berkeley, 2009

A thesis submitted in partial fulfillment
of the requirements for the degree of
Master of Science
Recording Arts Program (MSRA)
2026
University of Colorado Denver

© 2026

JUSTIN ALLEY

ALL RIGHTS RESERVED

This thesis for the Master of Science degree by

Justin Alley

has been approved for the

Recording Arts Program

by

Cecilia Wu, Chair

Jeff Merkel, Adviser

Alex Thomen

May 16, 2026

Alley, Justin (MS, Recording Arts Program)

Convolving the Convoluted: The Long & Winding Road to Acoustigrammetry

Thesis directed by Assistant Professor Cecilia Wu

ABSTRACT

Numerous approaches have been taken to address the problem of generating navigable virtual models for multi-volume acoustic spaces. The general practice for creating empirically informed interactive models of multi-volume acoustic spaces, as embodied by the Spatially Oriented Format for Acoustics, is to discretely sample emitter-receiver pair positions. For a user to then navigate between these discrete positions involves cross-fading, blending, or otherwise perceptually interpolating between corresponding zones. This thesis outlines a new approach which instead involves the continuous 3-dimensional sampling of acoustic spaces, much as is done with 3D visual spaces in photogrammetry. To achieve this result, a first-of-its-kind consolidated ambisonic impulse response capturing apparatus has been designed and built. This apparatus combines a 3rd-order ambisonic microphone array with a 2nd-order ambisonic loudspeaker array and is designed to be moved through a space with maximal ease. AD/DA conversion, playback, and recording are all handled on a central compute platform. In parallel, a software workflow has been developed which can be implemented in Unreal Engine, as well as other game engines. To solve general issues of spatial audio in game engines, a custom encoding and decoding framework has been implemented. Then, to map the continuous ambisonic impulse response onto a virtual space, a spline mirroring the sampling path is drawn through the space. On the DSP side, an impulse response is extracted from any arbitrary point along the spline by way of the Common-slope Model for coupled spaces. Future work for better addressing early reflections and minimizing the theoretical intermediary of the Common-slope Model is

discussed. Additionally, special use cases for the technology developed in this thesis are also discussed.

The form and content of this abstract are approved. I recommend its publication.

Approved: Cecilia Wu

DEDICATION

To the students of the Shajareh Tayyebah girls' elementary school and all those who seek knowledge in the face of evil.

ACKNOWLEDGEMENTS

I wish to acknowledge the undying support and encouragement of my family and loved ones, in particular Brian and Stephanie Alley, Jimmy La, and Alex Kimoni, and my mentors Professor Krys Bobrowski and Dr. Cecilia Wu, and the other members of my committee, Professors Jeff Merkel and Alex Thomen. I also wish to acknowledge the supportive atmosphere provided by my fellow students and by many others in the educational community not explicitly named herein.

TABLE OF CONTENTS

I. INTRODUCTION	1
Outline	4
II. BACKGROUND	6
Ambisonics	6
B-format	7
A-format	10
Microphone arrays	11
Reverberation	13
Early reflections	13
Late reflections	14
Algorithmic & empirical reverb	15
Impulse responses	17
Spatial Room Impulse Responses	18
Multivolume acoustic spaces	18
Common-slope model	20
Interactivity	21
Angular panning	21
6DoF	22
Interfaces	22

A survey of FOSS for ambisonic reverberation	23
IEM Suite.....	24
SPARTA	26
Envelop for Live	30
Conclusions	31
III. SOFTWARE DESIGN.....	33
Overview.....	33
Conventions.....	33
Software.....	36
Architecture	36
Algorithm.....	39
Unreal Engine.....	39
Max MSP as build environment	43
Patch structure	45
User interface	50
IV. HARDWARE DESIGN.....	52
Physical layout.....	53
Parts & Materials.....	57
Microphones.....	62
Speakers.....	62

DAC-amp boards	63
Microcomputers	64
Connectivity.....	65
GitHub repository	68
V. FINDINGS	72
Case Study #1: SyntheticSTAIR.....	72
Case Study #2: WaysSTAIR.....	73
Comments on verisimilitude	78
Comments on practicality	79
Hardware performance	80
Software performance	80
VI. DISCUSSION	83
Future Work	83
Reducing theoretical intermediaries	83
Occlusion	86
Packageability.....	86
Spatial transforms	87
STAIR reference library	87
Synthetic STAIR generation	88
Multi-spline mappings	88

Microphone housing & polar pattern	89
Improved moveability	90
Applications.....	91
Games & virtual reality	91
AEC	92
Virtual production.....	93
Virtual acoustipresence.....	94
Creative installations.....	94
Concluding summary	95
REFERENCES	98
APPENDICES	108
A. Stereo decoder attenuation matrix	108
B. Binauralization matrix.....	108
C. Quad decoder attenuation matrix	109
D. 7.1.4 decoder attenuation matrix	110
E. STAIR spectrograms.....	111

LIST OF TABLES

TABLE

Table 1: Ambisonic channel numbering	36
Table 2: Stereo decoder attenuation matrix.....	108
Table 3: 3OA A-format array azimuth and elevation angles.....	48
Table 4: Binauralization matrix	108
Table 5: Quad decoder attenuation matrix	109
Table 6: 7.1.4 decoder attenuation matrix.....	105
Table 7: Ambisonic channel orderings.....	50
Table 8: Directivity deviation	63

LIST OF FIGURES

FIGURE

Figure 1: Diagram of a true B-format mic array.....	9
Figure 2: Halliday-Nimbus B-format microphone array	10
Figure 3: Double stage decay.....	19
Figure 4: FdnReverb binaural spectrogram	25
Figure 5: RoomEncoder binaural spectrogram	26
Figure 6: AmbiRoomSim binaural spectrogram	27
Figure 7: MultiConv binaural spectrogram.....	28
Figure 8: MatrixConv binaural spectrogram.....	29
Figure 9: B-Format Convolution Reverb binaural spectrogram	31
Figure 10: Gyroelongated square bipyramid (GSB).....	34
Figure 11: Software architecture.....	38
Figure 12: Orthogonal views of Listener spheres	40
Figure 13: Stereo equilateral triangle.....	46
Figure 14: Frame face CAD drawing.....	53
Figure 15: Fitted dome CAD drawing	54
Figure 16: Compute platform CAD drawing	55
Figure 17: Combined STAIRArray CAD drawings.....	56
Figure 18: Frame faces.....	58
Figure 19: Fitted domes	59
Figure 20: Compute platform.....	60
Figure 21: Assembled STAIRArray.....	61

Figure 22: STAIRArray connectivity.....	67
Figure 23: SyntheticSTAIR virtual environment	73
Figure 24: Photos of residential passageways	75
Figure 25: WaysSTAIRcpp project screenshots.....	76
Figure 26: WaysSTAIR spline mapping	77
Figure 27: WaysSTAIR virtual environment	78
Figure 28: SyntheticSTAIR spectrograms	111
Figure 29: WaysSTAIR spectrograms.....	112
Figure 30: Folded and shaped STAIR.....	85
Figure 31: Foucault pendulum in the Panthéon, Paris	95

ABBREVIATIONS

ABBREVIATION

3OA	3 rd -Order Ambisonics
AEC	Architecture, Engineering, & Construction
C	Center
CAD	Computer-Aided Design
Cft	Center-front top
Cftm	Center-front top-middle
Crt	Center-rear top
Crtm	Center-rear top-middle
DAQ	Digital Acquisition
DSP	Digital Signal Processing
EDC	Energy Decay Curve
FFT	Fast Fourier Transform
FOSS	Free Open-Source Software
GSB	Gyroelongated Square Bipyramid
HOA	Higher-Order Ambisonics
IFFT	Inverse Fast Fourier Transform
ILD	Interaural Level Difference
IR	Impulse Response
ITD	Interaural Time Difference
L	Left
Lfb	Left-front bottom

Lfbm	Left-front bottom-middle
LFE	Low-Frequency Effects
Lh	Left height
Lhr	Left-height rear
LPF	Low-pass filter
Lr	Left rear
Lrb	Left-rear bottom
Lrbm	Left-rear bottom-middle
Ls	Left side
Lst	Left-side top
Lstm	Left-side top-middle
MEMS	Microelectromechanical System
R	Right
Rfb	Right-front bottom
Rfbm	Right-front bottom-middle
Rh	Right height
Rhr	Right-height rear
Rr	Right rear
Rrb	Right-rear bottom
Rrbm	Right-rear bottom-middle
Rs	Right side
Rst	Right-side top
Rstm	Right-side top-middle

SFL	Spectral Frame Looper
SH	Spherical Harmonics
SOFA	Spatially Oriented Format for Acoustics
SRIR	Spatial Room Impulse Response
STAIR	Spatio-Temporal Ambisonic Impulse Response

I. INTRODUCTION

We are living in a renaissance of 3D technologies. From material sciences to 3D graphics and even 3D audio, we are seeing rapid development and adoption of new technologies. With new technologies come new standards and practices. This is best exemplified in the field of 3D graphics. In that field, there exist common interchangeable file formats and integrated suites of software touching every step of the production workflow [1]. One can create a common interchange file such as an .obj or .stl in various CAD software, import that into 3D animation software like Maya or Blender, then bring the dynamic mesh into a game engine like Unreal. Underscoring the full-circle integration for visual modelling, one can even 3D-print physical models from the very same .obj or .stl file. While there is and has been a flurry of activity around 3D audio technologies, the same degree of codification of standards and best practices cannot yet be claimed. A variety of proprietary formats like Dolby Atmos and Sony 360 abound [2],[3]. Open-source formats like ambisonics, the Spatially Oriented Format for Acoustics, and the Audio Definition Model (ADM) exist [4]; and newer formats like the Immersive Audio Model and Formats (IAMF) are being developed by Samsung and Google under the auspices of the Alliance for Open Media [5]. As one can see, we have a patchwork of proprietary and open-source formats, and the landscape for working with these formats, be that in film, music, gaming, or VR, is similarly a minefield of often discontinuous use-cases. Consider, for instance, that while popular game engines and audio middleware like Unity, Unreal Engine, Wwise, and FMOD can natively handle ambisonic assets, due to the reliance on the paradigm of unitary audio listeners, ambisonic assets are almost always downmixed to a stereo output [6]. To be able to work with 3D audio in the ambisonics domain requires yet more third-party software. The problem is further compounded if one wishes to use room impulse responses (RIRs) in VR and game

engines. SOFAs, one of the most common research formats for handling complex RIR datasets, have no pre-built software for game engines. The research embodied in this thesis cannot claim to solve all the issues of balkanization in 3D audio. However, it strives to take a decisive step in the direction of unification in the arena of navigable 3D virtual acoustic models.

To achieve this end, a series of intermediary objectives have to be fulfilled. Firstly, some of the underlying technical issues of working with audio in the ambisonic domain in game engines needs to be addressed. Secondly, we need a way of implementing more seamlessly navigable virtual acoustics in virtual spaces. And thirdly, we need a means of moving away from discretely capturing impulse responses to continuously capturing impulse responses.

As stated, the first of the issues we need to address is the ability to encode and decode ambisonic audio for game engines. Essentially, we need a way to circumvent the paradigm of audio listeners. An audio listener is in effect a virtual microphone. To stretch the metaphor, it is, in fact, a stereo microphone. This picks up the stereo field from the perspective of a single game user. Regardless, of how sophisticated we may get in terms of distributing sound sources in a 3D space, ultimately the complex sound field we have built up will get folded down to stereo. Major game engines like Unity, Unreal, and Godot are not designed to natively employ multiple audio listeners [7],[8],[9]. Some plugins like Magic Listener [10] and middleware like Wwise [11] unlock the ability to use multiple audio listeners for split-screen scenarios; however, even then these are summed to stereo and tapping the submixes to instantiate an array of virtual microphones is by no means straightforward. Instead, to solve this problem, we will describe a simple geometric approach to encoding game audio into 3rd-order ambisonics and then decoding it for various output formats such as stereo (with or without binauralization), quadraphonics, 7.1.4, and raw A-format ambisonics.

The second issue we need to address is how to realize more seamlessly navigable virtual acoustics in virtual spaces. A common method for approaching more realistic virtual acoustics is to apply multiple reverb zones across a virtual space. These reverb zones can be given a variety of shapes, say, rectangular prisms or spheres. Yet each reverb zone corresponds to a single reverb effect. This approach in games has its correlate in the realm of academic research on acoustics in the form of the Spatially Oriented Format for Acoustics. Therein, separate room impulse responses are jointly encoded with metadata for their positions. Then one moves between these separate RIRs by cross-fading, blending, or otherwise perceptually interpolating between the given data points. It should go without saying that the way we experience acoustics in the real world is rarely so compartmentalized as the typical approach of reverb zones. Our acoustic perspective is continuously shifting as we move throughout a space. And while SOFAs can attempt to robustly sample a space with multiple pairs of emitter and receiver positions, it is impossible to predict all of the different compelling changes of acoustic perspective in a space. Rather, we should pursue a technique and technology more akin to that of photogrammetry wherein a more complete 3D model is built up through freely moving through a space.

And the third major issue we need to address is in the hardware side of how to capture continuous ambisonic impulse responses. Typically, sound sources and microphones are kept separate in the sampling of room acoustics. This simply will not work if we wish to be able to scan a space while also moving through it. We need a maximally compact setup. Thus we combine a 3rd-order microphone array and a 2nd-order speaker array into a single consolidated apparatus. In order to make this apparatus as ergonomic and moveable as possible, we strive to contain the recording and playback platform to this apparatus along with an attached power supply. This apparatus can then be used to capture what are called Spatio-Temporal Ambisonic

Impulse Responses (STAIRs). A STAIR is an indefinite number of IRs spread over a mapped route through a space. This mapped route can be of any length (contingent on memory requirements) and of any shape (assuming in topological terms that it is contractible).

Once we have trodden all of this theoretical and design background, we will glean some findings from bringing real-world impulse recordings into the virtual realm. We will consider subjective first impressions, as well as compare objective acoustic and performance data. From this, we will derive pluses and minuses for the technology as it exists at this stage of research. Subsequently, we will discuss future work to be done to polish and expand upon the technology. To finish things out, we will examine a variety of applications, from the obvious (e.g., game audio and VR) to the less obvious (e.g., architectural acoustics and virtual acoustipresence). Taken in its full breadth, the technology becomes something more than a tool for virtual acoustics. It becomes a stepping stone for a discipline we nominate as “acoustigrammetry”.

Outline

Section I: Background

We give an overview of the current state of knowledge pertaining to ambisonics, microphone arrays, reverberation (real and artificial), impulse responses (including SRIRs), multi-volume acoustics, and 6DoF navigability.

Section II: Software Design

We methodically layout the entirety of the software architecture necessary for utilizing STAIRs in a game engine such as Unreal. The geometric encoding and decoding paradigm is walked through. We see how a STAIR is mapped onto a virtual space by way of splines and positional looping of spectral frames. A Max MSP patch for the backend processing involved in

extracting IRs from the looped spectral frames and outputting the convolved audio for various channel formats is explained.

Section III: Hardware Design

The design for a first-of-its-kind consolidated ambisonic IR array is detailed. Parts, materials, electronics specifications, connectivity, software setup, and design choices are all covered.

Section IV: Findings

The STAIRArray is used to sample {two} real world spaces. In a third case, a STAIR is also synthetically generated. All of these assets are applied in demo Unreal projects. Objective and subjective data regarding the results are discussed. Technical useability findings are also discussed.

Section V: Discussion

A host of future directions in which the technology could be further developed is considered. A range of applications from game and VR experiences and beyond are explored. We conclude with a final summary lookback.

II. BACKGROUND

As discussed in the introduction, there are a significant number of proprietary and open-source 3D audio formats. The range of use-cases for these formats is discontinuous. Some are suited for game and VR audio, some are not. In this research, we will focus on the ambisonics format. The reasons for this are that it is a well-established open-source format and that it is ideally suited for use in interactive 3D audio use cases involving dynamic tracking of user movement.

Ambisonics

The era of ambisonics audio finds its beginning with the seminal work of Michael Gerzon [12]. Gerzon initially referred to his approach as “periphony”, from the Greek *peri-*, meaning around or surrounding. Previously there had been primarily horizontal approaches to multichannel audio, such as quadraphonics. Gerzon’s first insight was to add height channels. His second and more paradigm-shifting insight was to realize that the total 3D sound field could be captured and reconstructed mathematically using the theory of spherical harmonics. The concept of spherical harmonics dates back to Fourier and Laplace [13], but perhaps finds its first practical purchase in chemistry in describing the residence of electron pairs in orbital shells [14]. From there, the idea finds further development in quantum mechanical descriptions of electron clouds and particle spin dynamics [14]. Thus by the mid-twentieth century, the idea of describing distribution patterns in space with greater and greater resolution through the superimposition of higher-orders of spherical harmonics was becoming more entrenched in the advanced sciences.

Succinctly, the theory states that values on the surface of a sphere can be decomposed into spherical harmonic functions. These functions have positive and negative lobes, and thus take the lobar or figure-of-eight shape. In the case of a spherical sound field, each harmonic

represents a sound pressure gradient in a particular direction. The so-called zeroth order information, the omnidirectional component of all higher orders, represents sound pressure in all directions. The number of channel components necessary to represent a full-sphere is given by the formula $(l + 1)^2$, where l is the spherical harmonic order [15]. The basic form of ambisonics audio embodying the core of this mathematical approach, known as B-format ambisonics, is explained below.

B-format

In B-format ambisonics, a source signal S is encoded into spherical harmonics $WXYZ$ by taking its azimuth angle θ and elevation angle φ in a spherical coordinate system and multiplying according to the following trigonometric formulae [15]:

$$\begin{aligned} W &= S \cdot \frac{1}{\sqrt{2}} \\ X &= S \cdot \cos \theta \cos \varphi \\ Y &= S \cdot \sin \theta \cos \varphi \\ Z &= S \cdot \sin \varphi \end{aligned}$$

W is the omnidirectional component and thus represents the average energy in all directions.

To then decode B-format ambisonics into a basic, equally spaced playback system including height channels, the spherical harmonic components are matrixed thusly:

$$\begin{aligned} Lf &= W + \frac{1}{\sqrt{2}}(X - Y + Z) \\ Lfh &= W + \frac{1}{\sqrt{2}}(-X - Y + Z) \\ Rf &= W + \frac{1}{\sqrt{2}}(X - Y - Z) \\ Rfh &= W + \frac{1}{\sqrt{2}}(-X - Y - Z) \\ Lr &= W + \frac{1}{\sqrt{2}}(X + Y + Z) \\ Lrh &= W + \frac{1}{\sqrt{2}}(-X + Y + Z) \end{aligned}$$

$$Rr = W + \frac{1}{\sqrt{2}}(X + Y - Z)$$

$$Rrh = W + \frac{1}{\sqrt{2}}(-X + Y - Z)$$

So that, for instance, if S is located at 45° azimuth and 30° elevation, then its level normalized in the above speaker array will be as follows:

$$\begin{aligned} Lf &= -11.445 \text{ dB} \\ Lfh &= -8.484 \text{ dB} \\ Rf &= -2.597 \text{ dB} \\ Rfh &= 0 \text{ dB} \\ Lr &= -31.858 \text{ dB} \\ Lrh &= -29.261 \text{ dB} \\ Rr &= -23.554 \text{ dB} \\ Rrh &= -20.777 \text{ dB} \end{aligned}$$

Since B-format ambisonics is made up of an omnidirectional component and successive lobar components, a microphone array for capturing B-format ambisonics would have to be constructed similarly. Namely, it would have to have one omnidirectional microphone, and 3 bidirectional microphones as shown in Figure 1. This is practical enough for first-order B-format ambisonics, but even then it is often simplified by omitting the height component, as in Figure 2, which shows the array that Dr. Jonathan Halliday used to produce ambisonic recordings for Nimbus records [16].

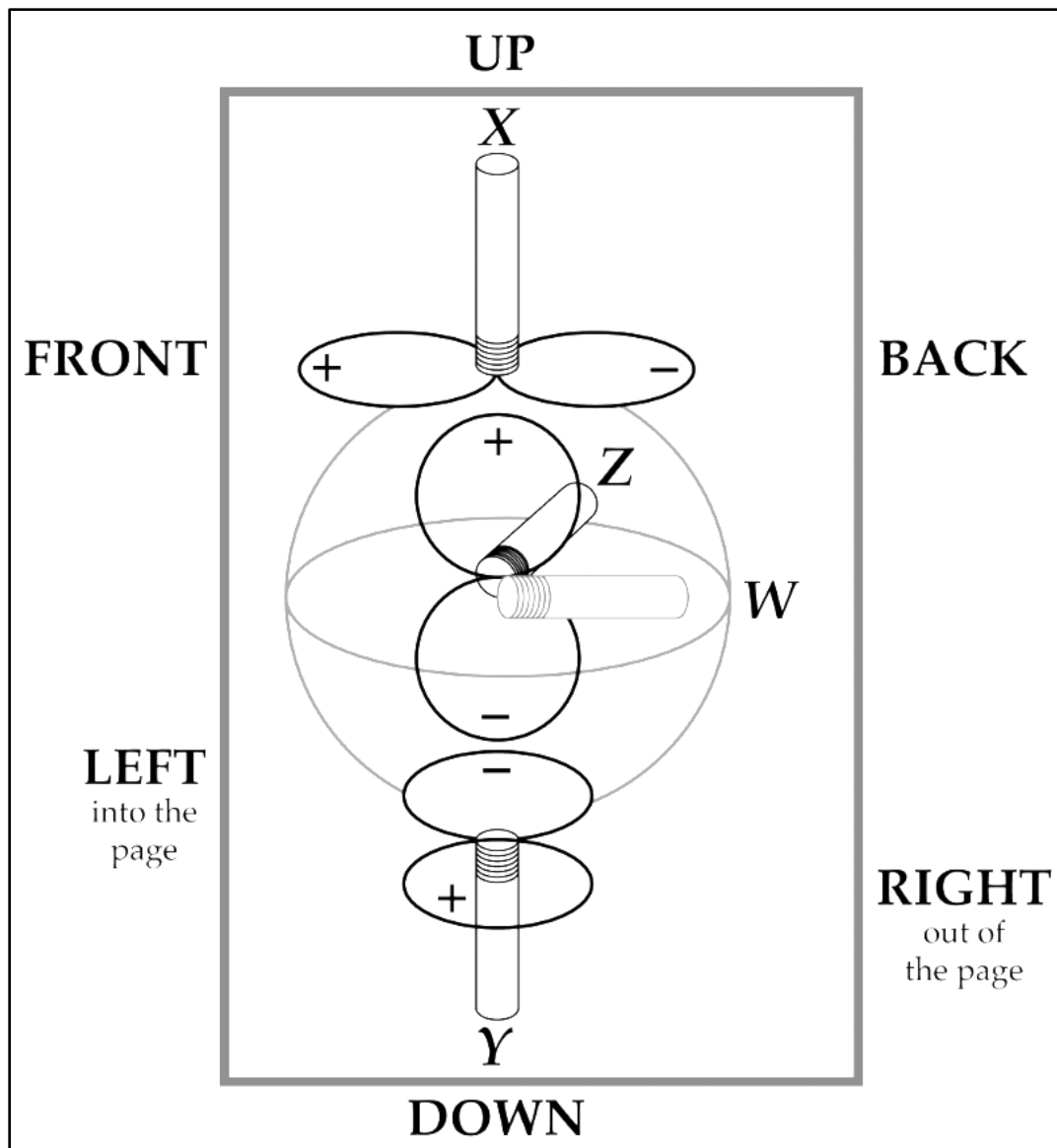


Figure 1: Diagram of a true B-format mic array (from [17])

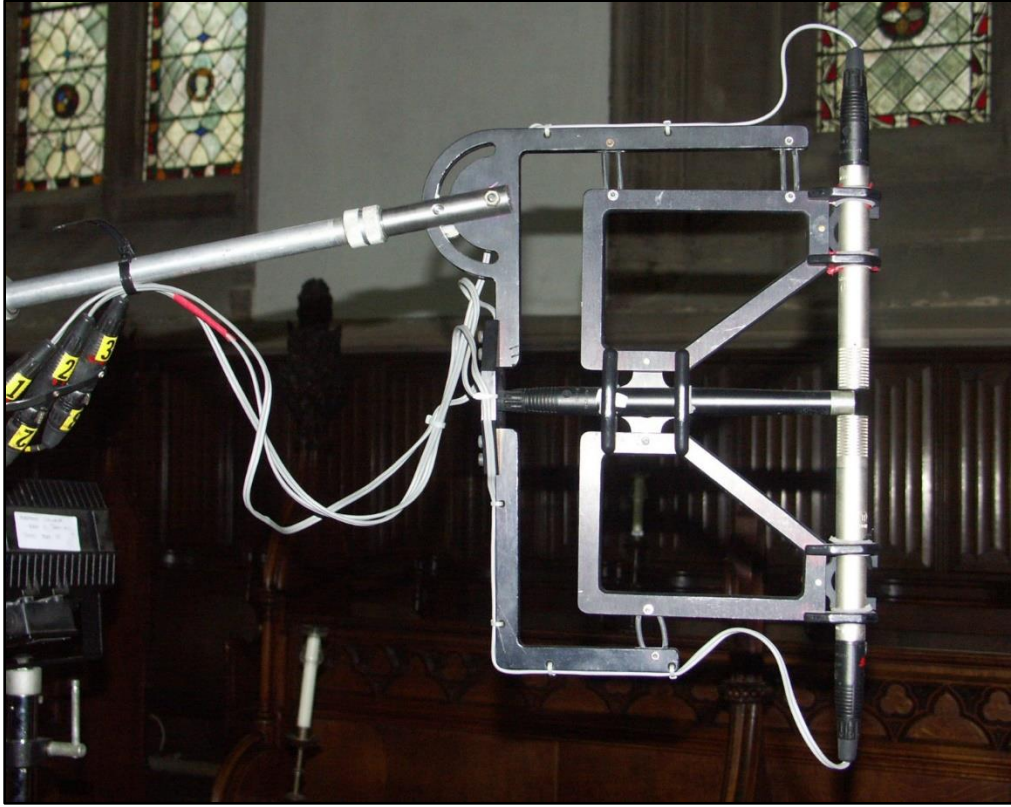


Figure 2: Native B-format microphone array designed by Dr. Jonathan Halliday for Nimbus records; note that it omits the additional figure-8 mic that would be necessary to record the height component [16]

A-format

A-format ambisonics is the raw input and output format for ambisonics. As stated above, B-format ambisonics implies the usage of lobar polar patterns to represent spherical harmonics. As the ambisonic order increases, so does the number of spherical harmonics, and thus so does the number of superimposed lobar patterns. As a consequence, it becomes impractical to create more than a first-order B-format ambisonic microphone array. It simply becomes harder and harder to position bidirectional microphones around an omnidirectional microphone. Furthermore, there is no such thing as a B-format ambisonic loudspeaker array, for the simple reason that bidirectional loudspeakers are not a typical phenomenon.

For all of these reasons, when higher-order ambisonic audio is recorded, or when *any* ambisonic audio is converted to channel-based audio, it enters the domain of A-format ambisonics. What is lost with A-format ambisonics is the ability to perform a set of standard ambisonic transformations such as spatial rotations and distortions [18]. Nevertheless, an ample amount of software exists for converting between A- and B-format ambisonics [19],[20]. Ultimately, the same formulae that are used to encode a signal into B-format ambisonics and then decode B-format ambisonics into some specified output format can be used with little to no changes to convert between A- and B-format ambisonics. All that needs to be known, as with the source signal itself, are the azimuth and elevation angles for each A-format channel. Consequently, A-format microphone and loudspeaker arrays can take many configurations, such as tetrahedra, octahedra, spheres, and even horizontal rings [21] – [24]. All that is necessary is that the geometry of the array and microphone polar patterns is precisely specified, with equalization of values of course simplifying the math that is involved in decoding and encoding the results.

Microphone arrays

Microphone arrays have been designed for a variety of purposes, from sound field recording to room acoustic analysis to teleconferencing to robotic sensing [25],[26],[27]. In this part of the discussion, we will focus on microphone arrays that are designed to record in the 360° sound field domain. Not all such microphone arrays are natively ambisonic, but many are. Research clearly shows that as the number of microphones goes up, spatial acuity increases, by which is meant the ability to accurately localize a sound in space [28]. In the case of ambisonics, this research shows that the sweet spot for spatial acuity is between the 5th and 7th orders

inclusive, meaning such arrays necessitate 36 to 64 microphones. Above the 7th order are diminishing returns [29].

Of the variety of microphone arrays, we can divide them into 3 categories: 1. rigid, 2. open, and 3. rotating. Rigid arrays, such as the Eigenmike em 64, are built into closed housings [30]. Open arrays, which are usually purpose-built by researchers, mount mics on a frame that lacks closed boundaries [31]. Finally, rotating arrays use a lower number of microphones but rotate them in space to upscale their spatial resolution [32]. Rigid arrays are more physically precise, use well-understood math, and thus enable applications involving narrow beamforming [31]. Open arrays are more functionally flexible, as well as acoustically transparent, particularly at high frequencies, due to their reduced scattering from the lack of a rigid shell [31]. Finally, rotating arrays allow for unlimited upscaling in the spatial reconstruction of the sound field; in addition, because they start with fewer mics (e.g., 2 to 16) for every position, they are good for capturing multiple binaural perspectives [32].

As the number of mics in any array increases, the connectivity issues naturally compound. Above the 2nd order, we see a divergence from using purely analog connections to using digital protocols like USB audio [33], A2B [34], DANTE [35], and, in the case of high-density arrays using hundreds of mics, lab-grade digital acquisition (DAQ) equipment [36]. Similarly, as the number of microphones increases, the mechanical size of the microphones becomes an added problem. Above the 2nd order, we see a move away from traditional electromechanical microphone designs and toward electret and micro-electromechanical systems (MEMS) [33],[34]. The basic MEMS microphone package is omnidirectional and, when using quality components, is remarkably sensitive, flat, and RF immune [37],[38],[39]. The primary

loss relative to larger-diaphragm microphones is in the low-frequency response [37],[38],[39]. However, given the poor localization of low frequencies, this seems an acceptable tradeoff [40].

Reverberation

Reverberation is the prolonged decay of acoustic energy due to the conditions of an environment [41]. As sound reflects off of surfaces and objects in a space, it reaches a listener multiple times with successive delays. Eventually, enough acoustic energy is lost with each successive reflection that the echoes fall below the threshold of detection, often defined as T_{60} , the time it takes such a sound to fall below -60 dB.

Over the course of transiting a space and arriving again and again to a listener, the reflected sound imparts additional layers of information to a listener. Direct sound, that is, sound before any reflections, gives a listener a base sense of directivity and distance [42],[47]. Following the direct, line-of-sight sound, propagation through the space enters two subsequent stages—early reflections and late reflections—to be discussed in greater detail below.

Early reflections

Early reflections are the distinctive echoes of a sound that occur in the first 50 – 100 ms after the direct sound [43]. Due to the sparsely random character of early reflections, short of directly simulating them through ray tracing or image-source methods, there exists a dearth of rigorous general models for understanding them. That said, one of the more rigorous findings with respect to early reflections is to be found in the work of Abel and Huang [44], which allows for the estimation of so-called mixing time (known in other work as the cross-over time [45]). The mixing time is the transition point between the early and late reflections of a reverb response. To estimate this, we measure the convergence of the amplitudes of the reflections to a Gaussian process [44]. Similarly, Götz has developed a method which can be exploited in multi-

channel audio whereby the space-time diffuseness of the sound field is measured [46]. In other words, when all channels reach a more or less uniform saturation, the late reverberation stage can be said to be met. Finally, it is worth recognizing that this very distinction between late and early reflections is due to the limited temporal resolution of human hearing [42],[47]. An analogy can be drawn to the distinct perception of low-frequency and high-frequency oscillations.

Following on these psychoacoustic considerations, the contributions that early reflections have on our perceptions of a space account for our primary sense of the geometry and materiality of a space. For instance, early reflections can give us a sense of whether a space is wide or long, and it can indicate whether there are substantial objects causing acoustic shadows in one direction [42],[47].

Late reflections

After the first 50 – 100 ms of a sound transiting around a space, in which it reflects anywhere from three to a dozen times, individual reflections start to stack up and blend together [43]. If we recall that early reflections have a stochastic character, then we can infer that as these stochastic events multiply and become more dense in time, they will take on a noise profile. Specifically, late reflections converge toward Gaussian noise [48]. As energy continues to dissipate with each reflection, the amplitude of this Gaussian noise in turn begins a fairly predictable decay. Late reverb is primarily defined by this decay rate. It is precisely because of the predictability of late reverb that we are able to produce generalized models for it. Nevertheless, late reverb still imparts two highly important psychoacoustic pieces of information about a space. First, it gives a sense of the size of a space. Intuitively, longer decays correspond to larger spaces. And, second, it gives a sense of the overall absorptive properties of a space

[42],[47]. Again, the more absorptive a space is, say, through sound dampening materials, the shorter the decay will be.

The effects of reflectivity and absorption can be brought together in the following equations:

$$R_{energy} = \frac{Z_{wall} - Z_{air}}{Z_{wall} + Z_{air}}$$

Where R_{energy} is the reflected energy in terms of the impedance of a surface boundary Z_{wall} and air Z_{air} . In turn, the effect of absorption on acoustic energy is given by:

$$A_{energy} = 1 - |R_{energy}|^2$$

Thus the reflectivity and the acoustic impedances of a space determine its acoustic energy decay curve.

Algorithmic & empirical reverb

Within the field of virtually modelling reverb, which itself can be understood as an analog for the acoustic response of a system, there are two broad approaches [41]. The first is what we might term the algorithmic approach. In this approach, reverb is modelled through base algorithmic processes such as delays and filtering (which are themselves an abstraction on top of time differentials). The second approach is the empirical approach. In this approach, reverbs are modelled by sampling characteristics of actual spaces. The most common way this is done is by taking an impulse response of an acoustic system and then convolving that with a novel input. Impulse responses are discussed in greater detail below. But it is worth noting that the empirical approach need not only work with impulse responses. It could also involve sampling 3D meshes using LIDAR and photogrammetric technologies and then running wave solvers on those meshes. Broadly speaking, algorithmic approaches are more efficient but less realistic than empirical approaches.

As with artificial reverberation generally, so too in the case of multi-volume spaces, approaches to developing navigable models diverge along algorithmic and empirical lines. Algorithmic approaches draw upon algorithmic reverb models such as feedback delay networks (FDNs) and graphical models of ray tracing and image source methods. Empirical approaches draw upon techniques of impulse response sampling and metadata encoding. The benefits of algorithmic approaches are that they can be derived from arbitrary geometric meshes and that in some cases they are more computationally efficient. The downsides are that they often sound less realistic and have to be fine-tuned. Alternately, empirical approaches come ready-made with the fine details of reality but need to be derived from particular spaces and involve more computationally taxing convolutional processing. Hybrids exist, such as reverbs that use FDNs for early reflections and IRs for late reflections [49]. Hybridization exists too at an even deeper level in that FDNs can be trained on reality [50] and convolutional processing always involves algorithmic considerations [51].

In terms of empirically driven approaches to modelling complex, positionally-variant spaces, the prevailing paradigm is embodied in the Spatially Oriented Format for Acoustics (SOFA) [52]. In this standardized container format, multiple discrete source and receiver positions are encoded with attendant metadata. SOFAs have a number of use cases, but, in interactive immersive implementations, these discrete positions can be navigated between with varying degrees of freedom. Some of the more sophisticated methods for handling transitions between positions take a perceptually informed approach [53]. It remains, however, that discretely capturing a space can be time consuming and can leave open questions of robustness and completeness.

Impulse responses

The concept of an impulse response (IR) was first introduced in the quantum mechanical work of Paul Dirac [54]. Succinctly, a certain class of systems known as linear-time invariant (LTI), i.e., a system whose *total* response is not dependent upon when it is sampled in time, can be completely modelled by its response to a known impulse. An ideal impulse is infinitely short and infinitely high in amplitude. This is written:

$$\delta[n] = \begin{cases} 1, & n = 0 \\ 0, & \text{elsewhere} \end{cases}$$

Meaning that at time zero the delta function is at its maximum and everywhere else it is at its minimum [55].

If we set the impulse to a system equal to the delta function, we get:

$$\delta[n] \xrightarrow{\text{yields}} h[n]$$

Thus $h[n]$ denotes the response of the system. As we shall see later, once we have $h[n]$, we can convolve this with any arbitrary input and observe how the system would respond.

In the audio domain, we also want an impulse to contain all frequencies within certain bounds (usually 20 Hz to 20 kHz). In reality, it is not possible to have an impulse which is infinitely short and infinitely loud. An infinitely loud impulse would destroy any audio equipment. And an infinitely short one would make frequencies illegibly compact in the time domain.

Within the realm of IRs, we can also distinguish between those that are finite (FIRs) and those that are infinite (IIRs). Those that are finite will return to zero in a finite amount of time after being perturbed. Those that are infinite will not. They involve some feedback component [55].

An important property of an IR is its energy decay curve (EDC). The EDC measures acoustic energy over time. Additionally, by having the full EDC, we can determine the reverb time (RT) of an IR. The formal mathematical definition of the EDC is:

$$EDC(t) = \int_t^{\infty} h^2(\tau) d\tau$$

One famous method for computing the EDC from an IR is Schroeder’s so-called “backwards integration” [56]. Backwards integration derives this integral from the *back* or tail of the impulse where the response is assumed to be normalized below a certain threshold. From that point backwards, a smoother version of the EDC can be interpolated from a spikey IR.

Spatial Room Impulse Responses

It is possible to take impulse responses of many different kinds of systems, such as mechanical and electrical systems. When an impulse response is specifically of room acoustics, it is known as a Room Impulse Response (RIR). When it is a room impulse recorded in multiple dimensions, as with an ambisonic impulse response, it is known as a Spatial Room Impulse Response (SRIR). In this thesis, we will speak primarily of Ambisonic Impulse Responses and a special kind that is indefinitely spread out over time which we call a Spatio-Temporal Ambisonic Impulse Response. A STAIR is kind of hybrid between an FIR and an IIR. A STAIR is an answer to alternate methods for spreading SRIRs through space such as the SOFA discussed throughout this background section.

Multivolume acoustic spaces

Multivolume or coupled acoustic spaces are ones with appreciably different reverberation characteristics for different locations. Typically, coupled spaces are understood as spaces connected by an aperture of some kind [57]. In this text, where possible, we shall prefer the term “multivolume” because it does not only connote spaces connected by an aperture but also spaces

with sufficiently non-uniform geometries such as undulating or serpentine passageways. The primary reverb parameter that is “anisotropic”, i.e., variable with position and orientation, in multivolume spaces is late reverb [58]. Because sound decays faster in a smaller room than a larger room, when a smaller and a larger room are connected, we get two superimposed energy decay slopes, as pictured in Figure 3.

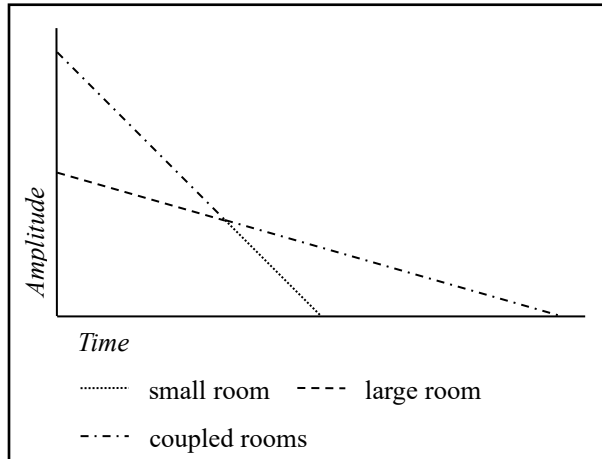


Figure 3: Double stage decay

A range of approaches to modelling multivolume spaces have been proposed. The Image Source Method for modelling acoustic spaces immediately presents itself as well-suited for multivolume spaces because it does not require the sound source to be in the line-of-sight of the listener [59]. However, as with ray-tracing and other path-plotting methods, sound is only approximately specular, that is, line-like, at high frequencies [60]. Thus such approaches may continue to fail to capture all of the empirical properties of an actual acoustic space. Feedback delay network approaches also exist whereby FDNs are grouped according to the various volumes of the space to which they correspond. Yet, as with FDNs generally, the problem of tuning the FDNs remains, now further exacerbated by the problem of tuning the coupled

feedback matrix [67]. Other methods such as differentiable GFDNs, that is, trained GFDNs [50], and neural acoustical fields exist [62].

In the end, the question is not so much which of these models is “correct”, but, rather, which is applicable to the situation. We will consider one final method which is most applicable to our situation, the common-slope model.

Common-slope model

The common-slope model treats late reverb in multivolume spaces as a linear sum of exponential decays, much like the waterfall graph which is a derivative of an IR through backwards integration [63, 56]. The common-slope model proceeds from the fundamental conceptualization of multivolume acoustics in terms of the following equation [63]:

$$d(\mathbf{x}, t) = \sum_{k=1}^K A_{k,b}(\mathbf{x}) e^{-\frac{13.8t}{T_{60_{k,b}}}} + N_{0,b}(\mathbf{x})(L - t)$$

In this formula, $\mathbf{x}$ denotes the 4-tuple encoding of the source and receiver positions in 2 dimensions, and $A_{k,b}$ denotes the amplitude of a spectral bin b for a given space k . $N_{0,b}$ is a noise term that establishes a floor to the tail of the decay. And L is the length of an energy decay curve (EDC) as determined by Schroeder’s backwards integration. Thus as we acquire more source and receiver data for a multivolume space, and as we perform EDC curve analyses on more spectral bins, we develop a more complete model of the complex dynamical acoustics of such a space.

This formulization of the common-slope model is exceptionally compact and ripe for parsimonious DSP application. Nevertheless, one acknowledged criticism of the common-slope model is that it requires an impossibly robust dataset of RIRs, otherwise the model will yield

anomalous results [63]. A guiding question of the research embodied in this thesis arises: What if we could efficiently capture an extremely large dataset for a space?

Interactivity

How does a user's activity dynamically affect the presentation of a sound field in real time? This is the question of interactivity with which we will now concern ourselves.

Angular panning

An ambisonic audio recording allows for an intrinsic kind of interactivity. Namely, once you have a recorded sound field, you are presented with the possibility of three-dimensional angular panning within that sound field. Firstly, this implies a decoding to a sub-sampling of the sound field in the form of a perspective on that sound field. Generally, the sound field is decoded to stereo and perhaps binaural playback. Secondly, the submix contained in that decoded audio stream depends on the angular disposition of the stereophonic perspective relative to the ambisonic sound field [15].

The exact character that angular panning takes in an ambisonic context depends on the decoder matrix that is used. Various decoding schemes exist for angularly distributing sources. The first and default method is energy-preserving decoding, whereby the spherical harmonic field is reproduced at speaker positions in a 1-to-1 fashion [64]. This reproduces a smoothly diffused sound field. But perhaps a crisper localization of sources is desired. Then the maximum energy vector (max-rE) method may be used. This method attempts to steer energy in the direction of the lobes [65]. It is akin to using more directional virtual microphones to sample from the sound field. Finally, and the most directional approach, builds upon vector-based amplitude panning (VBAP) [66]. In this method, energy is no longer spread across all speakers

but across clusters of speakers. Ultimately, while the choice of scheme depends on the desired end, EP serves as the pure ambisonic starting point.

6DoF

After sound becomes open to the possibility of shifting angular perspective, the next logical step is to open it to shifting translational perspective. If the three rotational vectors of pitch, roll, and yaw comprise the three-degrees-of-freedom, then combined with the translational vectors of lateral, horizontal, and vertical motion, we have the typical six-degrees-of-freedom [67]. Then the question becomes, how do we enable these three additional vectors when interacting with 3D sound? To sharpen the question further in the context of this thesis, how do we enable these three additional vectors when interacting with 3D virtual acoustics?

One of the most sophisticated approaches that exists for enabling 6DoF in virtual acoustics is to perceptually interpolate between the SRIR positions of a SOFA [53]. Given a finite number of SRIRs spread through a space, particularly a complex multivolume space, the problem becomes how to transition between these SRIRs with minimal artifacts. To do so, SRIRs are broken down into perceptually salient components such as direction-of-arrival from SH gradients, early reflection window from EDC, and RMS-based magnitude fitting of late reverberation. It is worth noting that the whole reason we perceptually interpolate between SRIRs is because it is considered impractical to sample the SRIRs of large and complex spaces with sufficient density.

Interfaces

Human ears, unlike those of certain animals such as canines and felines, are not poseable and thus have only one pivot point, that of the head joint [68]. On the other hand, human eyes have two pivot points, those of the eye sockets and, again, the head joint (amazingly, the animal

with the most physically complex ocular system, the mantis shrimp, has 5 pivot points) [69]. Because VR headsets generally have both headtracking and headphones built into one system, they inherently increase the coupling between visual perspective and auditory perspective in that both are locked to the motion of the same head joint. This is different from when one looks at, say, a computer screen since it is possible to turn one's head while keeping one's eyes locked on the screen. Or, in another case, one could be wearing headphones while looking at a computer screen. This creates a most unnatural scenario where, even if one is not looking at a screen, one's auditory perspective remains effectively locked straight ahead. Finally, and in a somewhat different sort of case, one could be sitting in front of a screen but be surrounded by loudspeakers. Thus one could naturalistically scan the sound field but at the loss of consistent visual feedback. We conclude, therefore, by noting that in terms of currently available technologies, some form of headtracking technology, be it a VR headset, smart glasses, or a head-mounted smartphone, provides the most synchronized coupling of our auditory and visual senses [70].

A survey of FOSS for ambisonic reverberation

Since ambisonic reverberation is a central concern of this research, it is worth now taking a look at the software that already exists for achieving some manner of ambisonic reverb. Furthermore, we will restrict our focus to free and open-source software (FOSS) because it is the most amenable to being extended by public research. Among the list of FOSS for ambisonic reverberation, three rise to the top: 1) IEM Suite, 2) SPARTA, and 3) Envelop for Live. In each case, we will examine the general philosophies, test out applications, peer into the code, and finally derive conclusions for possible future developments of software for ambisonic reverberation. Every effort was made to set each reverb within its category (i.e., algorithmic or empirical) to the same median settings. In all cases, the same program material of anechoic snare

and violin was used, which was panned centrally in the sound field. Similarly, where possible, the same 1st-order B-format ambisonic impulse response of York Minster Cathedral was used across empirical reverbs. In terms of a testbed, all unnecessary processing was disabled when running each instance of reverb. Except for the Envelop for Live plugins, which could only be tested in Ableton Live, all reverbs were tested in both Reaper and Ableton Live. Maximum CPU load was recorded along with the output audio and a resultant spectrogram. Relevant waveform statistics were thereupon computed. Besides the aforementioned objective data, phenomenological and UX observations were also made. Listening tests were performed binaurally and over a 7.1.4 Genelec speaker array.

IEM Suite

The IEM Suite of plugins was developed at the Institute of Electronic Music and Acoustics at the University of Music and Performing Arts, Graz. It is available under a GNU General Public License as published by the Free Software Foundation, Version 3 (GPLv3), and the most recent stable release was in April 10, 2025.

The two ambisonic reverb plugins in this suite are FdnReverb and RoomEncoder. Both were coded in C with a JUCE wrapper and dependencies on FFTW (a C subroutine library for computing Discrete Fourier Transforms) in Windows and Linux, with additional dependencies in Linux. They are available in the VST, VST3, LV2, AU, and AAX plugin formats and are capable of up to 7th-order ambisonics [71].

FdnReverb

FdnReverb employs one of the classic techniques of algorithmic reverb, namely the networking of multiple delay lines, here applied in the ambisonic domain. High- and low-shelf

filters and a switchable high-pass filter are supplied. Room size, reverb time, and feedback-delay network (FDN) size are adjustable.

This reverb has the characteristic grainy sound of a digital FDN-based reverb. The resultant ambience sounds like a diffuse cloud existing at a distance behind the source. Based on the spectrogram (Figure 4), one can observe the haziness of the spectral image as compared to some of the other reverbs discussed in this paper.

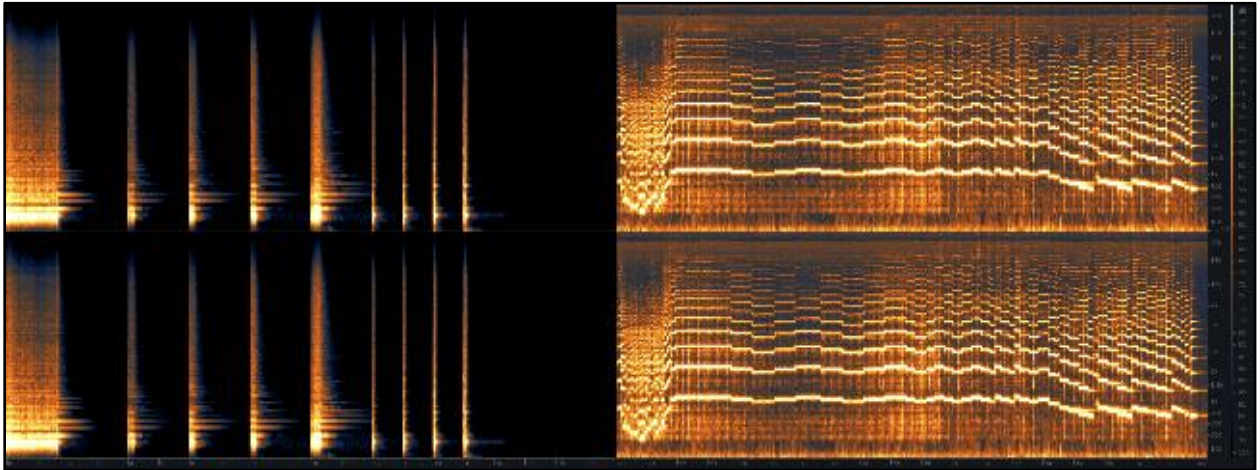


Figure 4: FdnReverb binaural spectrogram snare first half, violin second half; average total RMS across both channels: -15.41 dB.

RoomEncoder

RoomEncoder, while nevertheless also algorithmic, aims for a higher degree of realism than FdnReverb. It takes a physical modelling approach to reverb. Room dimension (depth, width, & height) are adjustable, as are source and listener positions within this specified space. Source position was kept centered, with listener position at half the distance between source and surfaces. Attenuation for every surface (front, left, right, & back walls, & ceiling & floor) is individually adjustable. With the underlying ray-tracing approach of this physical model, the number of reflections is variable from 0 to 236.

The results are serviceably room-like. The sound is broadly damped across multiple spectral bands, likely due to the use of spatially oriented absorptive coefficients (more on this in the discussion of AmbiRoomSim below).

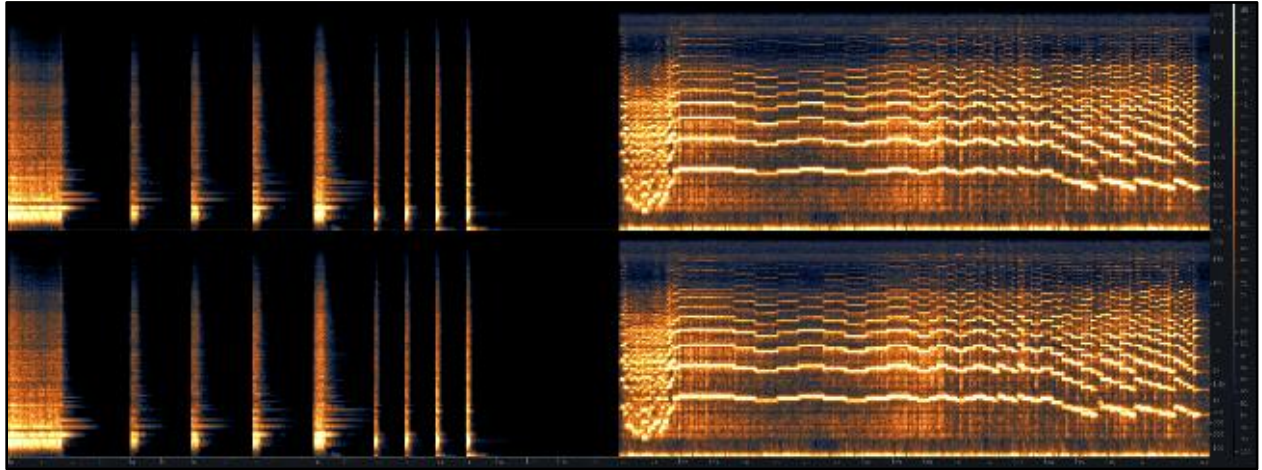


Figure 5: RoomEncoder binaural spectrogram; snare first half, violin second half; average total RMS across both channels: -15.98 dB.

SPARTA

The Spatial Audio Real-Time Applications (SPARTA) suite of plugins was developed at the Acoustics Lab of Aalto University, Finland. It is available under GPLv3, and the most recent stable release was in February 2, 2024.

SPARTA is based on Spatial_Audio_Framework, a cross-platform framework for developing spatial audio algorithms and software in C/C++, in a JUCE wrapper. There are 4 ambisonic reverb plugins in this suite: AmbiRoomSim, MultiConv, MatrixConv, and 6DoFconv. They are available in the VST and LV2 plugin formats and are capable of up to 10th order ambisonics [72].

AmbiRoomSim

Like RoomEncoder, AmbiRoomSim is a physical model-based algorithmic reverb. The two plugins are functionally and experientially quite similar. They both have controls for room

dimension, source and listener positions, number of reflections, and spatial-spectral attenuation. Indeed, when compared, the binaural spectrograms for the two plugins are nearly indistinguishable (cf. Figures 5 & 6). One definite difference is that the total RMS for AmbiRoomSim is nearly 2 db less. This is likely due to the median number of reflections being lower. Once again, source position was kept centered, with listener position at half the distance between source and surfaces.

As with RoomEncoder, AmbiRoomSim presents a believable simulation of small- to medium-sized rooms. One downside of this physical modelling approach to reverb is that because it is based on a simplified model of an acoustic phenomenon, it tends to overaccentuate the key parameters of that model. In particular, because it makes use of absorptive coefficients to lend greater acoustic realism to the basic ray-tracing algorithm, it broadly damps the spectral response of the reverb, resulting in what could be considered a duller, dryer sound.

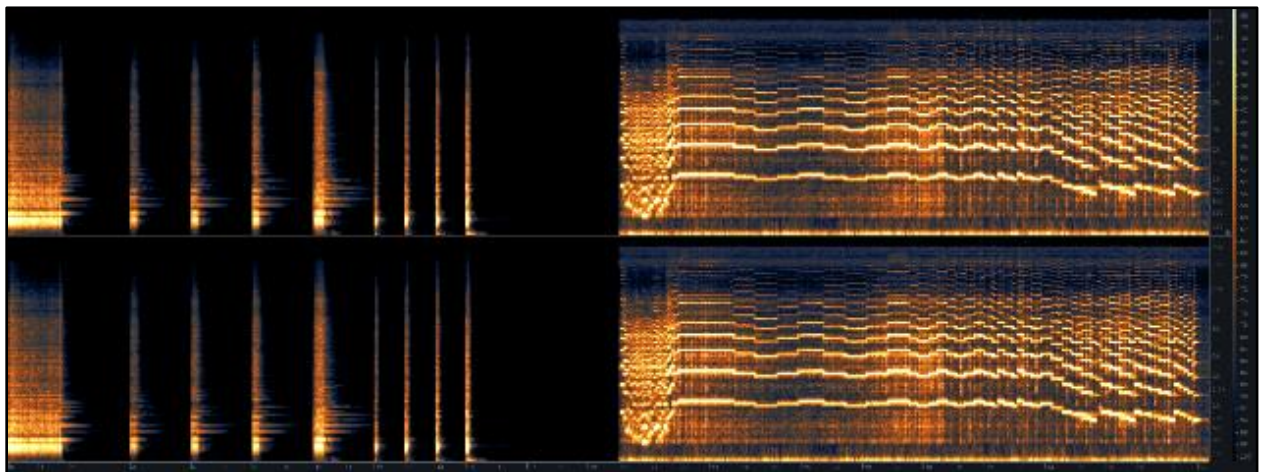


Figure 6: AmbiRoomSim binaural spectrogram; snare first half, violin second half; average total RMS across both channels: -17.97 dB.

MultiConv

The first of the empirical reverbs discussed in this section, MultiConv, is also the simplest. Unlike MatrixConv to follow, no matrixing is performed, such that the number of inputs = number of filters = the number of outputs [72].

When used with the York Minster Cathedral B-format IR, the complex echoing structure of real-world spatial acoustic coupling is audible. Incidentally, as compared to all the other empirical reverbs tested in this article, the reverb tail is inordinately extended. Since no decay controls are provided, this quality is baked into the effect.

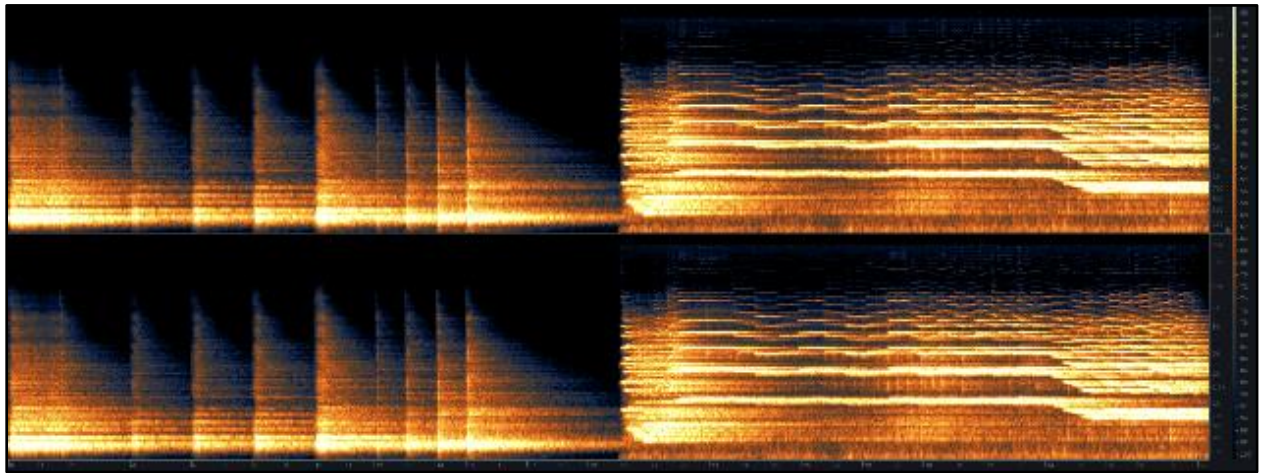


Figure 7: MultiConv binaural spectrogram; snare first half, violin second half; average total RMS across both channels: -15.76 dB.

MatrixConv

To the previous convolutional reverb mentioned, MatrixConv adds control over the matrixing of the inputs. If the number of input channels is arbitrarily increased, this is multiplied by the fixed number of output channels to get the total number of filters through which the signal is processed. As the number of filters derived from the IR is increased, the size of each filter in the time-domain is decreased.

This architecture has two audible results. The more filters the IR is divided into, the more resonant and the more gated is the reverb. So while this architecture allows for more control, it does not necessarily result in more naturalism.

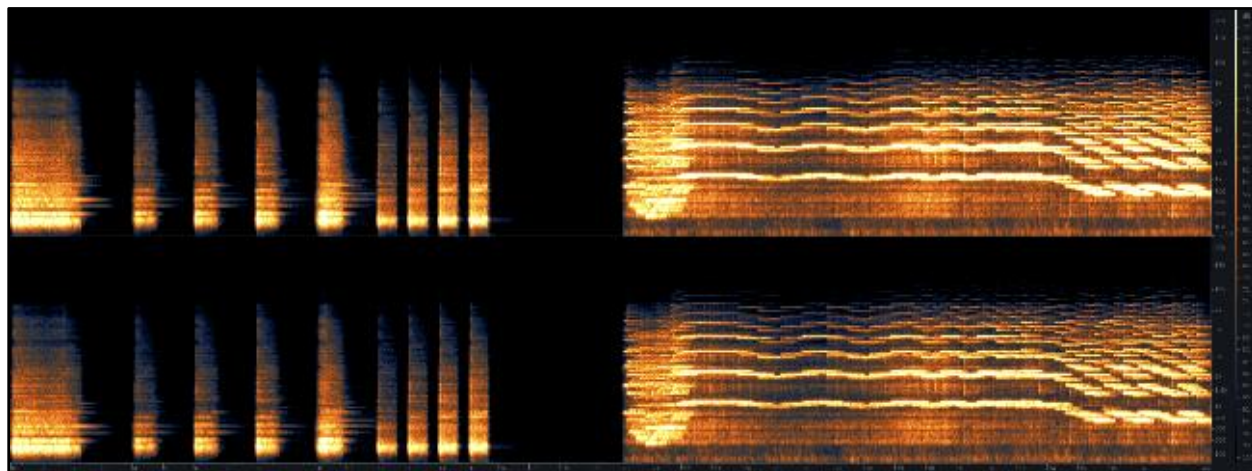


Figure 8: MatrixConv binaural spectrogram; snare first half, violin second half; average total RMS across both channels: -18.07 dB.

Finally, a comment can be added now regarding both MultiConv and MatrixConv, which is that they are both approximately 1.5 orders of magnitude more CPU intensive than all the algorithmic reverbs tested in this article.

6DoFConv

6DoFConv is theoretically the most sophisticated of all the reverbs discussed in this article. Rather than load raw multi-channel IRs, it works with the Spatially Oriented Format for Acoustics (SOFA) file specification. SOFAs are essentially datasets of ambisonic IRs with spatial metadata attached [52].

Due to the complexity of its implementation, the 6DoFConv plugin was not testable for this research. However, it remains conceptually interesting because it aims to address a fundamental issue of virtual acoustics, which is the navigation of multiple acoustically coupled spaces [53].

Envelop for Live

The Envelop for Live collection of Max for Live devices was developed by the Envelop nonprofit out of San Francisco. They are available under GPLv2, and the most recent stable release was in September 6, 2024 [73].

There are two reverb plugins in this collection. The first, AuraVerb, is an algorithmic reverb. However, it is deprecated on newer devices such as Apple Silicon and was also nonfunctional on a Windows 10 machine. Looking into the program, it contains a broken `zita_rev1_st_to_1h1v~` object. Nevertheless, the mention of the classic Zita-Rev1 algorithm suggests that AuraVerb is essentially an FDN-based reverb [74]. The other reverb, the B-Format Convolution Reverb, will now be discussed in greater detail below.

B-Format Convolution Reverb

The B-Format Convolution Reverb is built around the `multiconv~` object. While used in many stereo contexts, such as Ableton's own convolution reverbs, the `multiconv~` object was originally developed as part of the Huddersfield Immersive Sound System (HISS) Tools Project, with the intent that it be scaleable to multichannel contexts [75]. Thus it finds prime use in this ambisonic plugin.

The plugin itself is capable of taking in and outputting up to 3rd-order ambisonics; however, those channels are convolved through a 1st-order B-format ambisonic impulse response. The directional and omni components can be loaded individually. Then they are broken into early and late reflections, and alongside the dry and wet signals, each can be separately equalized and balanced. Additionally, the plugin allows for scaling of the decay and rotation of the sound field (incidentally using IEM code).

Of all the reverbs tested in this article, the B-Format Convolution Reverb sounds the most realistic. The decay is neither overly gated nor overly elongated. What is more, as can be seen from the spectrogram (Figure 9), in comparison to the algorithmic reverbs, decay is naturally linear across the frequency spectrum as opposed to being concentrated in the lower bands. Likewise, without the presence of granular haze, definition is maintained in the highest bands. At the same time, high frequency extension is much better than the two other convolution reverbs discussed in this article. Lastly, while no direct comparison can be made in terms of computational efficiency, it is safe to say that it is more intensive than the algorithmic reverbs; whether it is more or less so than the other convolution reverbs remains an open question.

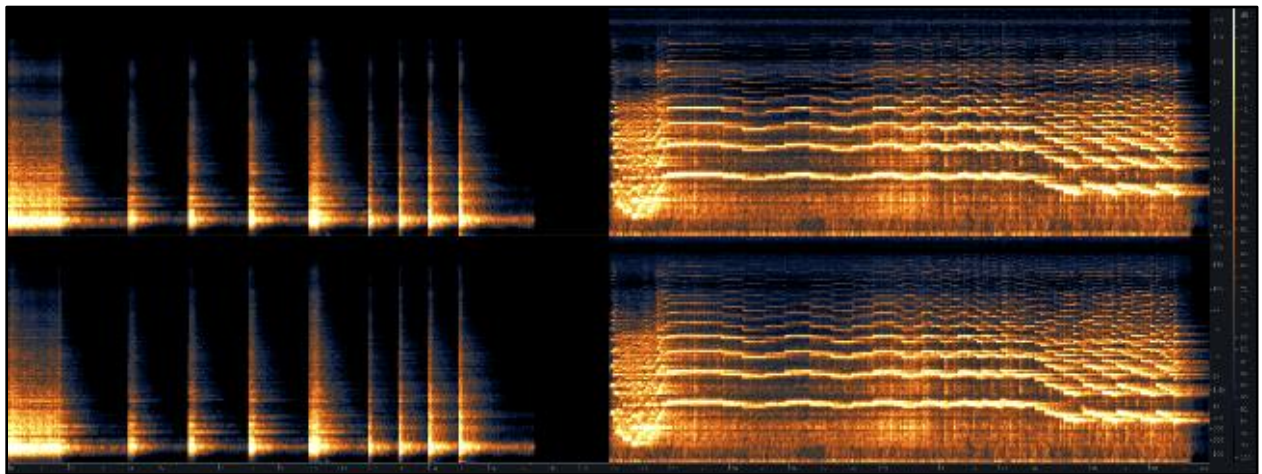


Figure 9: B-Format Convolution Reverb binaural spectrogram; snare first half, violin second half; average total RMS across both channels: -17.91 dB.

Conclusions

The overarching metric by which all ambisonic reverbs have been assessed is what we might call their “verisimilitude”, namely their ability to simulate real acoustic spaces. This ability depends principally on two factors: accuracy and efficiency. If a model does not accurately fit real-world expectations, it will fail at verisimilitude. Yet if it does not operate efficiently in real-time, it will also fail at verisimilitude—especially in the context of game and VR audio. Thus

somehow a balance must be struck between the efficiency of the algorithmic and the accuracy of the empirical.

Looking to the future, one of the next steps for virtual acoustics is to provide navigable models of multivolume acousttic spaces. Traditional algorithmic reverbs inherently fail at the simulation of large, complexly reverberant spaces. In a mathematical sense, such spaces have too large a parameter space to be captured by a linear parametric algorithm [76]. Rather, an adaptive algorithmic approach could hold promise. Such adaptive algorithms have previously found purchase in acoustically informed data compression [77],[78],[79]. Add to this the fact that research has shown that “grouped” feedback delay networks (GFDNs) can reliably simulate coupled acoustic spaces while maintaining computational efficiency and stability [61]. While this may be so, it would be impractical to hand tailor these GFDNs for every acoustic space. Thus the issue is of how these algorithms are allowed to adapt to unique situations.

On the other side, empirical reverbs face their own challenges. According to its documentation, 6DoFConv provides a perceptually informed model for how to interpolate between RIRs with different listener positions. One possible critique, though, is that its dependency upon discrete RIRs means it leaves unaddressed how to robustly sample the RIRs of multivolume spaces. Instead, perhaps we should take a cue from photogrammetry and look at ways of continuously sampling spaces. Another possible critique is that the perceptual model might overfit the data. An alternative approach would be to train a neural net on energy decay data, as discussed by Götz et al. [80]. In sum, possible paths for extending ambisonic reverberation technology to coupled spaces exist. Which path to take remains an open and active area of research.

III. SOFTWARE DESIGN

Overview

The next two chapters of this document concern the parallel design tracks for the software and hardware respectively. While the software and hardware are capable of existing separate from each other and finding their own independent use cases, there are considerable correspondences between aspects of their design, particularly around geometry. It may be necessary at times to make cross-references between the software and hardware designs. In that vein, we will begin with some general conventions applying to both software and hardware. Following that, we will focus for the remainder of this chapter on critical aspects of the software design.

Conventions

For ease of communication, we need to define some conventions for how to orient ourselves in a spherical coordinate system. We should underscore that these conventions apply primarily to the 3OA A-format array of this research and are not intended as a one-to-one replacement for more extensible B-format component ordering systems such as Furse-Malham [81] or ACN [82].

3OA requires us to distribute 16 points evenly on a sphere. There are different algorithms for evenly distributing points on a sphere [83]. The first general class, and also the least satisfactory in terms of exactitude, is the stochastic class of algorithms. In short, x points are randomly thrown at y sections of a sphere. Perhaps the process is repeated n times with some additional fitting constraint c so that the points converge toward maximum evenness. Such algorithms are thoroughly unsatisfactory for ambisonics work for a host of reasons, primarily to

do with the difficulty of yielding symmetrical quaternions corresponding to orders of spherical harmonics.

The second general class of algorithms is the spiral class of algorithms. Basically, x points are equally spaced on a cord and that cord is wrapped around a sphere w times. The higher w grows, the more even is the distribution. However, it is always asymptotic with respect to maximum evenness. Furthermore, the points will not necessarily coincide with spherical harmonics.

The third and most satisfactory (for our purposes) class of algorithms is the Platonic class. Simply put, for x points, find a regular polyhedron with f faces, where $x = f$; then place each point in the center of each face tangent to a given sphere. These algorithms only work if the number of points equals the number of faces of a regular polyhedron. These algorithms work great for ambisonics and implicitly appear in many ambisonics array designs. In our case, the convex polyhedron with 16 regular faces is the gyroelongated square bipyramid (GSB)[84], pictured in Figure 10.

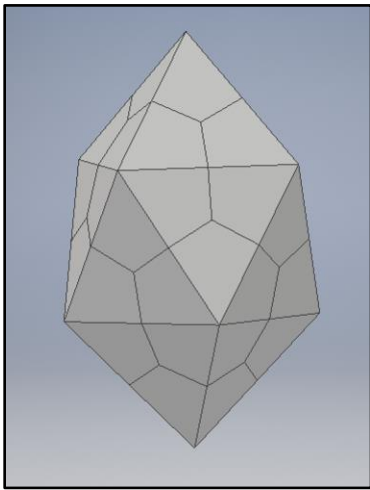


Figure 10: Gyroelongated square bipyramid (GSB)

The default orientation for all 3OA arrays herein is for the vertex of all four convergent edges of one square pyramid of the GSB to be located at 90° elevation, with one face of that pyramid perpendicular to 0° azimuth.

Both default channel ordering and naming are chosen for maximum legibility and are used throughout this document and anywhere else those names appear, such as code and design files.

The default channel ordering proceeds from the top down, with the topmost face perpendicular to 0° azimuth as channel 1, the topmost face perpendicular to -90° azimuth as channel 2, the topmost face perpendicular to 90° azimuth as channel 3, the topmost face perpendicular to 180° azimuth as channel 4, and so on down.

The default naming convention for channels is as follows. If a channel is oriented perpendicular to 0° or 180° azimuth, it is known as a center channel (C). If a channel is oriented perpendicular to a negative azimuth angle, it is known as a left channel (L). If a channel is oriented perpendicular to a positive azimuth angle (0° and 180° exclusive), it is known as a right channel (R). If a channel is oriented perpendicular to -90° or 90° azimuth, it is known as a side channel (s). If a channel is oriented perpendicular to an azimuth angle less than -90° or 90° , it is known as a front channel (f). If a channel is oriented perpendicular to an azimuth angle greater than -90° or 90° , it is known as a rear channel (r). If a channel is located in the top square pyramid, it is known as a top channel (t). If a channel is located in the bottom square pyramid, it is known as a bottom channel (b). If a channel is located in one of the faces of the gyrolongated body which is perpendicular to a positive elevation, then it is known as a top-middle channel (tm). If a channel is located in one of the faces of the gyrolongated body which is perpendicular to a negative elevation, then it is known as a bottom-middle channel (bm). Naming components

are concatenated in the order given in this paragraph. Table 1 lays out the correspondence between channel order and channel names.

Table 1: Ambisonic channel numbering

Channel order:	Channel name:
1	Cft
2	Lst
3	Rst
4	Crt
5	Cftm
6	Lstm
7	Rstm
8	Crtm
9	Lfbm
10	Rfbm
11	Lrbm
12	Rrbm
13	Lfb
14	Rfb
15	Lrb
16	Rrb

Software

Architecture

To begin the discussion of the software implementation, let us give a high-level overview of its architecture. Broadly, the two main software environments we will work with are Unreal Engine and Max MSP. Unreal Engine handles gameplay. In other words, it handles user interactions. It also handles the playback of source audio. Source audio is spatially deposited about a game map. Then, through a process of distance attenuated submix sends explained in the section below on Listeners, that source audio is encoded into 3rd-order ambisonics. Another crucial process that is handled in the game engine is the positionally based looping of spectral frames from the STAIR. This is described in the section on spectral frame looping. Finally, a three-part shared memory bridge will be explicated. This bridge consists of 1) a C++ compiled submix sender effect in Unreal; 2) a purpose-built Max external; and 3) a Max MSP receiver

object. The Max patch which receives the audio from Unreal will be referred to throughout as the STAIRProcessor. The STAIRProcessor does two main things: 1) it carries out the DSP algorithm to be described in the following section; and 2) it decodes the audio for various output formats: stereo (with or without binauralization), quadraphonics, 7.1.4 ITU surround, and raw A-format ambisonics.

Figure 11 gives a visual representation of the signal flow for this software architecture. Some of the details of this signal flow will become more clear in subsequent sections.

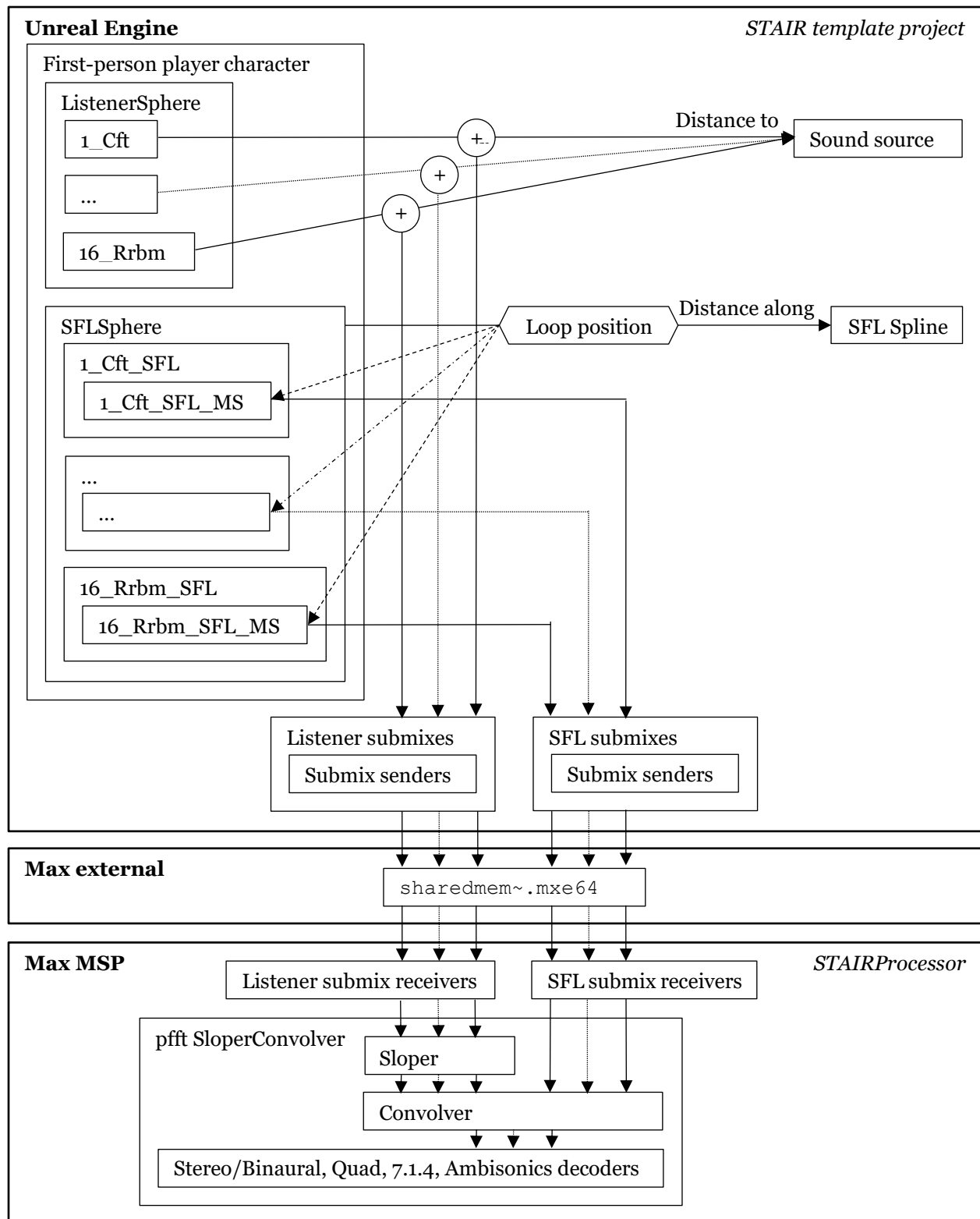


Figure 11: Software architecture

Algorithm

The basic algorithm underlying the digital signal processing for the STAIR concept is as follows:

STAIRProcessor Algorithm: exponential decay function, spectral convolution

Input: Listener audio streams & Spectral Frame Looper (SFL) audio streams
Output: Convolved audio output

Let L_n denote a given Listener channel.

Let S_n denote a corresponding SFL channel.

forall L_n and S_n , perform a Fast Fourier Transform using a Blackman window-envelope function, such that $\text{FFT}(L_n) = \mathbf{R}_L + \mathbf{I}_L$ and $\text{FFT}(S_n) = \mathbf{R}_S + \mathbf{I}_S$, where $\mathbf{R}$ and $\mathbf{I}$ denote the real and imaginary components.

A Blackman window-envelope function is chosen because some smearing of the SFL and the sloped Listener spectra is desirable.

forall $\mathbf{R}_L$ and $\mathbf{I}_L$, convert from the Cartesian coordinates of $\text{FFT}(L_n)$ to polar coordinates, such that $\mathbf{R}_L = \mathbf{A}_L$ and $\mathbf{I}_L = \mathbf{P}_L$, where $\mathbf{A}$ and $\mathbf{P}$ denote the amplitude and phase components for a particular spectral bin b .
The conversion is necessary so that we can apply an exponential time decay to the amplitudes and phases of the spectral bins of $\text{FFT}(L_n)$.

forall $\mathbf{A}_{L,b}$ and $\mathbf{P}_{L,b}$, apply an exponential decay function such that

$\mathbf{A}_{L,b}, \mathbf{P}_{L,b}(t) = (\mathbf{A}_0, \mathbf{P}_0)e^{-6.19t/T_{60}}$, where T_{60} is the established standard of how long it takes acoustic energy to decay to -60 dB .

The constant of -6.19 is derived from the fact that -60 dB is a drop by a factor of 10^{-6} , the natural log of which is 13.815, which in turn is multiplied by -2 in the general exponential decay function, and hence simplifies to -6.19 .

end

Convert from polar coordinates back to Cartesian coordinates, such that $\mathbf{A}(t) = \mathbf{R}_L'$ and $\mathbf{P}(t) = \mathbf{I}_L'$.

Convolve $(\mathbf{R}_L', \mathbf{I}_L')$ as well as $(\mathbf{R}_S, \mathbf{I}_S)$ by taking the complex product, such that $\mathbf{R}_C = (\mathbf{R}_L' \times \mathbf{R}_S) - (\mathbf{I}_L' \times \mathbf{I}_S)$ and $\mathbf{I}_C = (\mathbf{R}_L' \times \mathbf{I}_S) + (\mathbf{I}_L' \times \mathbf{R}_S)$.

end

Finally, perform an Inverse FFT, $\text{IFFT}(\mathbf{R}_C, \mathbf{I}_C) = C_n$, where C_n is the convolved audio output.

end

Unreal Engine

The following will describe the basic project setup necessary for getting the Unreal game engine to implement the STAIR concept. It is worth noting that a similar setup can and has been

implemented in the Unity game engine. However, Unreal has been focused upon for this research due to its next-gen status in terms of graphics and audio functionality.

Basic setup

To begin with, we create a blank C++ project in Unreal. For our purposes, we will call this project STAIRUnrealProject. After we save this project, we close it and then start Unreal's demo first-person game. From this project, we migrate essential content to our STAIRUnrealProject, specifically all content folders except for LevelPrototyping. Then we close the first-person demo game and reopen our STAIRUnrealProject.

Listeners

Once back in our STAIRUnrealProject, we open the BP_FirstPersonCharacter Blueprint . As a child to FirstPersonCamera, we create a sphere collider with a radius of 150 and call this "ListenersSphere". Then as children to the Listeners sphere collider, we create 16 additional sphere colliders, each with a radius of 50. These correspond to the 16 channels of our 3OA array. Figure 12 shows orthogonal views of this spherical structure.

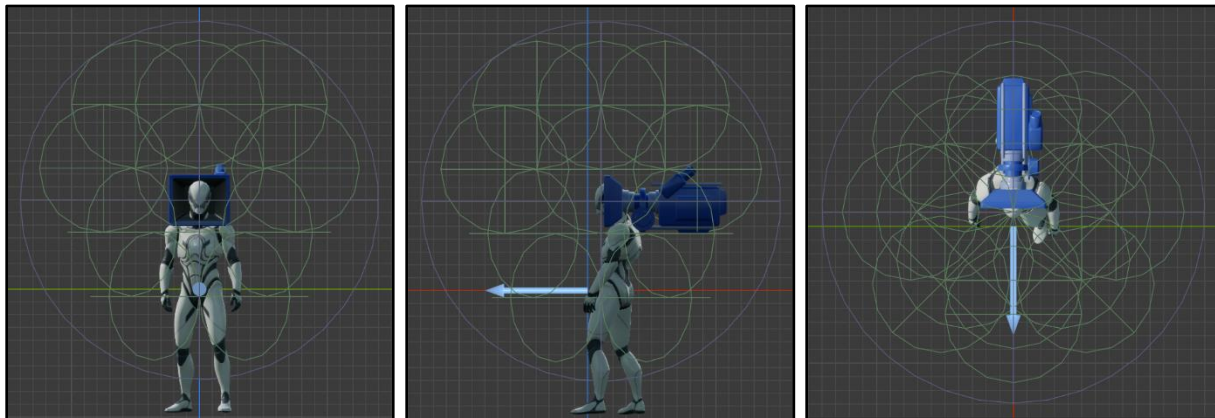


Figure 12: Orthogonal views of Listener spheres

We note that the centers of the child spheres of the ListenersSphere are situated centrally within the faces of a gyroelongated square bipyramid (GSB), The GSB is important to the

software and hardware design for STAIRS, and mention of it will recur from hereon out. It is important to us because it is the only known convex polyhedron with 16 regular faces [84]. Thus it is ideal for 30A.

Finally, we copy the location coordinates of FirstPersonCamera and paste these to the location coordinates of Listeners so that the visual and auditory perspectives share coincident origins.

Back in the content folder of our STAIRUnrealProject, we create a subfolder called Submixes. In this folder we instantiate 16 Sound Submixes and name them according to the following convention: \$ChannelNumber_\$ChannelName_Submix. We create an additional Sound Submix we call Custom_Master_Submix. We route each of the aforementioned 16 Submixes into Custom_Master_Submix and then in project settings change our default Master Submix to Custom_Master_Submix. This way everything we hear in the game will be routed through our 16 channel-specific submixes and not be heard otherwise.

Because Unreal, as well as various audio middleware software like FMOD and Wwise, only effectively allow for one stereo audio listener, our 16 channel-specific submixes serve as multiple listeners for routing audio according to the inverse square law for acoustic energy. In other words, each of the 16 sphere colliders we have created represents a microphone for picking up sound from the 3D soundfield of the game.

The only thing that needs to be changed for each instantiation of this Blueprint concept is the node with the audio component name to match the audio component to which it is attached.

Spectral frame looper

Next we construct the infrastructure within Unreal for positionally looping spectral frames from our given STAIR.

First we duplicate our entire Listeners sphere and all its children and set its parent to the FirstPersonCamera. We should rename the sub-parent sphere to “SFLSphere” and suffix its children with “_SFL”.

Unlike the listeners sphere, we want to lock the rotation of the SFL sphere. This way, the SFL sphere tracks the translational movement of the first-person character but not their rotational movement. Why do we want this? When the STAIR is captured, the STAIR array is not meant to rotate at all. Imagine what would happen if when we capture a STAIR we have the reflections from one surface to our left but in the game the SFL sphere is allowed to rotate as the first-person character rotates their perspective. The result would be that the recorded reflection from that left surface would be transferred to a different point in space. To the best of our ability, we always want to preserve the directionality of reflections. We only want to change the location from which all directions are captured. The simplest way to lock the rotation of the SFL sphere in UE 5.7 is to switch its rotation type from relative to world in the details panel.

Next we want to build the apparatus for looping spectral frames from our STAIR recordings. We can do this by building on Unreal’s MetaSound sources. We begin with some housekeeping by creating a folder called “IRs” in our content folder. In there we create a new MetaSound Source and call it “SFL”. Our MetaSound source will need three inputs: OnPlay, Wave Asset, and LoopStart. The Wave Asset Input node we connect to a Get Wave Info node and to a Wave Player (1.0, Mono) node. We also connect the OnPlay Input node to the play pin of the Wave Player (1.0, Mono) node. Then we convert the time value of the Duration pin of Get Wave Info to a float and feed this into the Out Range B pin of a Map Range (Float) node. To the In pin of this same node, we feed the LoopStart Input node. Then we convert the float Out Value of the mapped range back to a time value and feed this into the Loop Start of the wave player. There are

two settings of the wave player to note. One, we want to check the box to enable looping. And two, we want to set the loop duration to 1.0 for 1 second. Why this specific setting? We want a spectral frame to be able to represent all audible frequencies according to the Nyquist-Shannon theorem. If the STAIR was recorded at 48 kHz, then this means the spectral frame needs to be one second to have the chance of reproducing any arbitrary frequency in the audible spectrum. If we had recorded at 96 kHz, the spectral frame could be 500 ms. And so on. Finally, the Out Mono of the wave player is sent to the Out Mono Output node.

Conceptually, we note that the LoopStart is controlled by the first-person character's distance along a given spline. This position is mapped between 0.0 and 1.0. Thus within the MetaSound Source patch we also need to map this to the duration of the wave asset, which is precisely what we take from the Wave Info node.

We duplicate SFL so we have a total of 16. We change their names according to the following format: \$ChannelNumber_\$ChannelName_SFL_MS. Then, within each, we change the default value of the wave asset to correspond to the appropriate channel of the STAIR.

Max MSP as build environment

In an ideal scenario, all code would be brought into and packaged with the Unreal game project dependencies, either as Blueprint s, C++ classes, or an Unreal plugin. However, these present time and resource challenges beyond the horizons of this current research. In addition, Unreal's audio engine is heavier than, say, Unity's, while at the same time seeming to offer constrained processing resources on the audio thread. In a simple test, more than 10 instances of Unreal's built-in convolution reverb effect at a sample rate of 48 kHz and a buffer size of 1024 samples causes audible CPU crunch. To support convolution-type reverb in 3OA, we need the equivalent of 16 instances. Another downside of building all of our audio processing in Unreal is

that the engine does not make audio outputs easily addressable. Short of installing an unsupported ASIO plugin, the way in which Unreal addresses output channel routing is by responding to the standing configuration as defined by the operating system.

One approach to handling our audio processing and that solves our build challenges is to change our venue to Max MSP. Max MSP allows us to quickly iterate our software. It handles audio processing more flexibly and thereby potentially more efficiently; and it allows us to address specific audio outputs for surround and other configurations. To make this setup work, we need to be able to send audio from Unreal to Max. We could take several approaches, such as using virtual ports (i.e., JACK or VB-Cables) or network sockets (i.e., UDP or TCP). However, the approach that yields the lowest latency and the highest quality is to transmit audio streams via a Memory Mapped File in shared memory. One caveat is that, other than building on a library such as Boost.Interprocess which abstracts OS differences (outside of the scope of this research), this approach only works on Windows. We will now proceed to describing the process for achieving shared memory streaming, and then we will step back to look at the overall structure of our Max patch.

Shared memory streaming

To achieve shared memory streaming of audio from Unreal to Max, we need to implement two chunks of code. On the Unreal side, we need a C++ submix effect that sends the audio of a submix. On the Max side, we need a custom built external compiled against the Max SDK.

We will not go into detail here about the code structure of either of these two things. Suffice it to say, they can be found in GitHub repository discussed later in this document. All that needs to be said for now regards the optimal settings. For a combination of maximum stability,

quality, and minimum latency, it is recommended to run a ring buffer size of 8192, an I/O vector size of 256 and a signal vector size of 256 while at a sample rate of 48 kHz.

Patch structure

To bring everything together in Max, we create a Max patch called STAIRProcessor. This patch receives the audio streams from the ListenerSphere and the SFLSphere. These are all broken out accordingly and then sent into various subpatches.

The first subpatch of note is the SloperConvolver. This implements the STAIRProcessor algorithm laid out in pseudocode toward the head of this chapter.

After this, we have 16 convolved audio streams. These are sent two to four decoder subpatches. The concepts underlying these decoder subpatches will now be discussed in considerable detail.

Stereo

By far the most common channel format in which people listen to audio is stereo [85],[86]. Therefore, a stereo decoder is highly warranted. An ideal stereo setup creates an equilateral triangle between the listener and the left and right channels. To geometrically derive the stereo decoder, we place the vertex representing the listener position at the origin of our spherical harmonics; and then, keeping the equilateral triangle on the zero elevation plane, we place the left and right channels tangent to the unit circle (cf. Figure 13).

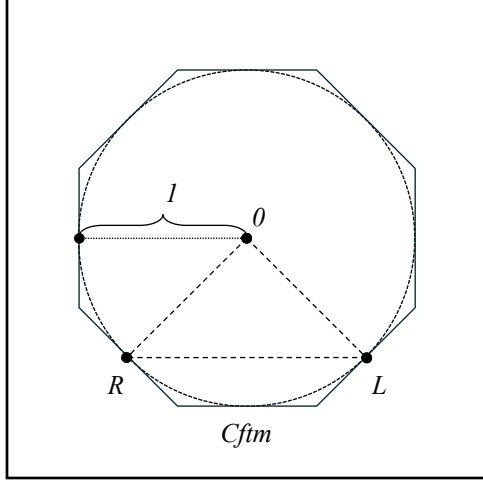


Figure 13: Stereo equilateral triangle with vertices at 0 and tangent to the unit circle inscribed in the zero elevation plane of the GSB

Next, as in all other decoder cases, we take the inverse square of the vector distance between the tangent point of each face of the GSB to the unit circle and the left and right vertices of the stereo triangle. The general formula for computing the attenuation coefficients for this and all subsequent decoders is as follows. If $\overrightarrow{L_W} = (L_x, L_y, L_z)$ is the world-location for a Listener channel, and $\overrightarrow{D_W} = (D_x, D_y, D_z)$ is the world-location for a decoder channel, then the vector distance between L_W and D_W is:

$$V = \sqrt{(D_x - L_x)^2 + (D_y - L_y)^2 + (D_z - L_z)^2}$$

Given this vector distance and the inverse square law for acoustic intensity over distance, we get that the attenuation coefficient $C = \frac{1}{V^2}$ or, substituted and then simplified, $C =$

$$\frac{1}{(D_x - L_x)^2 + (D_y - L_y)^2 + (D_z - L_z)^2}.$$

These values are clamped and scaled between 0.0 and 1.0, where 0.0 is $-\infty$ dB and 1.0 is unity gain. Computed for every Listener and decoder channel pair, this yields the attenuation matrix of Table 2 found in Appendix A.

Binauralization

Not all stereo audio is consumed over stereo loudspeakers. More often than not, it is consumed over headphones [86],[87]. Therefore, to achieve a higher degree of immersion over headphones, binauralization may be desired. Consequently, on top of the stereo decoder, we have implemented a switchable binaural setting which performs the following binauralization processes: 1. interaural level difference (ILD); 2. interaural time delay (ITD); 3. head-shadow filtering; and 4. azimuth & elevation pinna filtering [47].

ITD is given by the following formula:

$$ITD(\theta) = ITD_{max} \cdot \sin \theta$$

ITD_{max} is defined on an average sized head and is taken to be the time it takes sound to transit from one side of the head to the other, here set to be 0.6 ms.

ILD is given by the following formula:

$$ILD(\theta) = ITD_{max} \cdot \sin \theta$$

ITD_{max} is defined along similar lines, here set to be 10 dB.

The typical high-frequency boost at around 3 – 4 kHz for on-ear sounds is given by the following formula:

$$HF = 1 + 0.15|\sin \theta|$$

We can convert this to dB by taking $20\log_{10} HF$, which in turn gives us the gain for a high-shelf filter with a fixed cutoff of 4 kHz and slope of .5.

The azimuth-dependent head-shadow low-pass filter cutoff is given by the formula:

$$LP(\theta) = 1 - 0.5|\sin \theta|$$

And the azimuth-dependent pinna notch filter cutoff is given by the formula:

$$N(\theta) = 1 + 0.1 \sin \theta$$

Furthermore, the elevation-dependent pinna low-pass and notch filter cutoffs are given by the formulae:

$$LP(\varphi) = 8000(1 + 0.002\varphi)$$

And:

$$N(\varphi) = 7000(1 - 0.003\varphi)$$

Intuitively, as the elevation moves higher, the notch moves higher, and, alternately, the LPF cutoff moves lower.

The azimuth and elevation angles for the channels of our 3OA A-format array are given in Table 3.

Table 3: 3OA A-format array azimuth and elevation angles

	Azimuth (θ):	Elevation(φ):
<i>Cft</i> :	0°	56°
<i>Lst</i> :	90°	56°
<i>Rst</i> :	270°	56°
<i>Crt</i> :	180°	56°
<i>Cfm</i> :	0°	18°
<i>Lstm</i> :	90°	18°
<i>Rstm</i> :	270°	18°
<i>Crtm</i> :	180°	18°
<i>Lfbm</i> :	315°	-18°
<i>Rfbm</i> :	45°	-18°
<i>Lrbm</i> :	135°	-18°
<i>Rrbm</i> :	225°	-18°
<i>Lfb</i> :	315°	-56°
<i>Rfb</i> :	45°	-56°
<i>Lrb</i> :	135°	-56°
<i>Rrb</i> :	225°	-56°

From these values, and the above formula, we can calculate a complete binauralization matrix, which is shown in Table 4 in Appendix B. Note that filter cutoff values are cumulative across type, such that $LP(\theta) \cdot LP(\varphi)$ and $N(\theta) \cdot N(\varphi)$.

In our Max MSP implementation of this binauralization matrix, decibels are converted back to linear values to provide the main argument for the $\sim$ object, and milliseconds are

robustly converted to samples to provide the main argument for the `delay~` object. Finally, and importantly, because the above binauralization processes result in an overall level increase of 1.61 dB as determined by white noise testing, we add a commensurate boost on the stereo output to enable level-matching with the binaural output.

Quadraphonics

The quadraphonics decoder is not substantially different from the stereo decoder. All that is necessary is to duplicate and then invert all values of the stereo decoder around the x axis. This gives Table 5 in Appendix C.

Surround

It is difficult to make any overarching statement about what constitutes the most common surround format. However, what we do know is that 7.1.2 is the recommended minimum channel format for Dolby Atmos [88]. Furthermore, if we push this to 7.1.4, we can then easily derive lower channel formats such as 5.1.4 just by subtracting channels

To setup the geometry in the case of 7.1.4, we inscribe a cube in a unit sphere, with additional center and side channels equidistant between the lower front corner and the lower side corners. Once again, we take the inverse square of the vector distance between the tangent points of the GSB and each of the non-LFE channels of 7.1.4. Finally, we scale these values between 0.0 and 1.0 and place them into an attenuation matrix as given in Table 6 in Appendix D.

Ambisonics

In our setup, audio in Unreal is encoded into A-format 3OA. A-format is the default output format for ambisonics in that there is a one-to-one correspondence with loudspeaker channels. Thus there is no additional step necessary to output audio for 3OA playback systems. To downmix for a lower ambisonic order, only subtraction of channels is necessary, as given in

Table 7. If a B-format interchange file is desired, there is plenty of existing software for converting A- to B-format ambisonics [19],[20].

Table 7: Ambisonic channel orderings

30A:	20A:	10A:
Cft	Cft	Cft
Lst	Lst	-
Rst	Rst	-
Crt	Crt	Crt
Cftm	-	-
Lstm	-	-
Rstm	-	-
Crtm	-	-
Lfbm	-	-
Rfbm	-	-
Lrbm	-	-
Rrbm	-	-
Lfb	Lfb	Lfb
Rfb	Rfb	-
Lrb	Lrb	-
Rrb	Rrb	Rrb

User interface

Throughout the software for utilizing STAIRs, there are a number of user adjustable settings.

Within the Unreal project, the primary user adjustable setting to be aware of is the power of attenuation slope. This can be found on any sound-emitting entity to which the relevant Blueprint has been attached. It can be made to be different for each unique instantiation; however, the default setting is 1.5. A setting of 1 is linear, whereas a setting of 2 would correspond to a hyperbolic slope, which is the rate of falloff as dictated by the inverse square law for distance attenuation of intensity in acoustics. In testing, it was found that this hyperbolic power of attenuation slope was too precipitous. Hence why the default is set to be between a linear and hyperbolic falloff.

Within the STAIRProcessor Max patch, there are considerably more settings that can be dialed in by a user. The first and most significant is the decay size of the sloping function. By default, a size of .5 produces the model-dictated exponential decay. One may wish, though, to use more artificial decay sizes. As the size approaches zero, the slope of the curve asymptotically approaches 0. As the size approaches one, the slope of the curve asymptotically approaches 1.

The next group of settings pertains to the equalization of the input to the reverb. These settings can be switched between a cut and a notch configuration. Generally, a cut of around 50 Hz helps with some muddiness in the low end and therefore is the default of this setting. The high cut is left at its maximum.

It requires little comment, but it is worth noting that the dry and wet levels of all 16 channels of the 3OA audio can be adjusted.

Decoders can be selected with a radio dial, and binauralization can be enabled on stereo output. The default output channel numbering is as follows:

1. Stereo: 1, 2
2. Quad: 1, 2, 29, 30
3. 7.1.4 (L, C, R, Lr, Rr, Ls, Ls, Lh, Rh, Lhr, Rhr): 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11
4. 3OA: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16

If a user's particular system differs from these defaults, they can go into the subpatchers for the decoders and change the numbering in the respective `dac~` objects.

IV. HARDWARE DESIGN

Historically, the hardware for generating ambisonic (or at the very least omnidirectional) impulses and the hardware for recording the responses have been kept functionally separate. Examples of the hardware used to produce impulses for room acoustics measurements are the DS3 Dodecahedron Speaker by NTi Audio [89], the OmniPower Sound Source Type 4292-L by Hottinger Brüel & Kjaer [90], and the Odeon Omni by DTU Electro [91]. Any number of mic arrays for capturing the responses could be named, e.g., the ZYLIA ZM-1 by ZYLIA Sp. z o.o [33] or the em64 EigenMic by mh acoustics LLC [30]. In our case, we wish to consolidate ambisonic impulse response capture into one acoustigrammetric hardware apparatus we call the STAIRArray. While this apparatus has been designed primarily with the intent of capturing STAIRs, it is worth noting that other use cases are also possible (cf. VI. DISCUSSION, Applications). Focusing on the design of the STAIRArray, we will first give a broad level overview before getting into the particulars.

At the broadest level, the array consists of an inner sphere and an outer gyroelongated square bipyramid (GSB) frame. The inner sphere houses 8 full-range speaker drivers for 2nd-order ambisonic impulse generation, as well as compute, AD/DA, and connectivity hardware. On the outer GSB frame are mounted 16 digital MEMS microphones for recording 30A audio. The array is subject to extensive shock absorption. Each MEMS microphone board is suspended by elastic bands, and the inner sphere is suspended vertically by heavy springs and horizontally by elastic bands. It is conceivable that the STAIRArray could be moved through a space through multiple means, for instance a pulley system, a robot, or manually by a person. For now, the method employed is that of a boom pole, with additional spring suspension in vertical and horizontal directions to maintain axial stability of orientation.

Physical layout

As noted in the earlier discussion of algorithms for evenly distributing points on a sphere, the GSB is the only known regular-face convex polyhedron with 16 faces. This is why it is chosen as the shape for the body of the STAIRArray.

The physical structure of the STAIRArray is that composed of 16 equal frame faces (CAD render pictured in Figure 14). These faces suspend the MEMS boards between elastic bands, and the faces are connected together into the overall shape with hook and loop ties.

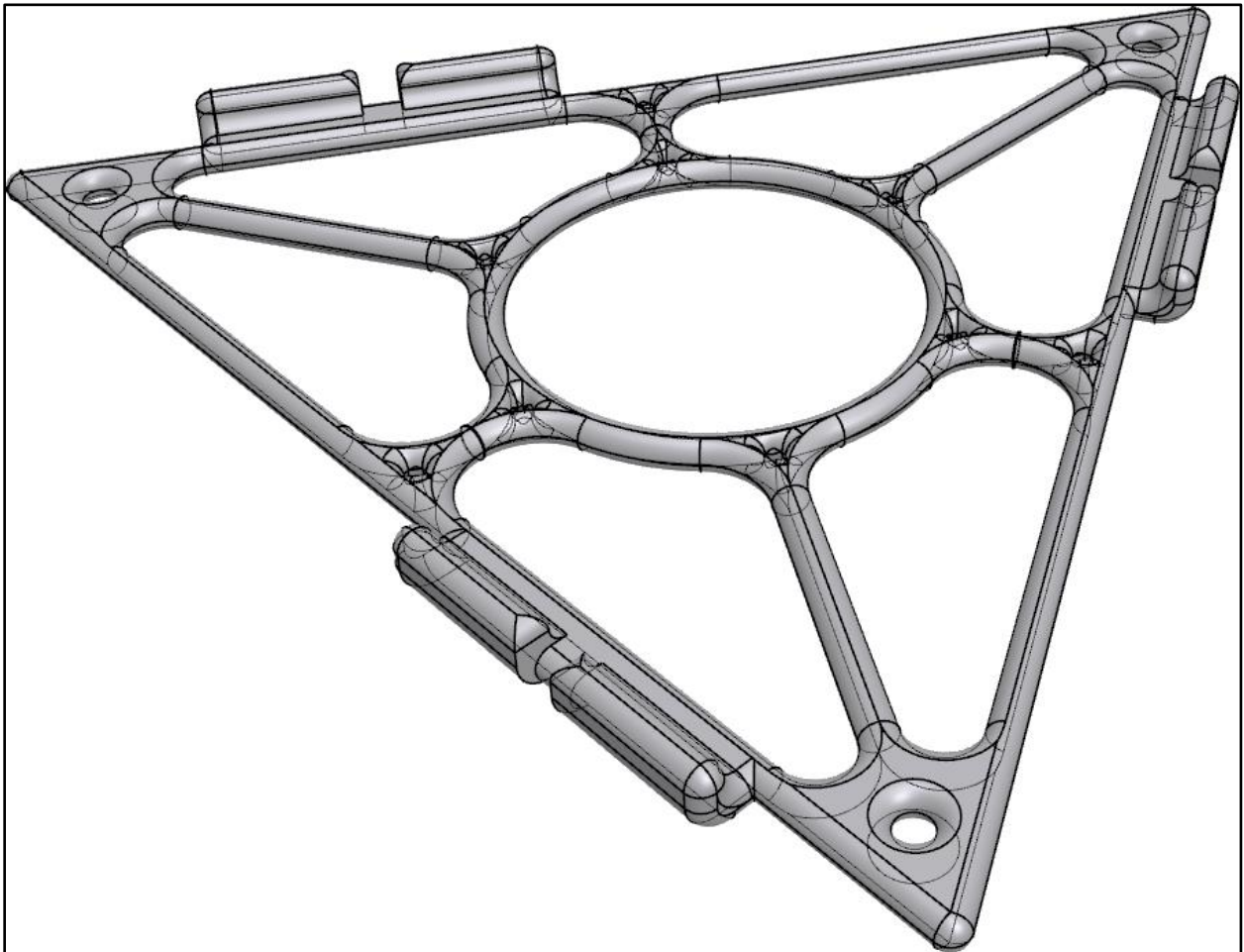


Figure 14: Frame face CAD drawing

Inside of the GSB formed by these faces is a sphere which contains the speaker drivers and compute platform. Waveguides for the speakers are designed into the sphere (see Figures 15 and 16 for CAD renders).

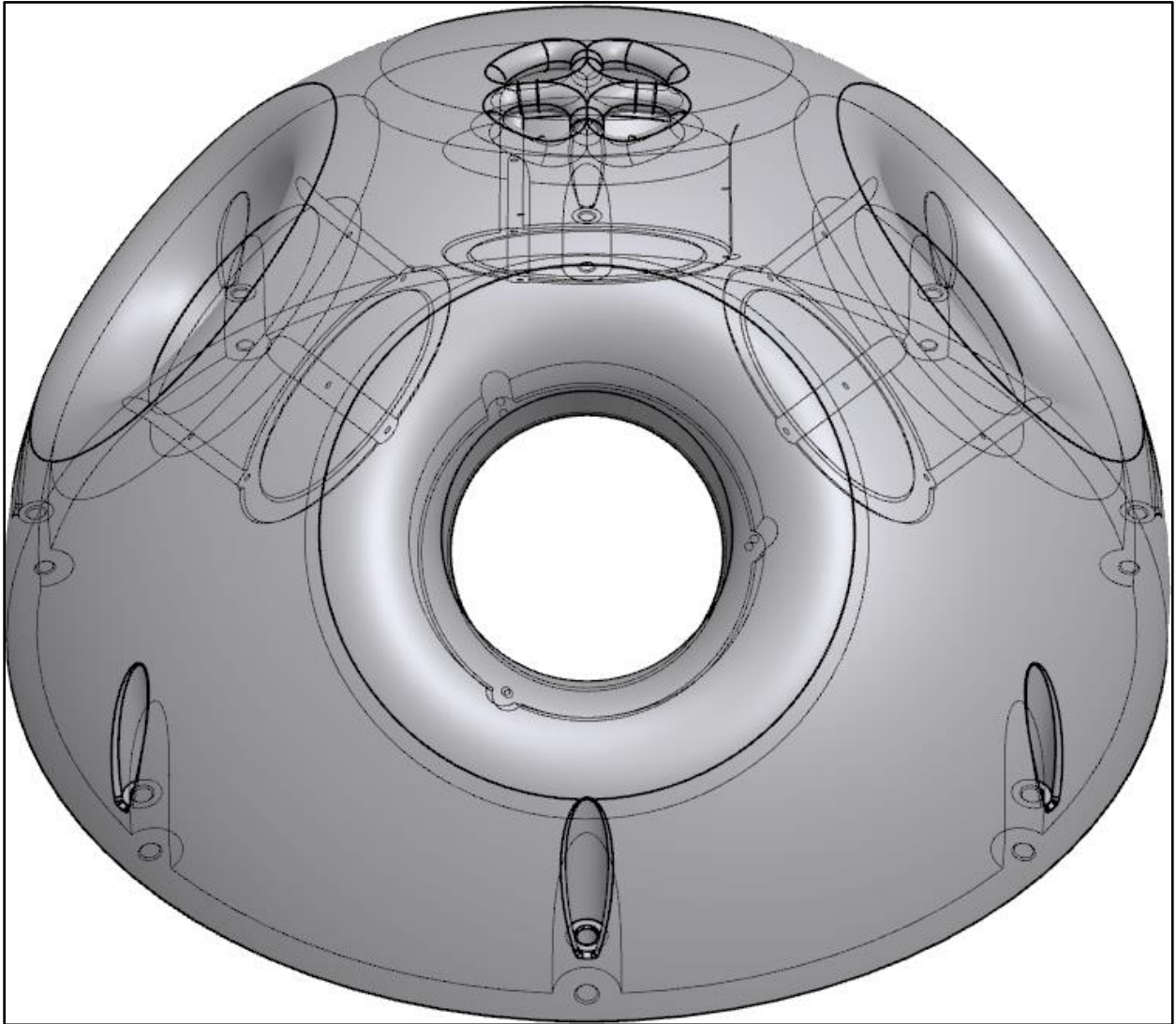


Figure 15: Fitted dome CAD drawing

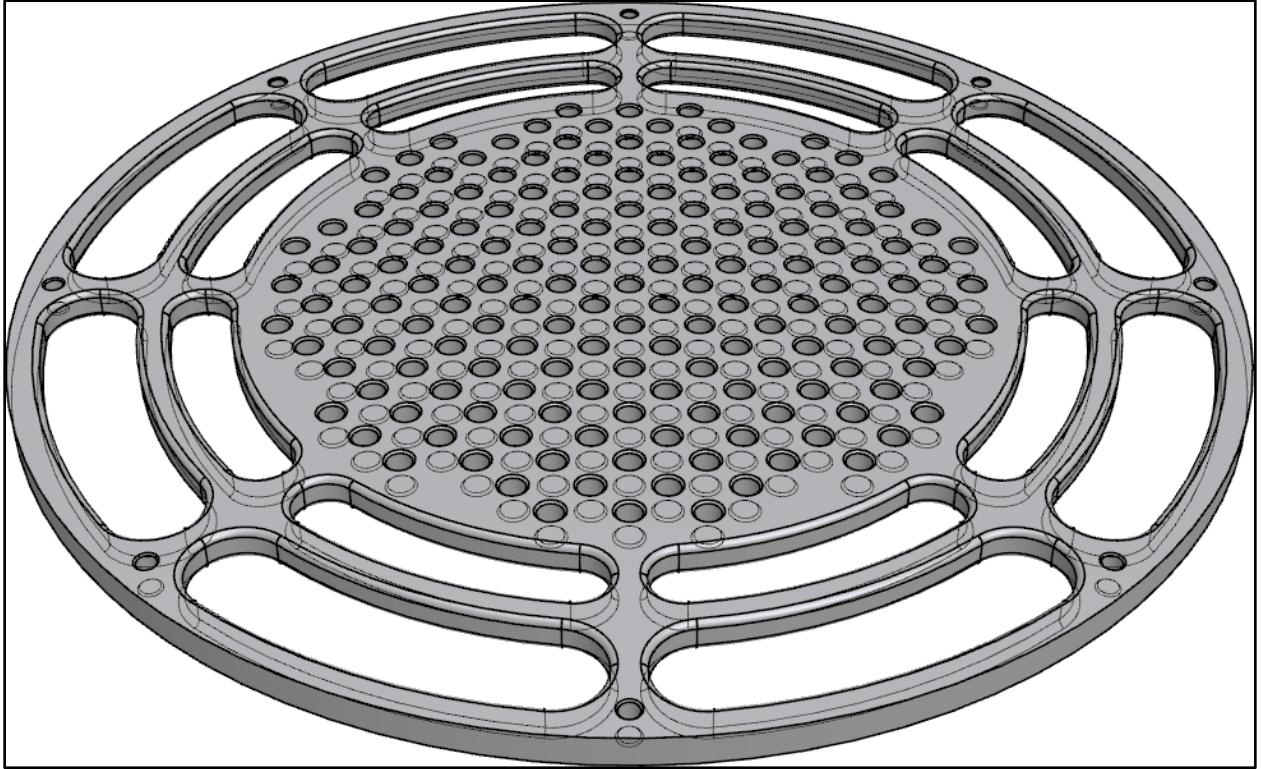


Figure 16: Compute platform CAD drawing

Through hook and loop ties, elastic and spring suspension, and paracord linkages, these component shapes are brought together into the complete STAIRArray form (CAD render pictured in Figure 17).

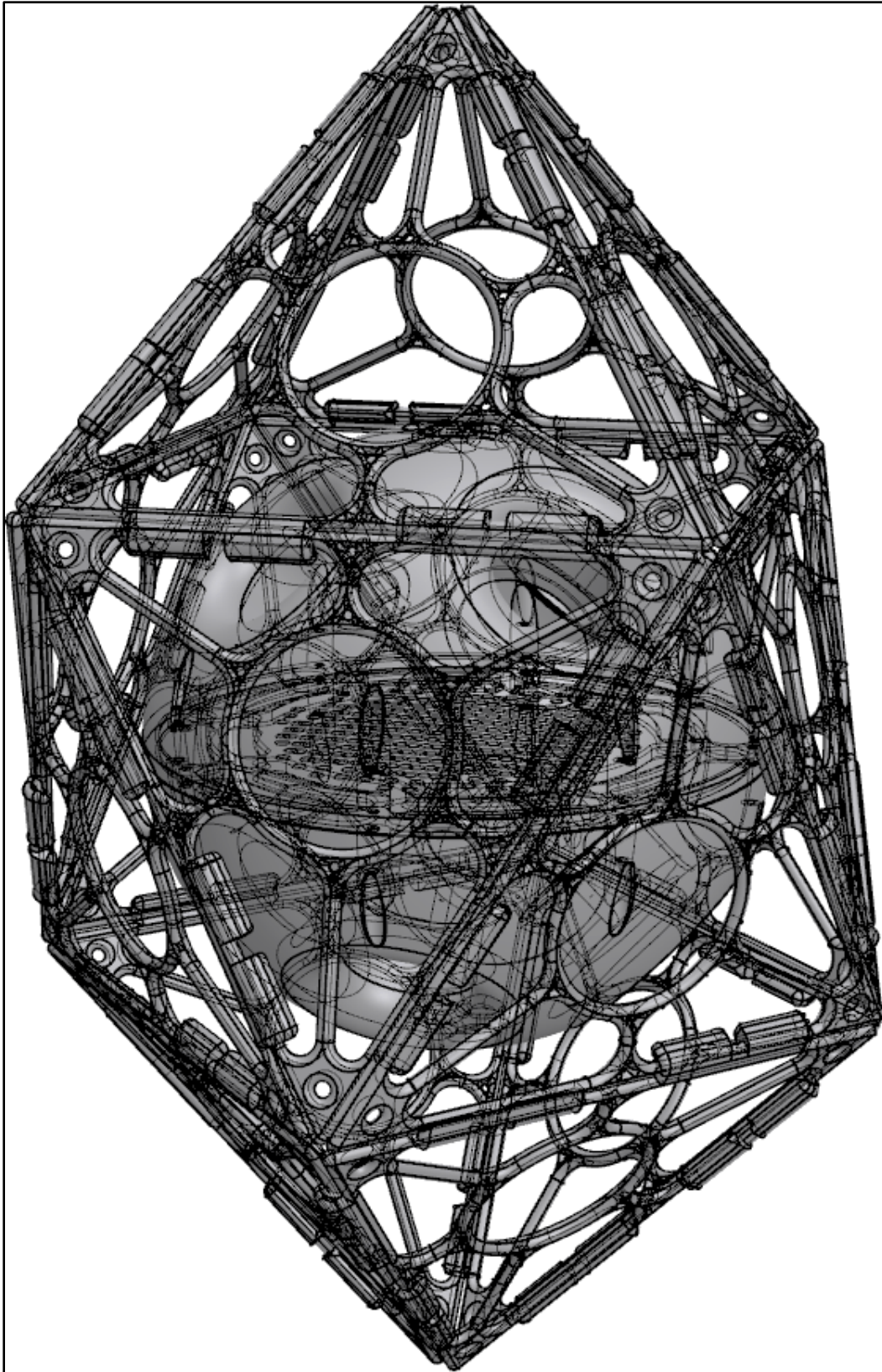


Figure 17: Combined STAIRArray CAD drawings

The cross-sectional dimension for any component is chosen to be <20 cm for the reason that many 3D printers max out at this print area. One upshot is that the STAIRArray is not inordinately larger than the average human head, perhaps aiding in binaural decoding.

Parts & Materials

Substantial electronics parts will be discussed in more detail below. As to materials, we note that the STL files generated for this design were printed with carbon-fiber reinforced acrylonitrile styrene acrylate filament. This has good tensile strength, immunity to acoustic resonance, and environmental durability [92]. The actual prints and disposition of parts can be pictured in the following series of figures.

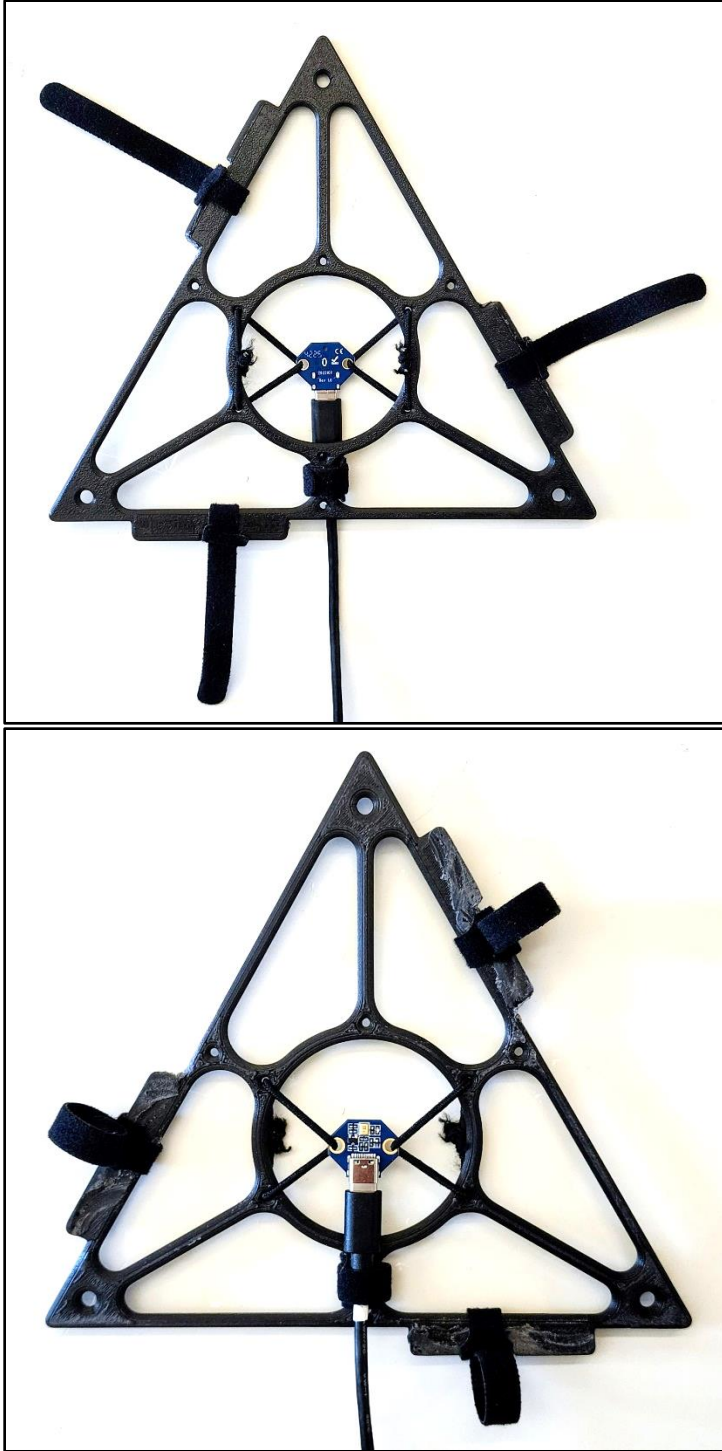


Figure 18: Frame faces, with shock-mounted MEMS boards, USB cable, and hook and loop ties; front view (top), reverse view (bottom)

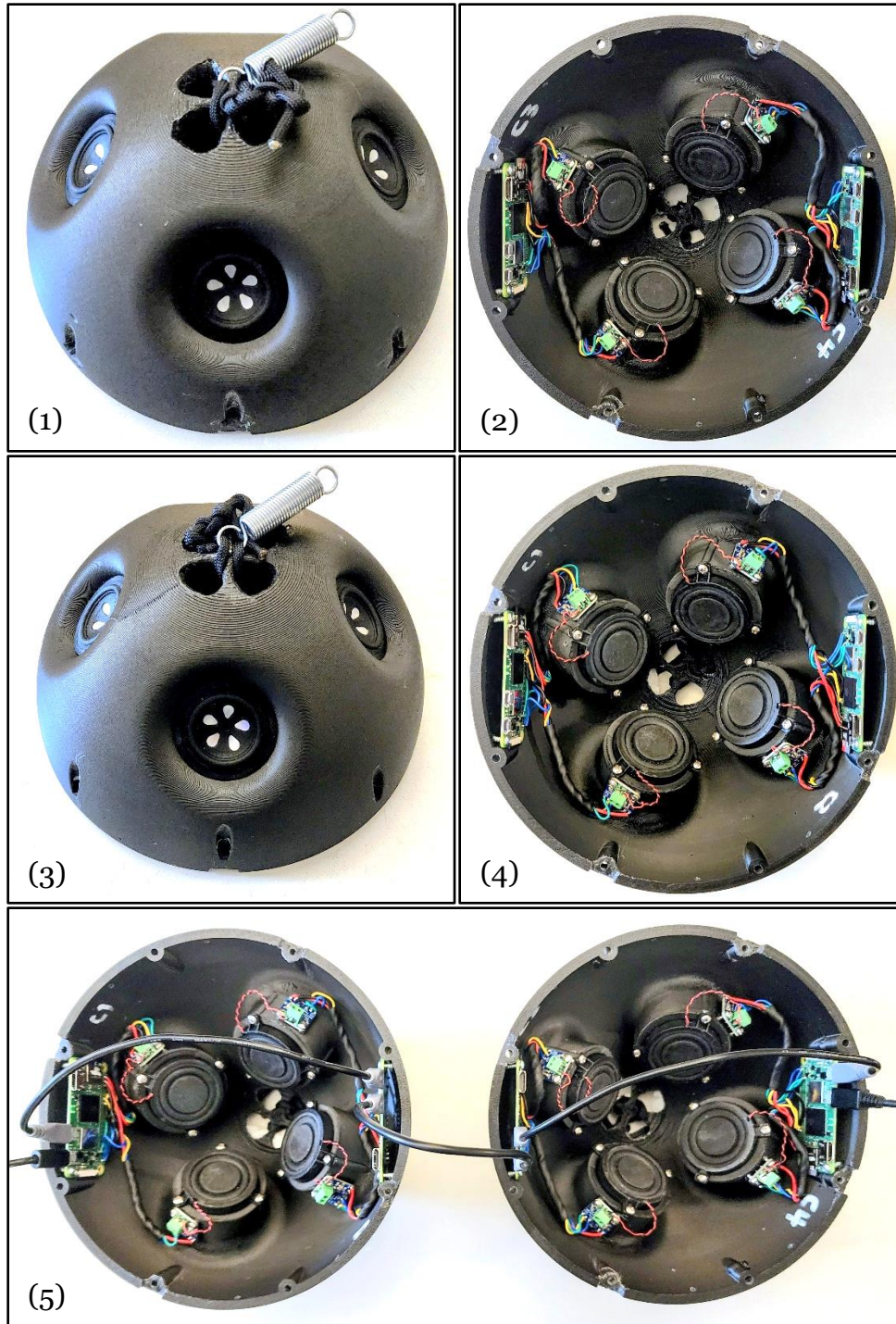


Figure 19: Fitted domes, with speaker drivers, DAC-amp boards, Raspberry Pi Zero 2Ws, power and I2S wiring, USB cables, and upper and lower spring dampers; (1) upper dome, exterior view, (2) upper dome, interior view, (3) lower dome, exterior view, (4) lower dome, interior view, (5) linked domes

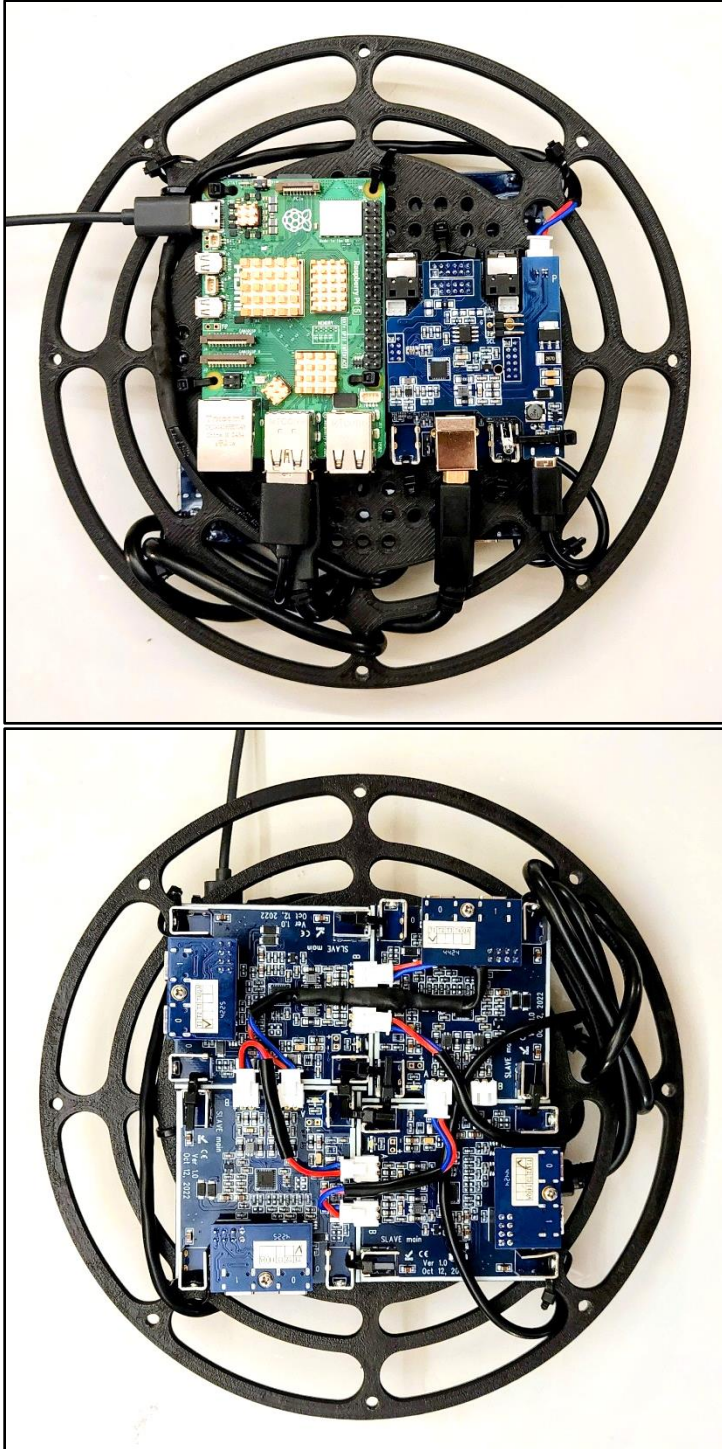


Figure 20: Compute platform, with Raspberry Pi 5, A2B interface boards, A2B cables, and USB cables; top view (top), bottom view (bottom)

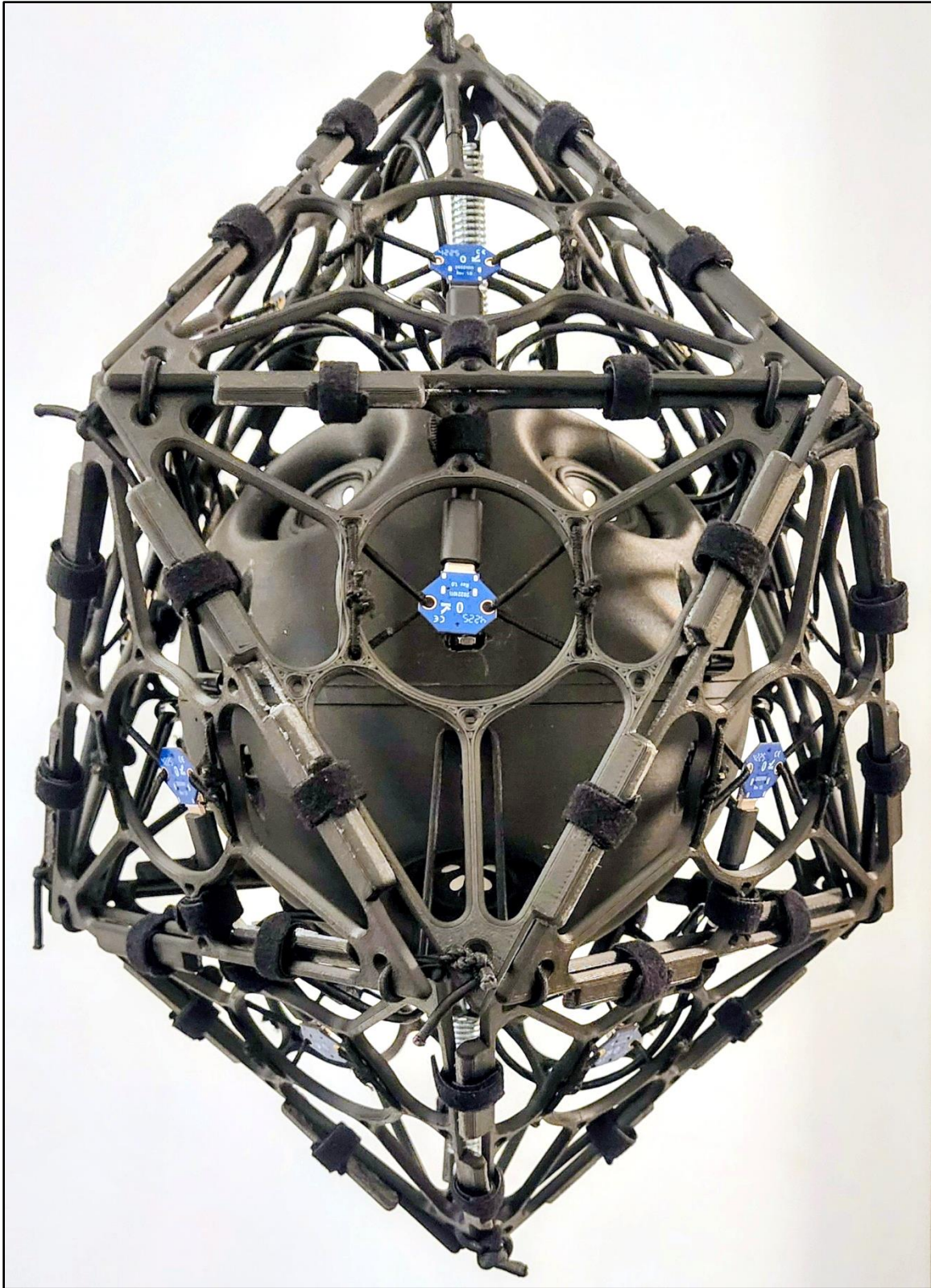


Figure 21: Assembled STAIRArray

Microphones

The MEMS microphones utilized in this array are part of the UMA-XL system by miniDSP [34]. These are built around the ubiquitous Knowles MEMS mic. Like most MEMS mics, they are omnidirectional. They have a remarkably flat response with little variance between 400 Hz and 20 kHz and resist distortion up to relatively high SPLs (130 dB SPL). Each mic is matched for uniform response, which is particularly important in an ambisonic setting. This mic system was specifically designed for room acoustics. Up to 16 channels can be linked together by A2B boards. To minimize weight and maximize space in the STAIRArray, the A2B boards have been removed from their aluminum housings.

Speakers

As with microphones, speaker selection is important in terms of the frequencies that may be represented. In our case, we are especially restricted by size. As such, we want a speaker driver of between 25 to 55 mm that is capable of delivering a reasonably full-spectrum response. Having witnessed a prototype phased array at the Center for Computer Research in Music and Acoustics that used Tang Band speaker modules, that company's catalog of offerings came recommended. Ultimately, the T1-2025SC sound module was selected [93]. The voice coil has a diameter of 25.4 mm. The reported frequency response is from 95 Hz to 20 kHz. Looking at the reported spectrograph, we note an approximately 20 dB falloff from 200 to 20 Hz. To compensate for this while maintaining sufficient headroom for the overall signal, a gentle low-shelf boost of 5 dB starting at 200 Hz and a high-shelf cut of 2.5 dB starting at 5 kHz was printed on the noise impulse used for STAIR capture.

ISO Directivity deviation

We have an arrangement of 8 speakers in a 2nd-order outwardly facing array. The goal is to saturate the sound field with the noise impulse as uniformly as possible in all directions. The ISO specifies two standards (ISO 140 & ISO 3382-1) for directivity deviation for a sound source that qualifies as omnidirectional in acoustic testing [94],[95]. ISO 140 is intended for building acoustics, and ISO 3382-1 is intended for room acoustics and is the more stringent of the two standards. We use ISO 3382-1, which in turn references ISO 140-3. Accordingly, Table 8 gives the allowed deviation in specific frequency bands (extended ranges are optional), along with our measured deviations.

Table 8: Directivity deviation

1/3-octave band center frequency:	Max deviation (approx.):	Measured deviation:
<i>100 Hz:</i>	6 dB	5.94 dB
<i>125 – 250 Hz:</i>	5 dB	4.46 dB
<i>315 – 500 Hz:</i>	4 dB	4.68 dB
<i>630 – 1000 Hz:</i>	3 dB	2.42 dB
<i>1250 – 3150 Hz:</i>	2 dB	4.11 dB

These numbers seem acceptable across most bands, with the only perceptible deviation being in the highest band.

DAC-amp boards

Providing the 8 audio streams to the 8 speakers is a non-negligible design issue. If all we intended to do was to feed the 8 speakers the same noise impulse response, it might be sufficient to take a standard stereo line or speaker output and amplify that to the levels needed for the speakers. However, to get the highest fidelity and to future proof the STAIRArray for other possible applications, such as ambisonic reamplification and virtual acoustipresence, we want each speaker to be independently addressable. Thus we need a unique digital-to-analog output for each audio stream. The method we have opted for is to use the I2S audio protocol to transmit

each audio stream to a combined DAC-amp board where the signal is then decoded and amplified for each speaker. The DAC-amp board we have used is the MAX98357A [96]. This is capable of up to a 96 kHz sample rate and of outputting 3.2 W at 4 Ω .

An additional complexity arises due to the fact that a single Raspberry Pi board exposes only one data line on its GPIO [97]. As a consequence, a single Raspberry Pi board can only output one stereo audio stream. One option would be to send this to a TDM time multiplexing board which could decode multiple interlaced streams. Designs for such boards do exist [98]; however, they are expensive and not widely available. Furthermore, software conflicts between using USB audio for the mic input and I2S audio for the speaker output would likely arise. A cheaper and more readily reliable option is to use 4 Raspberry Pi Zero 2Ws for the audio output. Each one outputs a stereo I2S stream, the two channels of which are then split to feed two DAC-amp chips for a total of the 8 independent audio streams we need.

Microcomputers

A total of 5 Raspberry Pis—1 Raspberry Pi 5 and 4 Raspberry Pi Zero 2Ws—drive this whole electronic system. The Pi 5 serves as the parent computer, handling the heaviest computing processes of 16-channel recording and clock synchronization; and the Pi Zeros serve as the children, running a minimal Pure Data patch for audio playback. Thus the 16 mics are recorded into an Ardour session on the Pi 5, and the speakers are fed by the Pi Zeros. All Pis are linked together on the same local network established by a smartphone hotspot. Each Pi can be remoted into over VNC. But to run a recording session, it is only necessary to remote into the Pi 5. A Lua script within the core Ardour session on the Pi 5 sends out a message over TCP to the Pi Zeros to trigger synchronized playback within a Pure Data patch..

Connectivity

Throughout the STAIRArray, we employ 5 connectivity solutions:

1. USB power
2. USB audio
3. A2B audio
4. I2S audio
5. Local Wi-Fi network

The rationality and interactions between these connectivity solutions will now be explained.

Firstly, all power is supplied over USB by a power brick capable of outputting 4.8 A (5 A is the target). Both the Pi 5 and the UMA-XL mic system are on the same rail. The Pi Zeros are all daisy-chained off of a separate rail.

Secondly, the UMA-XL mic system forms a class compliant USB audio interface which connects over USB to the Pi 5

Thirdly, the UMA-XL mic system connects via A2B (Automotive Audio Bus technology developed by Analog Devices) [98] to 4 daisy-chained A2B breakout boxes, each of which in turn feeds 4 MEMS mic modules via USB connections.

Fourthly, each of the 4 Pi Zeros transmits audio via I2S wiring (power, ground, DIN, BCLK, and LRCLK) to pairs of the MAX98357A DAC-amp boards which are connected by analog unbalanced wiring to the speaker drivers.

Fifthly, all Raspberry Pis need to connect via Wi-Fi to the same hotspot, in our case established by an Android smartphone. This local network allows us to establish a remote VNC connection with any of the Pis for troubleshooting and for control of the recording session. It also synchronizes playback of audio on the 4 Pi Zeros by way of a TCP connection.

The diagram below (Figure 22) illustrates the different connectivity solutions of the STAIRArray.

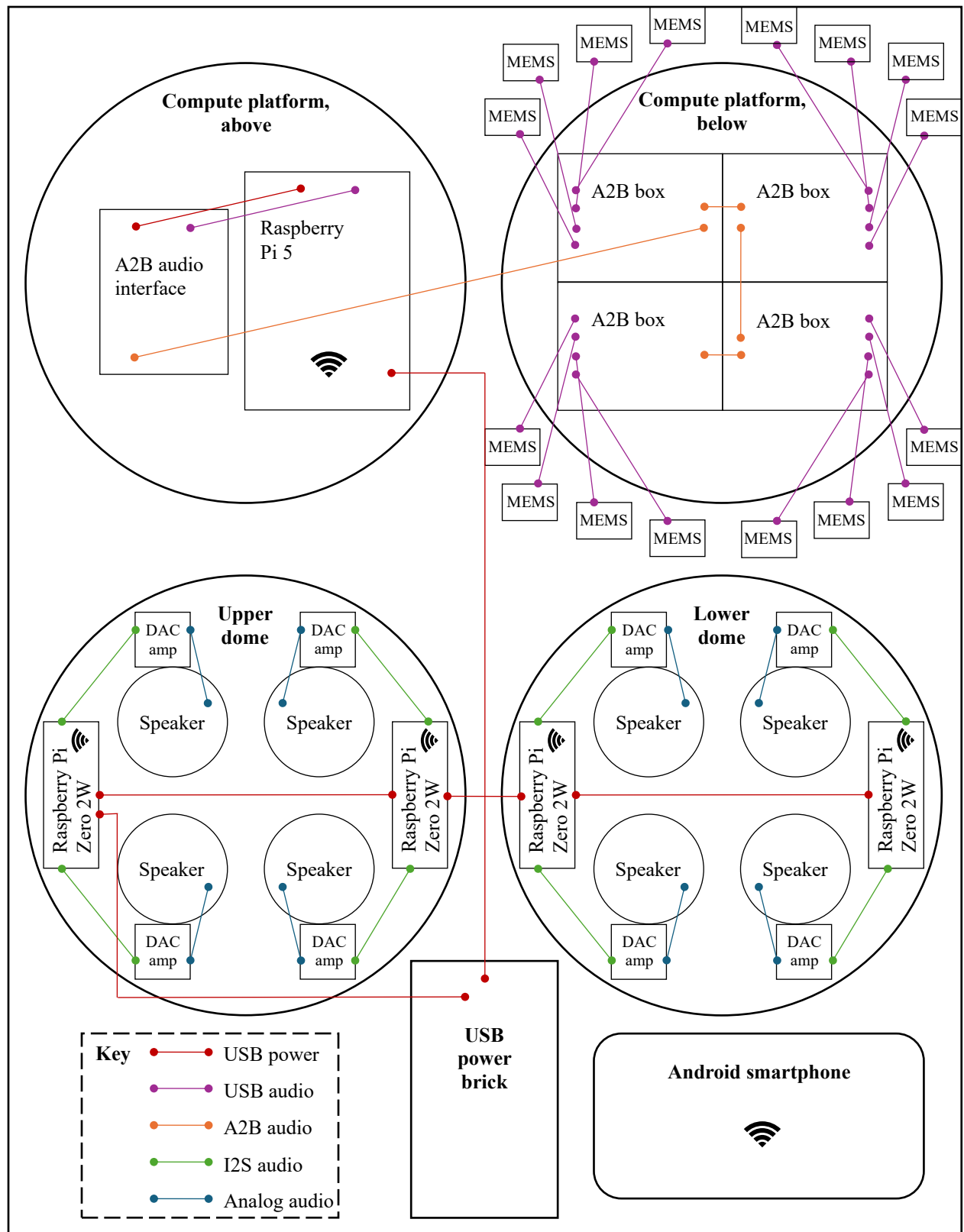


Figure 22: STAIRArray connectivity

Setup protocols

Extensive setup and tuning of the Raspberry Pis is necessary. It would be overly cumbersome to describe all of the scripting necessary here. Some of the details can be found on the GitHub repository for this research. Nevertheless, we will describe in broad strokes what is being done.

On the Pi 5, Ardour needs to be installed, and we need it to load our template recording session on boot. This Ardour session also contains a Lua action script that targets the IP addresses of the 4 Pi Zeros. The Pi Zeros have Pure Data installed on them with a patch that receives TCP messages to open and play a noise impulse recording. This patch is setup to autoload on boot. The Pi Zeros also have I2S protocols installed on them. All Pis are setup to maintain a local network and to allow for VNC. A host of changes were made to the firmware of the Pis to tune power distribution. Recordings are saved on the Pi 5 and then uploaded to a server to access elsewhere.

GitHub repository

A GitHub repository for documenting, sharing, and enabling independent work on all aspects of this project has been created. It can be found at:

<https://github.com/ALLETHIC/Acoustigrammetry>. The structure of the repository is as follows:

1. Software
 - a. STAIRcpp - Unreal template project
 - b. sharedmem~ - Max external
 - c. STAIRProcessor - Max patch
2. Hardware
 - a. STAIR Array STLs - design files for 3D printing the STAIR array
 - b. Hardware BOM - bill of materials for building the STAIR array

c. Raspberry Pi setup - documentation for setting up Raspberry Pis for ambisonic recording & playback

3. Misc. files

a. Unreal demo projects

b. STAIR recordings

c. Visual media (graphics, photos, and videos documenting the project)

Here are some basic tips for how to use the software files found in the repository:

1. Place `sharedmem~` external in default Max MSP library
2. Open Unreal game
3. Open STAIRProcessor Max MSP patch
4. In the STAIRProcessor, select whether you want stereo, quad, 7.1.4, or ambisonic output
5. If necessary, change output numbering in `dac~` objects in stereo, etc. subpatches to match your system
6. If desired, binauralization can be enabled on stereo output

The Unreal template project contains all the constructs necessary for working with STAIRs in an Unreal project. Here is a breakdown of the essential constructs included:

1. *A first-person character with a ListenersSphere and an SFLSphere.* The ListenersSphere is parent to 16 unit spheres corresponding to the 16 channels of 3rd-order A-format ambisonics. The ListenersSphere fully tracks the movement of the first-person character. The SFLSphere is similar to the ListenersSphere but differs in 2 important ways. 1) It only tracks the translational movement of the first-person character. 2) Each unit sphere is parent to a special Metasound patch

known as the Spectral Frame Looper (SFL). This loops a spectral frame of a STAIR channel depending on the first-person character's distance along a spline.

2. *3rd-order ambisonic encoding*. Any sound emitting entity has attached to it a patch which sends sound based upon its distance to the 16 child spheres of the ListenersSphere to 16 corresponding submixes. The send amount can be adjusted between a more hyperbolic or more linear curve. A hyperbolic curve matches the inverse square law of acoustics. However, subjective testing revealed this falloff to be too precipitous. So a linear curve may be more desirable.
3. *Submix shared-memory sender effect*. A C++ class is used to place both listener submixes and SFL submixes into shared memory to be read in Max MSP where audio DSP and different output configurations can more easily be iterated.

To use the core constructs in a new project, simply migrate all of the content of the template project to the new project (the test recordings can be omitted if desired or replaced later). Also, make sure to place the SubmixEffectSharedMemoryPreset C++ Class in the C++ Classes folder of the new project (assuming it has one). Then, in the new project, under project settings, set the Default GameMode to BP_FirstPersonGameMode and the Default Pawn Class to BP_FirstPersonCharacter_30ASpheres, and make the Master Submix Custom_Master_Submix.

Next draw the SFLSpline through the STAIR path you want to take in your scene. Finally, attach the EventGraph of the Example_Sound_Emitter Blueprint to any sound source you wish to encode into 3rd-order ambisonics and change the audio component node to match your sound source.

All of that said, given some of the complexities of migrating the content of an Unreal C++ project to another Unreal project, the easiest option is to just duplicate the STAIRcpp project from the get-go and build from there.

V. FINDINGS

The following chapter documents the entire process of recording STAIRs and then utilizing them in game projects. Observations about what worked at every stage are recounted. Stereo and binaural auralizations of each space are recorded and discussed in terms of their subjective impact. On the other hand, objective data from spectrograms are analyzed. Some comparisons are made between real and synthetic STAIRs. Finally, computer and hardware performance are reported.

Case Study #1: SyntheticSTAIR

This first case describes a synthetic STAIR that was used extensively to test the software in early stages. Because not all people may have access to a STAIRArray hardware setup, being able to work with a synthetically generated stair can be useful. It can also help establish a baseline with which to compare real-world STAIRs.

This synthetic STAIR is generated by using the Envelop for Live B-format Convolution Reverb with the York Minster Cathedral IR. Size, decay, and position are linearly automated over one minute while our same noise impulse is run through it. One could think of this as simulating the movement of a listener from one smaller to one larger side of a cathedral-type space. A virtual environment, replete with sound sources to excite the virtual acoustics, is pictured in Figure 23. The spline mapping for this environment is fairly straightforward; it just stretches in a straight line from the smaller to the larger wall.

Two screen recordings of walkthroughs of the virtual space can be found at the following links:

[https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Playthroughs/SyntheticSTAIR\(stereo\).mp4](https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Playthroughs/SyntheticSTAIR(stereo).mp4) (stereo);

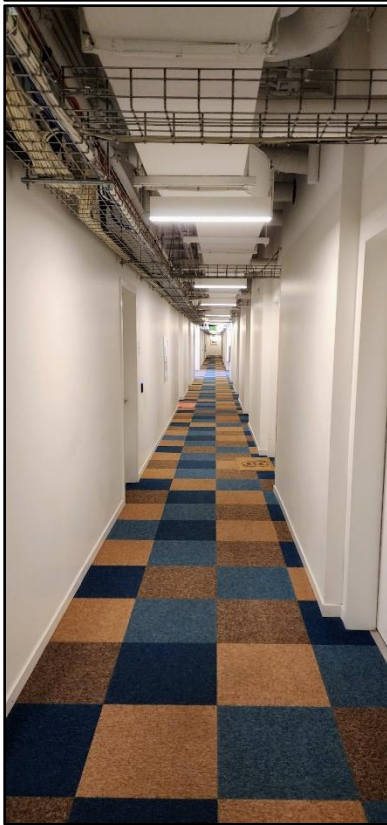
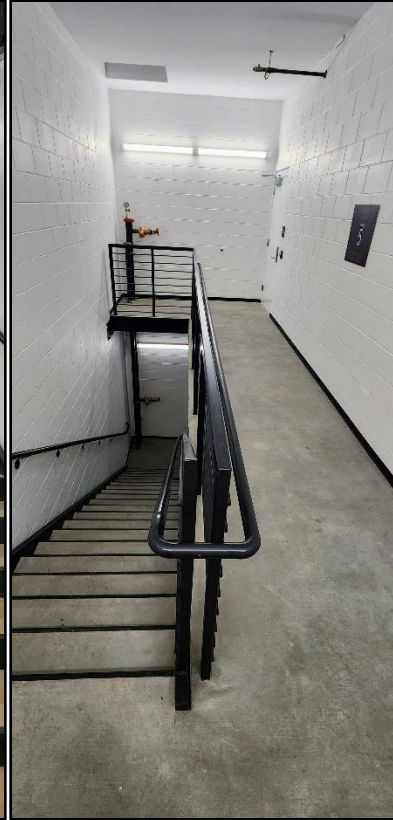
[https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Playthroughs/SyntheticSTAIR\(binaural\).mp4](https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Playthroughs/SyntheticSTAIR(binaural).mp4) (binaural).



Figure 23: SyntheticSTAIR virtual environment

Case Study #2: WaysSTAIR

For this site testing, we intentionally choose a meandering series of passageways. Such spaces cannot be modelled with common shoebox or hall geometries. They are inherently multivolume. This site includes two concrete stairwells, one that is 5 floors high and the other that is 6 floors high. Between them is a carpeted and windowed hallway that takes a right angle turn at half its length. The actual site is shown in the following pictures.



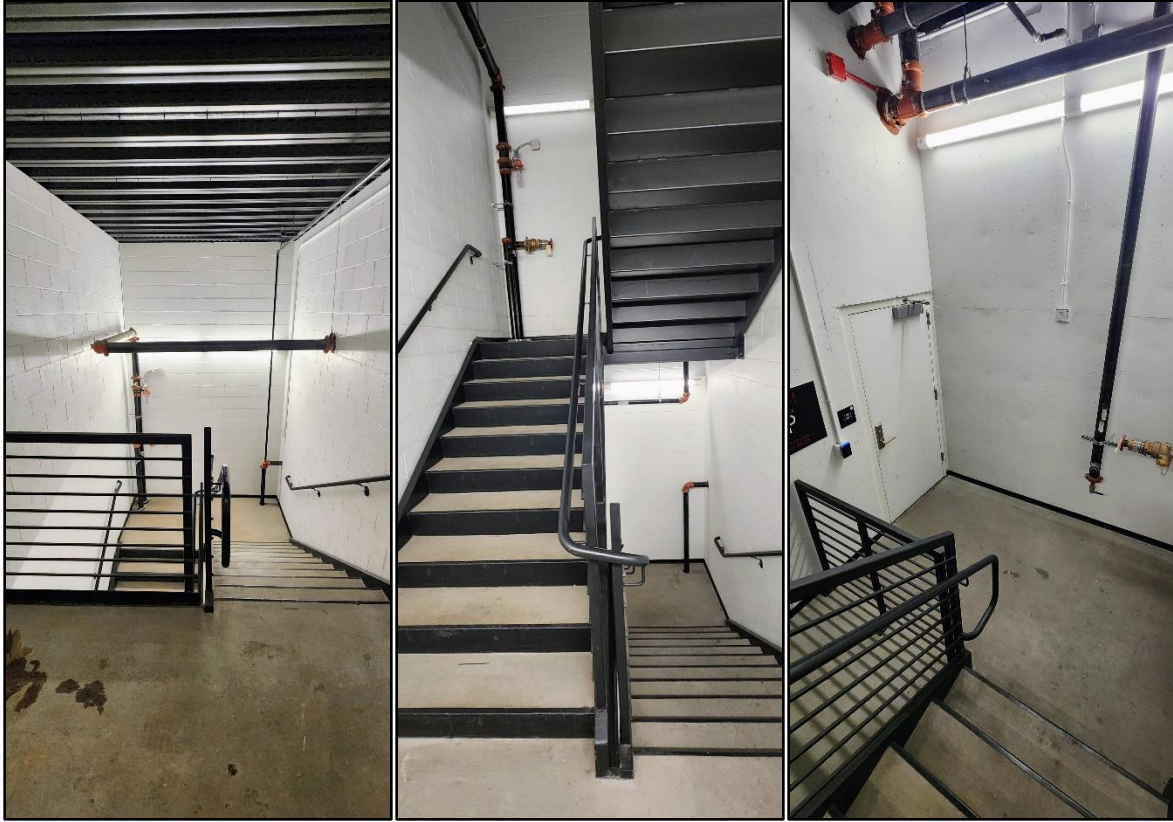


Figure 24: Photos of residential passageways

The Unreal project created to imitate this site is called WaysSTAIRcpp. It can be found on the GitHub repository for this project. The appearance and layout of the virtual site is captured in the following screenshots and birds-eye view.

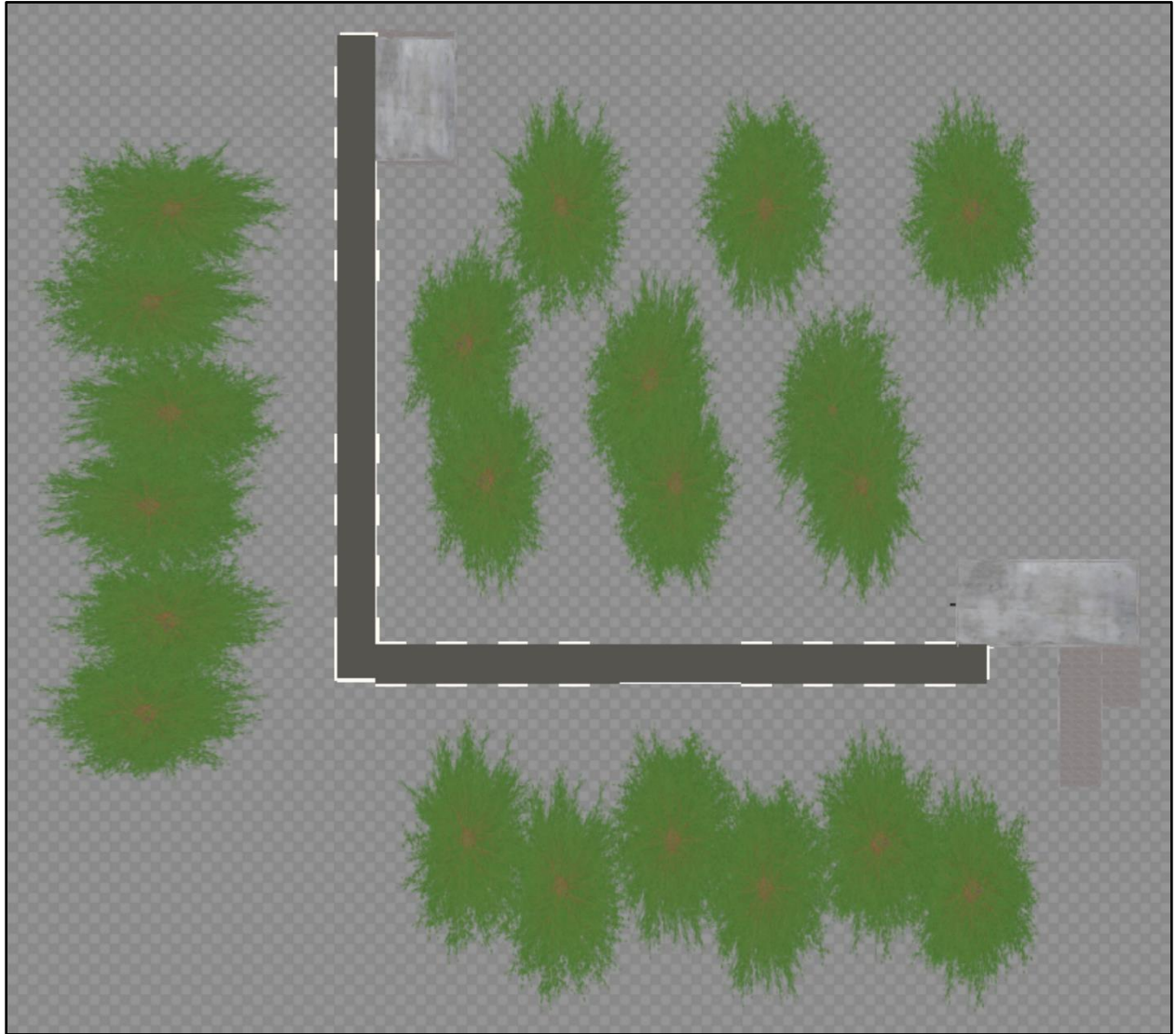


Figure 25: WaysSTAIRcpp project screenshots

Figure 26 shows the spline mapping through the virtual space. This mirrors the path that was taken in capturing the STAIR through the actual space.

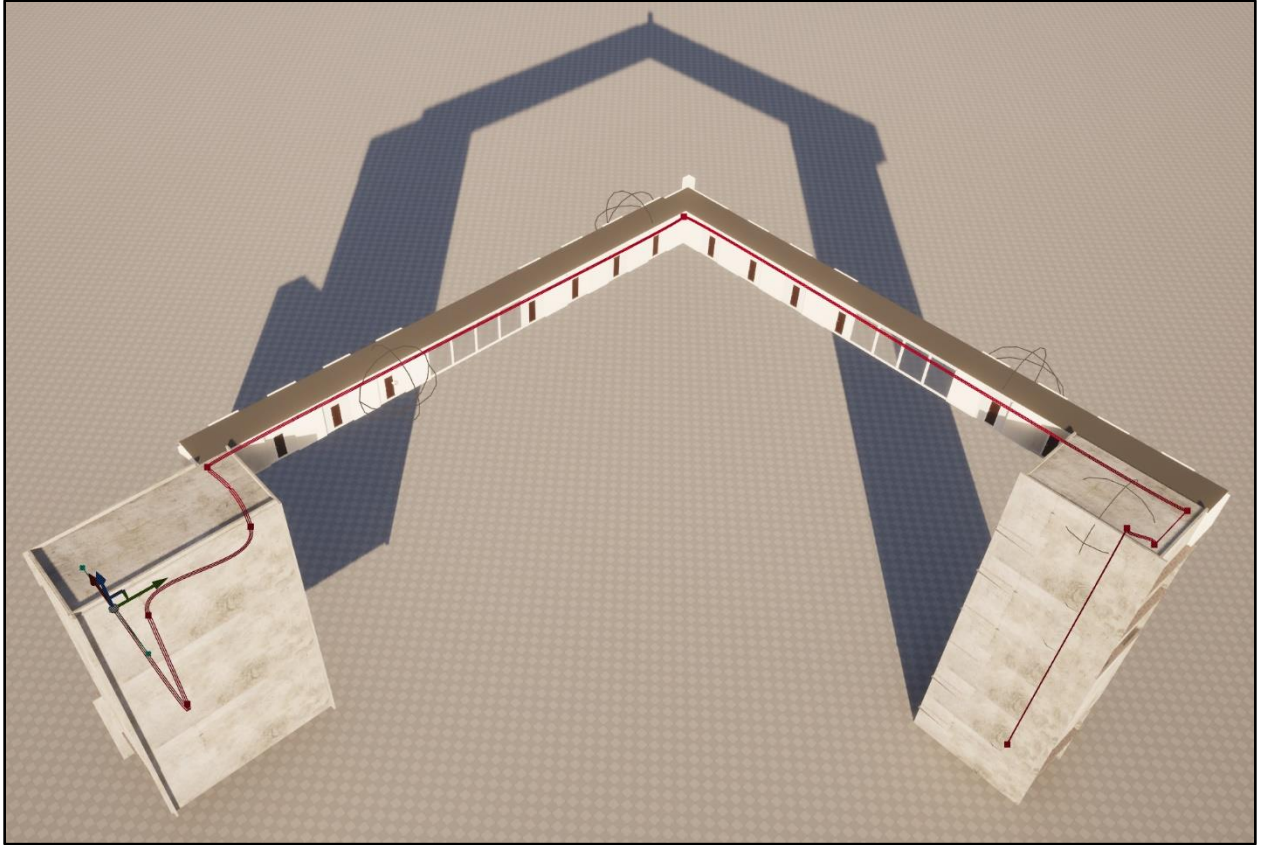


Figure 26: WaysSTAIR spline mapping

The virtual space is populated with sound sources chosen to mimic a typical residential building and to broadly stimulate the STAIR. It is possible to hear a dog barking, people partying in one apartment, a person practicing the sax, and HVAC sounds. Figure 27 shows screenshots of the virtual space.

Two screen recordings of walkthroughs of the virtual space can be found at the following links:

[https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Playthroughs/WaysSTAIR\(stereo\).mp4](https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Playthroughs/WaysSTAIR(stereo).mp4) (stereo);

[https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Playthroughs/WaysSTAIR\(binaural\).mp4](https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Playthroughs/WaysSTAIR(binaural).mp4) (binaural).

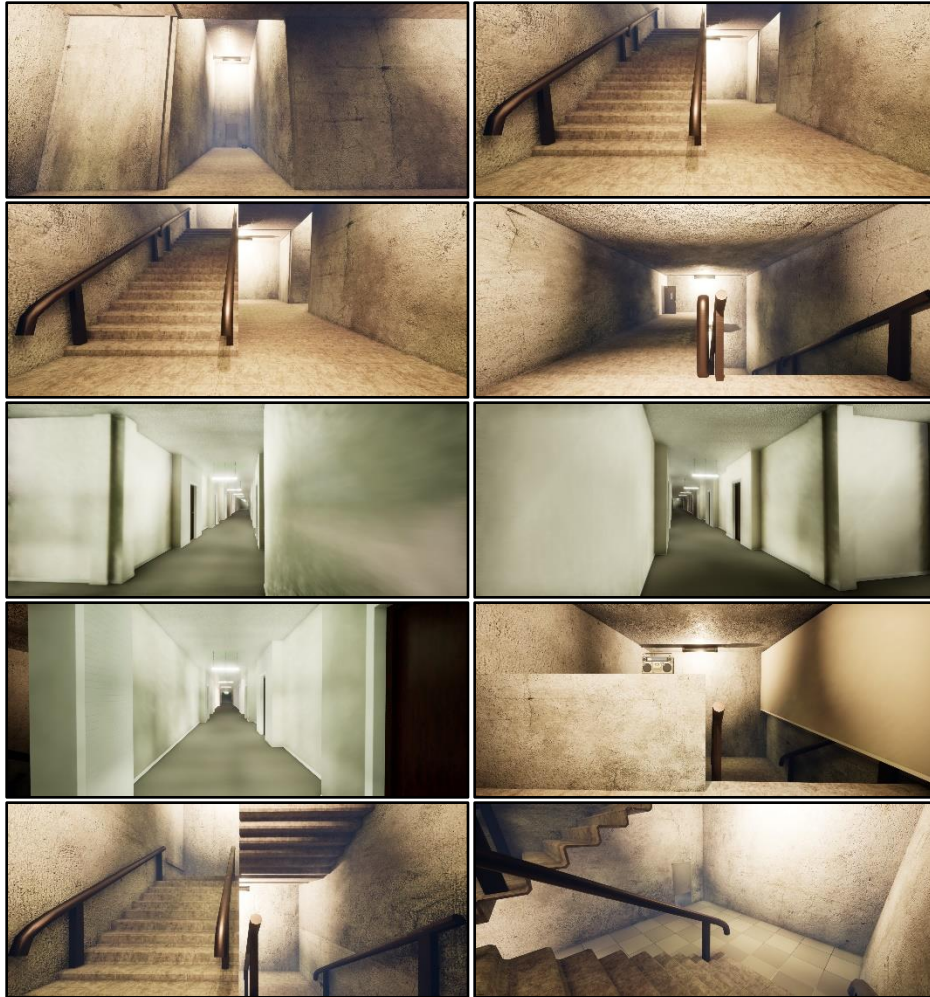


Figure 27: WaysSTAIR virtual environment

Comments on verisimilitude

Our initial discussion of the findings from the two cases above will focus on the sonic verisimilitude of the results. In both cases, we can report that the audio is devoid of glaring artifacts (though the process of screen recording them for the walkthroughs mentioned above may have produced a few clicks and pops due to screen recording being an additional strain on CPU resources). Generally speaking, the DSP algorithm produces a richly and smoothly enveloping reverb effect as one would expect from a convolutional ambisonic reverb. Spatial

transitions along the spline are clean. Compared to traditional reverb zones, there are no noticeable transition points.

More to the point, different positions along the STAIR yield distinct differences in the acoustics. One can trace this in the spectrograms for the SyntheticSTAIR and WaysSTAIR pictured in Figures 28 and 29 in Appendix G. In the case of the SyntheticSTAIR, one can see the spectra become more diffuse as the space grows larger, which is precisely what one would expect. This same dynamic is audible in the walkthrough of the virtual space. Indeed, the main change one can hear over the SyntheticSTAIR path is the move from the smaller to the larger side of the space.

In the case of the WaysSTAIR, we can see far more complicated interference patterns across the spectra. This is as we might expect from a real-world STAIR over a synthetic one. The primary thing one can hear in the WaysSTAIR and see in its spectrogram is the difference between the two stairwells and the carpeted hallways. The smaller of the two stairwells has a reverb decay of ~ 1 seconds, and the larger of the two stairwells has a reverb decay of ~ 1.25 seconds. Whereas the carpeted hallway has a reverb decay of ~ 0.285 seconds. In consequence of this fact, in the virtual implementation of the reverb, one can hear the decay become more damped as one transitions from the stairwells to the hallway. Furthermore, in the walkthrough, the reverb seems to be the longest at the bottom of the stairwells.

Comments on practicality

Having talked about the most important aspect of the STAIR technology, namely of whether it is capable of seamlessly producing dynamically navigable virtual reverberation, we will now move on to talk about some other aspects of its performance, beginning with the hardware used.

Hardware performance

One of the main concerns, naturally, for the STAIRArray was how well it would excite and capture the acoustics of a space. Both the mics and the speakers performed well, as can be evidenced by a brief recording exhibiting the tails of two STAIRs

(<https://github.com/ALLETHIC/Acoustigrammetry/raw/refs/heads/main/Audio%20Files/STAIRs/WaysSTAIR/Stairwell1&Stairwell2.wav>).

The spectrograms for the WaysSTAIR show some banding artifacts in the upper registers which may be the result of microphone self-noise or suboptimal cabling. Subsequently, efforts were made to clean up the cabling before running the final directivity deviation calculations reported in Table 8 of the hardware design section. This data showed some evening out of the directivity deviation over a previous test. It is conceivable that with this and additional tidying of the cabling that some of this banding in the spectrograms could be reduced.

The substantial efforts put into the design of the shock absorption seem to have paid off, as handling noise is negligible. While the STAIRArray weighs upwards of 10 lbs, it was possible to move it through an extended space without excessive difficulty. However, fatigue could genuinely become an issue in other scenarios. This is in tension with some need to increase the size of the STAIRArray to better deal with cabling and the containment of a power supply. With additional research on 3D printing materials, this tension could perhaps be mitigated. Further on the issue of moveability, given that the STAIRArray is currently being moved with a boom mic pole, some concern remains regarding acoustic shadowing from the operator.

Software performance

In the process of using STAIRs with our software pipeline, some useability findings were revealed.

The first to note is with regard to CPU load. Unreal games are known to be relatively heavy. On its own, a Windows build of either of our games on a PC with 64 GB of RAM, an 11th Gen Intel Core i7, and an NVIDIA GeForce RTX 3090 GPU consumed 50 – 60% of the CPU. The STAIRProcessor Max MSP patch, running at 48 kHz and a buffer of 1024, consumed another 30 – 40% of the CPU. Thus at times we were hitting the upper ceiling of the CPU and dropouts in the audio were audible. However, shifting from an audio sample rate of 48 kHz to 44.1 kHz cleared up the situation considerably, with CPU load diminishing by about 20 – 30% and audio dropouts disappearing. This resulted in smooth audio with a still good level of fidelity.

The next consideration is with regard to binauralization. While generic binauralization resulted in a greater sense of spatialization on headphones, the lack of customized head-related transfer functions is a limiting factor in terms of producing the most realistic immersion possible.

Continuing, it was found that power of attenuation slopes need to be hand tailored depending on the sound source. For loud and constant sounds, closer to the naturalistic slope of 2 could work. For gentler, more intermittent sounds, the slope could drop to as low as 1.2. There are no doubt important psychoacoustic factors that dictate which slope corresponds best to expectations, but a deeper consideration of these factors will have to wait for future work.

Another finding that came from limited initial user testing was that localization of sound sources could be improved upon. Alternately, localization of reverb tails seemed to be satisfactory. This gave rise to the thought of expanding the radius of the ListenerSphere by a factor of 2 while keeping the radius of the SFLSphere the same. This is akin to extending the lobes of the virtual microphones further so that there is a greater distance between the probe tips of each lobe and thus the differentiation in submix sends is greater. This change seems to have resulted in better localization of sound sources and is incorporated into the listening examples

and the software versions available on GitHub. The question stands as to how far out one might optimally extend the lobes. For now, the conservative factor of 2 seemed prudent.

The last finding with regard to the software concerns how to accurately map a STAIR onto a space by way of a spline. For instance, one does not want to accidentally map a stairwell portion of the STAIR onto the hallway portion. The main aim is that points along the length of the spline correspond to the correct points along the STAIR. In the case of the SyntheticSTAIR, this was a trivial matter because the STAIR was automated and the spline was a straight path. In more meandering paths, and particularly if the STAIR is broken up and then stitched back together (as was the case on both fronts with the WaysSTAIR), the risk of mapping the STAIR onto the wrong point of the spline goes up. One approach is to keep spline paths as simple as possible even if the sampling path was more complicated. Splines need only map distance through a space and not the actual route taken. The other aspect of this approach, then, is to maintain close measurements of the distances along a spline. One wants to maintain the same proportion between segments of the spline as between segments of the STAIR. Ultimately, the best approach to achieving the most accurate mapping of a STAIR onto a space is to log the GPS path taken in recording the STAIR and then import that with it into the game engine.

VI. DISCUSSION

In this final section, we will discuss two overarching matters. The first concerns how STAIR technology can be refined and extended through future work. And the second concerns practical applications for the technology, from the apparent (game audio and VR) to the less apparent (acoustipresence and creative installations).

Future Work

There is a wide scope for the continued refinement and extension of the underlying technology. Naturally, these breakdown into two complementary paths: revisions of the software and revisions of the hardware.

Reducing theoretical intermediaries

As noted in the background section of this document, early reflections are more difficult to model than late reflections. This is no doubt part of why the common-slope model only aims to reproduce the late reverb of multivolume spaces. All else being equal, differently sized volumes are primarily anisotropic with respect to late reverb. However, all is not always equal. Different regions in a multivolume space may not only differ dimensionally but also in terms of geometries and materials. As such, early reverb can contribute important perceptual cues about different regions within a space. Ultimately, therefore, we want to be able to bring early reflections back into the equation. Two proposals for future work answering this problem will now be discussed.

Building the most immediately on the implementation of the common-slope model in our software design, we propose using spectrally weighted probabilistic gating of the first 50 – 100 ms of a signal. The precise mixing time could also be established from analysis of discrete SRIRs along the spline path, and then a simple curve fitting algorithm could interpolate mixing

times for the whole path. Continuing on, we take the amplitudes of the spectral bins of a frame of a STAIR, and then we translate these amplitudes to probability weights. This corresponds to the acoustic intuition that the more intense a frequency is in the STAIR, the more likely it is to have arrived early from a surface with a high reflectivity coefficient. We then take this weight and probabilistically gate whether a spectral bin of the post-processed source sound will be let through. Thus we derive a semblance of sparsely random early reflections from the empirical frequency response data. The advantage of this approach is that it works on top of the architecture we have already developed.

The second proposal would require a more radical augmentation of the software architecture. In this proposal, a STAIR is cut into two parts at the point corresponding to the user's position along the spline. Then one part, likely the shorter part, though this is largely arbitrary, would be reversed. After the first 50 – 100 ms (as with the preceding proposal, precise mixing times could be analytically derived), a dynamic compression gradient of each spectral bin begins. The gradient has two dimensions: ratio of compression and overall attenuation. Each is exponential, but the exponent of the ratio is generally set to be larger than the exponent of overall attenuation (though the exact proportion is not known at this stage of the research). The idea of folding the STAIR is that acoustic energy is not only sampled for a particular point but for all points in time along a translational path. One can likely derive an even more precise modelling if quantities like the physical length of the sampling path and rate of movement along the sampling path are known. Finally, the idea of the dual exponential decay curves is that early amplitude spikes—corresponding to early reflections—are initially allowed through before being clamped down and shaped into the later decay stage.

As is typical, the resultant IRs of the partitioned STAIR are convolved with the source sound. Then the outputs of the two convolutional chains are halved in amplitude and layered together. Figure 30 illustrates the whole proposal.

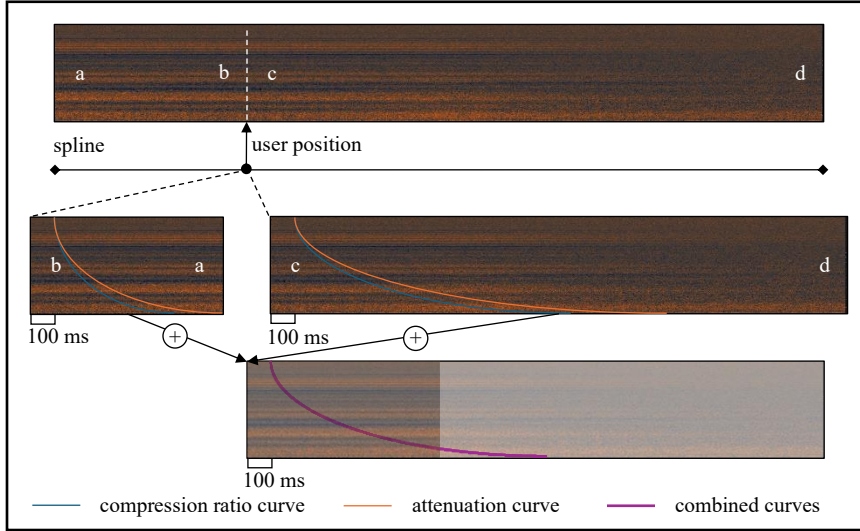


Figure 30: Folded and shaped STAIR

One problem with this approach is that processing demands would likely require that the STAIRs be preprocessed in some fashion. Done naively, this would create a dataset of x SRIRs, where $x = \text{sample-rate} \cdot \text{STAIR}_{\text{length}}$. Furthermore, if a 3 minute long 3OA STAIR was recorded at 48 kHz and 24 bits, and the average SRIR length is 1 second, then this would create a dataset of ~19 TB! One solution to these extreme memory needs would be to simply drop our translational spatial accuracy by some factor. Another solution would be to use primed preprocessing. Segments would only be preprocessed and then loaded and unloaded depending on their proximity to the user's current position. This of course trades hard drive space for RAM and CPU power. An additional problem remains of how to interpolate between SRIRs in our proposed system. One possibility would be to recapitulate the methodology of perceptual interpolation between SRIRs in SOFA datasets discussed earlier in the section on 6DoF. As we can see though, taken together, all of these components would place significant additional

burdens on computational resources. The upshot, on the other hand, would be one of the most complete virtual acoustic models presently feasible.

Occlusion

Because our model is not an algorithmic model such as a wave-solver or a ray-tracing model, it is not intrinsically responsive to the geometry of the mesh in which it finds itself. Thus it does not bake occlusion paths into the model itself. That said, future work in which occlusion is hard-coded into the software using collision and line-of-sight or ray-casting methods seems viable.

Packageability

Currently, our project makes use of Max MSP. This exists outside of the game engine. There were several reasons for this. The first was that Unreal internally throttles the audio thread more than external software such as Max MSP. The second was that Max MSP makes it easier to assemble and iterate upon the audio DSP. And the third was that Max MSP makes audio hardware outputs addressable in a way that Unreal does not.

When one wants to run a game with our virtual acoustic sound system, one first opens up the game and then the Max application. Ideally, however, we would want to package everything that is in Max inside the game deliverable. There are two ways we could do this. We could simply author the C++ classes necessary. Some earlier efforts to do this did not yield stellar audio results. Furthermore, C++ classes have a tendency to break between projects. To get around this latter problem in particular, the second way we could package the audio processing would be as an Unreal plugin. Max MSP's RNBO export would be one way we could compile the C++ code for an Unreal plugin. However, this would impose a licensing fee, should that be a future issue. Furthermore, there are other strategies for translating Max patches into C++.

Bringing the audio processing into Unreal could also reduce any latency that is the results of shared-memory sending.

Spatial transforms

Due to the well-understood mathematical theory underpinning ambisonics, there exist what are known as a standard set of ambisonic transformations [18]. These transformations are what allow us to do familiar things like rotate an ambisonic sound field. There is a further set of transformations that allows for the distortion of the sound field. For instance, the energy of a spherical sound field can be transformed so that it is more lobate in one or more directions. With these standard transformations, we could imagine a software that allows for morphing a STAIR as needed.

Two types of transformations could be envisaged. A proximity transformation could shift the energy of a STAIR as a user becomes closer to a mesh surface that is orthogonal to the spline path. An angular transformation could take a STAIR and bend it. For instance, a STAIR captured in a straight tunnel could be bent so that it more accurately models the acoustics of a bent tunnel.

STAIR reference library

Only one real-world site was formally tested for this research. For at least two major reasons, it is incumbent that more sites be tested. The first reason is epistemological; that is, we do not know if there are types of sites that might reveal deficiencies in the STAIR approach. The second is practical. If STAIRs are to be useful, we need a significant and diverse set of them that can be applied to different virtual scenarios. A distinct avenue of research could be devoted to developing such a STAIR reference library. As part of that work, it could be useful to develop (no doubt based on the past work of other researchers) a taxonomy of the topologies and

geometries of different acoustic spaces. Are some serpentine? Are some intersecting or nonlinear? Are some locally nonuniform but globally uniform? And so on.

Synthetic STAIR generation

For those who do not have access to a STAIR hardware array, it remains possible to synthetically generate STAIRs. All it takes is an ambisonic reverb plugin with settings that can be automated. As noted in the chapter on FOSS ambisonic reverberation, there exist a number of such free and open-source reverbs. In the GitHub repository accompanying this thesis, there is a demo project called “SyntheticSTAIR”. To produce the synthetic STAIR in that project, white noise was run through an ambisonic reverb while size, decay, and position were automated over time. This simulates moving from the smaller vertical wall to the larger vertical wall of a trapezoidal prism. It is conceivable that using these settings, as well as those afforded by other ambisonic reverbs, that an infinite array of spatial configurations could be programmatically generated. Indeed, it merits researching techniques for generating more topologically and texturally complex STAIRs.

Multi-spline mappings

Along with the development of STAIR reference libraries and a relevant geometric taxonomy is the problem of transitioning between multiple splines in a scene. It is possible to envision two ways in which this might arise.

First, even in fairly linear spaces, there may be the desire to jump between two STAIRs. Say STAIR_1 has been captured for one stretch of a space and then, for whatever reason, STAIR_2 has been captured for another contiguous stretch of the space. This may be done for either practical or methodological reasons. In any event, the question arises, How should one best transition between these two splines? The simplest and not unreasonable approach might be

to directly stitch together the two STAIRs. Or, at the very least, one might have the splines corresponding to the two STAIRs directly abut and then STAIR_1 is muted immediately when one is closer to a point on the spline of STAIR_2 than to the spline of STAIR_1 while STAIR_2 is simultaneously unmuted.

However, second to this, an even more complicated problem can arise in increasingly non-linear spaces. Say you have two STAIR splines crisscrossing. At which point should one STAIR prevail over the other? In a certain sense, it is arbitrary which STAIR should prevail at the exact point of intersection, as, theoretically, the spectral frame should be the same for the two STAIRs at that point. Then the problem is how to mute or unmute one STAIR as the user approaches the point of intersection. Perhaps the same algorithm as for two linear STAIRs applies. Mute or unmute whichever the user is closer to. The only additional condition we need add is which STAIR we should default to at the point of intersection. A kind of hierarchical algorithm would be necessary in the case of many intersections. Suffice it to say, this remains to be explored further in future work.

Microphone housing & polar pattern

The microphones utilized in this project are typical MEMS microphones as we find in smartphones and many other applications. Microphones of this kind are almost always omnidirectional out of the package. On the face of it, it may seem harder to shape the polar pattern of a MEMS microphone than a traditional larger format microphone. However, there is fascinating research inspired by the hearing system of the *Ormia ochracea* fly suggesting otherwise [99]. The *Ormia ochracea* fly is parasitic to crickets. It is able to directionally home in on the sound of a cricket's mating call in a field. It has one of the most directional hearing systems known in the animal world. Essentially, what the attendant research shows is that it is

possible to significantly shape the directionality of MEMS mics with biomechanically inspired housings.

Omnidirectional microphones may be desirable for most ambisonic arrays because they simplify the math involved. In our case, though, it may be desirable to reduce the amount of direct sound from the centrally mounted speaker array by positioning it within the nulls of more directional polar patterns. It could be worthwhile to test different microphone housing designs to see if they do indeed reduce the amount of direct sound and result in cleaner deconvolutions of the noise impulse from the response recording.

Improved moveability

Besides a large set of small-scale hardware improvements, such as scaling up the size of the STLs slightly, improving power distribution, and exploring different 3D print media (such as SLA and SLS), one of the most significant alterations to the hardware would be in the way of moveability. Initially, we have opted for the most humanly hands-on approach to moving the STAIRArray, namely with a boom mic pole. The next step might be to use pulley systems. Some early trial and error showed the difficulty of stabilizing pulley systems, and so they were abandoned. Nevertheless, cablecam systems used in broadcast and film show that this should be solvable [100]. In a similar vein, dolly systems could also be used. In the end, we want a system that is the quietest and that involves the least acoustic shadowing from the operator. Robots have been used to move microphone arrays [101]. Although somewhat limited by payload size, drones also deserve mention. In that case, the issue would be additional deconvolution of the drone noise from the recording.

Two final hardware augmentations pertaining to moveability are worth considering. While significant effort was put into axially stabilizing the STAIRArray and minimizing shock

deflections by way of springs and elastic bands, there is only so much that physical stabilization can do. The addition of a gyro-accelerometer could add software stabilization on top of this. Data from this sensor could be recorded and then used to compensate for rotations of the sound field. Also in the way of spatial tracking, a GPS module could have great benefits. A GPS module could help auto-generate splines and thus increase the accuracy and simplicity with which sampled STAIR paths are mapped onto virtual spaces. As such, GPS tracking of sampling paths could greatly aid revised DSP algorithms as discussed above.

Applications

Rounding out our whole discussion, this section will proceed from the more readily apparent applications of STAIR technology to games and virtual reality and conclude with more exogenous applications such as virtual acoustipresence and creative installations.

Games & virtual reality

The application of STAIRs to game and VR audio has been implicit throughout this text. All that is left is to demonstrate is that it is worthwhile to go to the trouble of applying STAIRs in games and VR. A 2022 study in *Virtual Reality* touches upon this very topic in the specific use cases of Unreal and Wwise. It strives to recreate sound propagation physics in both Unreal and Wwise. It then compares the results to third-party software specifically designed to simulate room acoustics, e.g., Odeon. It finds that Unreal's throttling of the computational resources of the audio thread is a limiting factor. Wwise provides enhancements over Unreal, primarily in its handling of RIRs in the virtual realm. Nevertheless, both fall behind Odeon in terms of the completeness of their simulations. Add to this the fact that a bevy of research shows more realistic virtual acoustics increases presence and immersion and improves perceptual cues in

gameplay [102]–[105], and the argument for extending the virtual acoustic resources of games and VR grows all the stronger.

AEC

A range of software tools is used by the architecture, engineering, and construction (AEC) industry to model, visualize, and, in some cases, auralize acoustic energy in a space. CATT-Acoustic by CATT [106], Cadna by DataKustik [107], Ease by AFMG [108], Odeon by DTU Electro [109], SoundPLAN by SoundPLAN GmbH[110], and Treble by Treble Technologies [111] are some of the main software tools used in the AEC industry. Most of these software tools use ray tracing and image-source methods for modelling room acoustics.

Similarly, it has become more common to use game engines in the AEC industry [112]. In fact, architecture has been one of the earliest adopters of VR. It has proved useful in allowing designers and clients to get closer to a “tactile” sense of an architectural model.

STAIRs are well-suited to provide both visualization and auralization of acoustic energy in an architectural setting. Auralization is achieved essentially in the same way as virtual acoustics is achieved in games. The more unique case is in the visualization of acoustic energy. The spectral intensity of a STAIR along a spline could be used to drive the colorization and opacity of particle systems or volumetric clouds and thereby procedurally generate a navigable visual representation of the acoustic energy in a space. Another practice could be to capture multiple spatio-temporal ambisonic recordings over the construction of a building. First, before a building is constructed, a spatio-temporal ambisonic recording could be captured on a site to gain a comprehensive image of environmental noise. Then, at various stages, additional spatio-temporal ambisonic recordings could be captured, eventually with impulse response layers also

captured. In sum, this could be used to study how noise abatement strategies are working and how environmental noise interacts with room acoustics.

Virtual production

Game engines such as Unreal have become a core component of the virtual production pipeline for film and television. Game engines are primarily used to create 3D visual models that can be interacted with in real-time. For instance, on a television show such as *The Mandalorian* (one of the first shows to popularize the technology), actors are filmed in a space known as “the Volume” [113]. The Volume at bottom is a curved LED screen. Sometimes it fully surrounds the subject matter. Other times it is just a background wall. One of the important aspects of the Volume is that it synchronizes 3D visuals in a game engine with the tracked motion of the camera, thereby maintaining realistic parallax. While the visuals projected in the Volume are sometimes used as is in the final product, they are often replaced by CGI in post-production. In both cases, their benefit is that they provide realistic environmental lighting and facilitate “informed intent” on set, that is, an awareness of what is intended to be happening in the final production.

STAIR technology could enable more realistic virtual acoustics in post-production. It is commonplace for most production sound to be replaced in post-production. Sound designers are often left to guess about the acoustic space in which a sound should be placed, drawing in turn on stock reverbs. In dynamic scenes with characters quickly moving through multiple spaces, the static nature of traditional reverbs can become all the more noticeable. An alternate approach would be to record STAIRs on set or on location so that they can be applied later. There is a correlate to this in CGI with the practice of recording high-dynamic-range imaging (HDRI) references on location using specular and matte spheres so that realistic environmental lighting

conditions can be convolved later with digital assets [114]. An additional upshot of STAIR technology over other proprietary surround formats such as Dolby ATMOS is that it is fully open-source and interchangeable [2].

Virtual acoustipresence

Apart from the acoustics virtualization software designed in this project, we can also imagine special use cases for the STAIRArray hardware on its own.

Firstly, the array could be used to establish virtual acoustipresence. Virtual acoustipresence is analogous to virtual telepresence [115]. Wherein telepresence allows a user to visualize themselves in real-time in a space other than the one in which they are actually situated, acoustipresence allows a user to hear themselves in a space other than the one in which they are actually situated. An extensive setup on the user's side would not be necessary. All that would be necessary would be at least one microphone of some kind. The polar pattern and the orientation of the microphone would need to be known. But with that information in hand, a person could be reamplified in three dimensions in some remote space and then decoded back to them, say, in binaural or otherwise.

Creative installations

Continuing to focus on the hardware, the STAIRArray could be used to synthetically generate sound in response to the shifting acoustics of a space. An inspiration for this might be something like Miller Puckette and Kerry Hagan's *Remnant* [116]. In this installation piece, presence and absence are explored by actively measuring the acoustic shadows that people's bodies cast in a space and using that information to modulate acoustically rich musical material.

Because the STAIRArray has an overbuilt compute platform, the possibilities for handling additional audio DSP and networking protocols are wide. An ambitious project would

be to use the STAIRArray as a kind of acoustic Foucault pendulum (see Figure 31). A Foucault pendulum is a pendulum with sufficient dimensions to be kept in motion over a 24-hour period by the earth's rotation [117]. The STAIRArray could be similarly suspended. Through the use of a future onboard GPS module, its motion could pinpoint different networks around the world. Audio streams from those locations would then be connected, and users could communicate with and hear themselves reamplified with those in the same acoustic space as the pendulum.

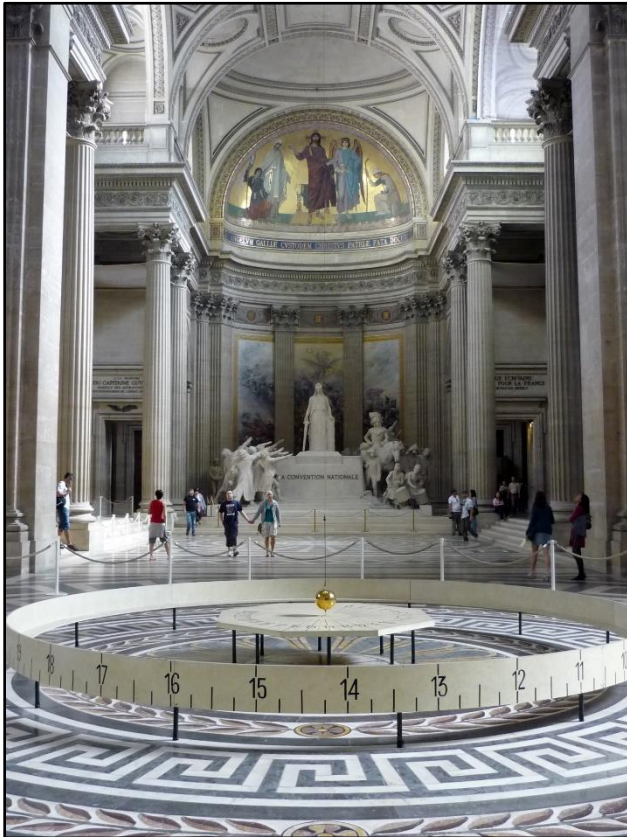


Figure 31: Foucault pendulum in the Panthéon, Paris [118]

Concluding summary

In this research, we have covered the theoretical underpinnings and justifications for how to realize virtual acoustic models that allow for 6DoF navigability. The fundamental motivation for this research has been to solve the problem of more robustly sampling an acoustic space and more seamlessly integrating the results with production software. To achieve this end, we have

drawn on the fundamental theoretical models for ambisonics and acoustic reverberation. We have looked at different approaches for modelling multivolume spaces. The common-slope model was seized upon as particularly well-suited for our purposes. We also looked at a paradigmatic example of how to achieve 6DoF by perceptually interpolating between SRIRs encoded in SOFAs. In both this case and the preceding case of the common-slope model, we saw that a pressure point was precisely the need to be able to efficiently sample more SRIRs. With this impetus in mind, we pursued our own technical solution to creating navigable virtual acoustic models. Our chief insight was in realizing that it would be possible to capture spatio-temporally continuous ambisonic IRs (which we gave the name of STAIRs). We could later positionally extract discrete SRIRs by way of the common-slope model. We developed a simple algorithm for doing so which then convolves the response with game sound that has been encoded into the ambisonic domain and then decoded to several output formats. On the physical side, to capture the real-world data to feed the model, we designed and built an apparatus for capturing STAIRs. This apparatus consists of a self-contained 3OA mic array, a 2OA speaker array, and a compute platform for managing playback and recording. We then used this apparatus to record a STAIR in a real-world location, specifically a meandering series of residential passageways. In parallel, we also generated a synthetic STAIR, both for comparison purposes and to explore the synthetic approach in its own right. We brought the attendant assets into Unreal and created virtual environments for testing the useability and verisimilitude of the results. The results were found to be suitably convincing, with differences of spatial position being smoothly and clearly audible. Nevertheless, we also discussed ways for further optimizing the software and hardware designs of this project. Next to last, we looked ahead to future work for extending and refining our technology. Importantly, we proposed two different ways of minimizing the theoretical

intermediary of the common-slope model and restoring early reflections and other unique characteristics of a space. Finally, we explored a gamut of applications for our technology, from the apparent cases of game and VR audio, to the less apparent cases of virtual acoustipresence and creative installations. The horizon is wide for building acoustigrammetry to its full potential.

REFERENCES

- [1] G. Malaperdas, “3D modeling and animation as artistic and technical practices,” *ISRG Journal of Engineering and Technology*, vol. 1, no. 3, pp. 8–11, 2025.
- [2] E. Pfanzagl-Cardone, “The DOLBY® ‘Atmos™’ system,” in *The Art and Science of 3D Audio Recording*. Cham, Switzerland: Springer, 2023, pp. 143–188.
- [3] E. Pfanzagl-Cardone, “SONY ‘360 Reality Audio’,” in *The Art and Science of 3D Audio Recording*. Cham, Switzerland: Springer, 2023, pp. 267–277.
- [4] International Telecommunication Union Radiocommunication Sector (ITU-R), *Recommendation ITU-R BS.2076-3: Audio Definition Model*, Geneva, Switzerland, 2025.
- [5] Alliance for Open Media, *Immersive Audio Model and Formats (IAMF) Specification*, AOM Working Group Draft, Apr. 21, 2025. Available: <https://aomediacodec.github.io/iamef> (aomediacodec.github.io in Bing). Accessed: May 16, 2025.
- [6] M. Debner and V. Hirvisalo, “Towards concurrent real-time audio-aware agents with deep reinforcement learning,” *OpenReview (NLDL 2025)*, 2024.
- [7] Unity Technologies, “Audio Listener,” *Unity Manual*, 2025. Available: <https://docs.unity3d.com>. Accessed: May 16, 2025.
- [8] Epic Games, “FListener — Audio Listener Structure,” *Unreal Engine API Reference*, 2024. Available: <https://docs.unrealengine.com>. Accessed: May 16, 2025.
- [9] Godot Engine Documentation, “AudioListener2D,” *Godot Docs*, 2024. Available: <https://docs.godotengine.org>. Accessed: May 16, 2025.
- [10] Magic Listener, “Magic Listener – Multi Listener Audio System for Unity,” *Unity Asset Store*, 2024. Available: <https://assetstore.unity.com>. Accessed: May 16, 2025.
- [11] Audiokinetic, “Integrating listeners,” *Wwise SDK 2025.1.4 Documentation*, 2025. Available: <https://www.audiokinetic.com>. Accessed: May 16, 2025.
- [12] M. A. Gerzon, “Periphony: With-height sound reproduction,” *J. Audio Eng. Soc.*, vol. 21, no. 1, pp. 2–10, 1973.
- [13] H. Kalf, “On the expansion of a function in terms of spherical harmonics in arbitrary dimensions,” *Project Euclid*, 1994.
- [14] E. V. Küçük, “The birth of quantum mechanics: A historical study through the canonical papers,” *arXiv:2503.13630*, 2025.

- [15] F. Zotter and M. Frank, *Ambisonics*, Springer Topics in Signal Processing, vol. 19. Cham, Switzerland: Springer, 2019.
- [16] Ambisonic Studio, “Native B-Format recording,” Aug. 2019. Available: <https://www.ambisonicstudio.com>. Accessed: May 16, 2025.
- [17] I. Corbett, *Mic It!: Microphones, Microphone Techniques, and Their Impact on the Final Mix*, 1st ed. Burlington, MA, USA: Focal Press, 2014.
- [18] M. Chapman and P. Cotterell, “Towards a comprehensive account of valid Ambisonic transformations,” in *Proc. Ambisonics Symposium*, Graz, Austria, 2009.
- [19] Audio Brewers, “ab Transcoder — Converts A-Format Ambisonics recordings to B-Format,” *KVR Audio News*, Jul. 10, 2023. Available: <https://www.kvraudio.com>. Accessed: May 16, 2025.
- [20] Acoustic Nature, “Best Ambisonic plugins: Free & paid options — Ambisonic converter plugins,” Sept. 28, 2023. Available: <https://www.acousticnature.com>. Accessed: May 16, 2025.
- [21] J. Perez-Lopez and N. Stefanakis, “Analysis of spherical isotropic noise fields with an A-Format tetrahedral microphone,” *J. Acoust. Soc. Am.*, vol. 145, no. 1, pp. 1–12, 2019.
- [22] D. Pinardi, A. Farina, and M. Binelli, “Transducer distribution on spherical arrays for Ambisonics recording and playback,” in *Proc. Immersive and 3D Audio: from Architecture to Automotive (I3DA)*, Bologna, Italy, 2023, pp. 1–9, doi: 10.1109/I3DA57090.2023.10289552.
- [23] D. Pinardi, “Spherical t-design for characterizing the spatial response of microphone arrays,” in *Proc. Immersive and 3D Audio: from Architecture to Automotive (I3DA)*, Bologna, Italy, 2021, pp. 1–8, doi: 10.1109/I3DA48870.2021.9610850.
- [24] Y. Qiao, V. Kothapally, M. Yu, and D. Yu, “Neural Ambisonic encoding for multi-speaker scenarios using a circular microphone array,” *arXiv:2409.06954*, 2024.
- [25] A. Figueroa-Duran and E. Fernandez-Grande, “Navigable reconstruction of reverberant sound fields using distributed microphone arrays,” in *Proc. Immersive and 3D Audio: from Architecture to Automotive (I3DA)*, Bologna, Italy, 2023, pp. 1–8, doi: 10.1109/I3DA57090.2023.10289135.
- [26] HOMULA RIR, “A room impulse response dataset for teleconferencing and spatial audio applications,” *arXiv:2402.13896*, 2024.

- [27] Kappis *et al.*, “Remote microphone virtual sensing with nested microphone sub-arrays,” *J. Acoust. Soc. Am.*, 2025.
- [28] T. Omoto, “A unified framework for direction and diffuseness estimation using tight frame microphone arrays,” *arXiv:2510.22183v1*, 2025.
- [29] B. N. Gover, J. G. Ryan, and M. R. Stinson, “Designing a spherical microphone array for the directional analysis of reflections and reverberation,” *J. Audio Eng. Soc.*, 2003.
- [30] mh acoustics, “Eigenmike em64 spatial microphone,” Product Documentation, 2024. Available: <https://www.mhacoustics.com>. Accessed: May 16, 2025.
- [31] S. Koyama, S. Koyama, and Y. Haneda, “Measured and theoretical performance comparison of a co-centred rigid and open spherical microphone array,” in *Proc. IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, New Paltz, NY, USA, 2017, pp. 1–5.
- [32] R. Ben-Hador and I. Neoran, “Capturing manipulation and reproduction of sampled acoustic impulse responses,” presented at the *AES Convention*, 2004. Available: <https://www.aes.org> (aes.org in Bing) (bing.com in Bing). Accessed: May 16, 2025.
- [33] Zylia Sp. z o.o., “ZYLIA ZM-1 Microphone Array,” 2024. Available: <https://www.zylia.co> (zylia.co in Bing) (bing.com in Bing). Accessed: May 16, 2025.
- [34] miniDSP Ltd., “UMA-XL: USB Microphone Array,” 2024. Available: <https://www.minidsp.com/products/usb-audio-interface/uma-xl> (minidsp.com in Bing) (bing.com in Bing). Accessed: May 16, 2025.
- [35] Voyage Audio, “Spatial Mic – Precision Spatial Audio Capture,” Voyage Audio, 2026. (Keep as originally provided.)
- [36] D. Ernst, R. Geisler, T. Kleindienst, T. Ahlefeldt, and C. Spehr, “Portable 512 MEMS microphone array for 3D intensity and beamforming measurements using an FPGA-based data acquisition system,” *Berlin Beamforming Conference (BeBeC)*, 2020, Paper D27.
- [37] B. Rose, “MEMS versus ECM: Comparing microphone technologies,” *DigiKey Electronics*, 2019. Available: <https://www.digikey.com> (digikey.com in Bing) (bing.com in Bing). Accessed: May 16, 2025.
- [38] TRX Electronics, “MEMS Microphones: Complete Engineering Guide & Comparison,” 2025. Available: <https://www.trxelectronics.com> (trxelectronics.com in Bing) (bing.com in Bing). Accessed: May 16, 2025.

- [39] C. Beeman, “How are electret and MEMS microphones different?” *Soundskrit AudioHub*, 2023. Available: <https://www.soundskrit.com> (soundskrit.com in Bing) (bing.com in Bing). Accessed: May 16, 2025.
- [40] R. Palmer, “Reassessing mechanisms of low-frequency sound localisation,” *Current Opinion in Neurobiology*, vol. 14, no. 4, pp. 457–460, 2004.
- [41] V. Välimäki, J. Parker, L. Savioja, J. Smith, and J. Abel, “Fifty years of artificial reverberation,” *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 20, no. 5, pp. 1421–1448, 2012.
- [42] D. M. Howard and J. Angus, *Acoustics and Psychoacoustics*, 4th ed. Oxford, U.K.: Focal Press, 2013.
- [43] J.-D. Polack, “Playing billiards in the concert hall: The mathematical foundations of geometrical room acoustics,” *Appl. Acoust.*, vol. 38, no. 2–4, pp. 235–244, Jan. 1993.
- [44] J.-S. Abel and P. Huang, “A simple, robust measure of reverberation echo density,” in *Proc. 121st AES Convention*, San Francisco, CA, USA, 2006.
- [45] N. Xiang and G. M. Sessler, Eds., *Acoustics, Information, and Communication: Memorial Volume in Honor of Manfred R. Schroeder*. Cham, Switzerland: Springer, 2015.
- [46] P. Götz, K. Kowalczyk, A. Silzle, and E. A. P. Habets, “Mixing time prediction using spherical microphone arrays,” *J. Acoust. Soc. Am.*, vol. 137, no. 2, pp. EL206–EL212, 2015.
- [47] J. Blauert, *Spatial Hearing*. Cambridge, MA, USA: MIT Press, 1997.
- [48] S. Leglaive, R. Badeau, and G. Richard, “Autoregressive moving average modeling of late reverberation in the frequency domain,” 2016.
- [49] T. Carpentier, M. Noisternig, and O. Warusfel, “Hybrid reverberation processor with perceptual control,” in *Proc. DAFx-14*, Erlangen, Germany, 2014.
- [50] O. Das, G. Dal Santo, S. J. Schlecht, V. Välimäki, and Z. Cvetković, “Differentiable grouped feedback delay networks for learning coupled volume acoustics,” *arXiv:2508.06686*, 2025.
- [51] W. G. Gardner, “Reverberation algorithms,” in *Applications of Digital Signal Processing to Audio and Acoustics*, M. Kahrs and K. Brandenburg, Eds. Boston, MA, USA: Kluwer Academic, 1998, pp. 85–131.

- [52] P. Majdak *et al.*, “Spatially Oriented Format for Acoustics 2.1: Introduction and recent advances,” *J. Audio Eng. Soc.*, vol. 70, no. 7/8, pp. 565–584, Jul./Aug. 2022.
- [53] N. McKenzie *et al.*, “Perceptually informed interpolation and rendering of spatial room impulse responses for room transitions,” in *Proc. 24th Int. Congr. Acoustics (ICA)*, Gyeongju, Korea, 2022, pp. 1–11.
- [54] P. A. M. Dirac, “The physical interpretation of the quantum dynamics,” *Proc. R. Soc. Lond. A*, vol. 113, no. 765, pp. 621–641, Jan. 1927.
- [55] T. H. Park, *Introduction to Digital Signal Processing: Computer Musically Speaking*. Singapore: World Scientific, 2010.
- [56] M. R. Schroeder, “New method of measuring reverberation time,” *J. Acoust. Soc. Am.*, vol. 37, pp. 409–412, 1965.
- [57] N. Xiang, A. Alamuru, I. B. Witew, and M. Vorländer, “Experimental investigations on sound energy propagation in acoustically coupled volumes using a high spatial resolution scanning system,” *J. Acoust. Soc. Am.*, vol. 143, no. 6, pp. EL437–EL442, 2018.
- [58] N. Xiang, Y. Jing, and A. C. Bockman, “Investigation of acoustically coupled enclosures using a diffusion-equation model,” *J. Acoust. Soc. Am.*, vol. 126, no. 3, pp. 1187–1198, 2009.
- [59] J. Borish, “Extension of the image model to arbitrary polyhedra,” *J. Acoust. Soc. Am.*, vol. 75, no. 6, pp. 1827–1836, 1984.
- [60] Treble Technologies, “Geometrical Solver – Image Source Method,” 2024. Available: <https://www.treble.tech> (treble.tech in Bing) (bing.com in Bing). Accessed: May 16, 2025.
- [61] O. Das, S. J. Schlecht, and E. De Sena, “Grouped feedback delay networks with frequency-dependent coupling,” *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 31, pp. 2004–2015, 2023, doi: 10.1109/TASLP.2023.3277368.
- [62] A. Luo, Y. Du, M. Tarr, J. Tenenbaum, A. Torralba, and C. Gan, “Learning neural acoustic fields,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 3165–3177, 2022.
- [63] M. Götz, S. Schlecht, and E. A. P. Habets, “Common-slope modeling of late reverberation,” in *Proc. IEEE Int. Conf. Acoustics, Speech, Signal Process. (ICASSP)*, Singapore, 2022, pp. 221–225.
- [64] F. Zotter, H. Pomberger, and M. Noisternig, “Energy preserving Ambisonic decoding,” *Acta Acustica united with Acustica*, vol. 98, no. 1, pp. 37–47, 2012.

- [65] M. Otani and H. Shigetani, “Reproduction accuracy of higher-order Ambisonics with Max rE and/or least-norm solution in decoding,” *Acoustical Science and Technology*, vol. 40, no. 1, pp. 23–28, 2019.
- [66] J. M. Batke and F. Keiler, “Using VBAP-derived panning functions for 3D Ambisonics decoding,” Technicolor Research & Innovation, Hannover, Germany, 2010.
- [67] IEEE Digital Reality, “The differences between 3DoF and 6DoF, and why,” 2022. Available: <https://digitalreality.ieee.org/publications/the-differences-between-3dof-and-6dof-and-why/> (digitalreality.ieee.org in Bing). Accessed: May 16, 2025.
- [68] Biology Insights, “What is the function of muscles that move ears?” 2025. Available: <https://biologyinsights.com> (biologyinsights.com in Bing). Accessed: May 16, 2025.
- [69] University of Bristol, “Extreme mobility of mantis shrimp eyes,” *EurekAlert!*, May 1, 2018. Available: <https://www.eurekalert.org> (eurekalert.org in Bing). Accessed: May 16, 2025.
- [70] D. Eckhoff, J. Schnupp, and A. Cassinelli, “Temporal precision and accuracy of audiovisual stimuli in mixed reality systems,” *PLoS One*, vol. 19, no. 1, e0295817, 2024.
- [71] Institute of Electronic Music and Acoustics, “IEM Plug-in Suite,” 2025. Available: <https://plugins.iem.at/> (plugins.iem.at in Bing). Accessed: May 16, 2025.
- [72] L. McCormack, “SPARTA Plug-in Suite,” 2025. Available: <https://leomccormack.github.io/sparta-site/docs/plugins/sparta-suite/> (github.io in Bing). Accessed: May 16, 2025.
- [73] Envelop, “Envelop for Live, Release 12.0.5,” 2025. Available: <https://github.com/EnvelopSound/EnvelopForLive/releases/tag/12.0.5> (github.com in Bing). Accessed: May 16, 2025.
- [74] P. Juul, “zita-rev1: Fons Adriaensen’s reverb algorithm,” 2025. Available: <https://github.com/PelleJuul/zita-rev1> (github.com in Bing). Accessed: May 16, 2025.
- [75] A. Harker and P. A. Tremblay, “The HISSTools Impulse Response Toolbox: Convolution for the masses,” in *Proc. Int. Comput. Music Conf. (ICMC)*, Ljubljana, Slovenia, 2012, pp. 148–155.
- [76] F. Hayashi, *Econometrics*. Princeton, NJ, USA: Princeton Univ. Press, 2000.

- [77] FFmpeg Developers, “atrac9dec.c – ATRAC9 decoder implementation,” 2025. Available: <https://github.com/FFmpeg/FFmpeg/blob/master/libavcodec/atrac9dec.c> (github.com in Bing). Accessed: May 16, 2025.
- [78] Firelight Technologies, “Core API – Sound::getFormat – sound_type,” FMOD Documentation, 2025. Available: https://www.fmod.com/docs/2.03/api/core-api-sound.html#sound_type (fmod.com in Bing). Accessed: May 16, 2025.
- [79] Audiokinetic, “Wwise SDK 2024.1.5 Documentation,” 2025. Available: https://www.audiokinetic.com/en/public-library/2024.1.5_8803/?source=SDK&id=index.html (audiokinetic.com in Bing). Accessed: May 16, 2025.
- [80] G. Götz, R. Falcón Pérez, S. J. Schlecht, and V. Pulkki, “Neural network for multi-exponential sound energy decay analysis,” *J. Acoust. Soc. Am.*, vol. 152, no. 2, pp. 942–953, Aug. 2022.
- [81] D. G. Malham, “Ambisonics—A technique for low-cost, high-precision, three-dimensional sound diffusion,” *Music Technology Group Technical Report*, Univ. of York, 1990.
- [82] J. Daniel, “Spatial sound encoding including near-field effect: Introducing distance-coding filters and a viable new Ambisonic format,” in *Proc. AES 23rd Int. Conf.*, 2003.
- [83] L. M. Del Bono, F. Nicoletti, and F. Ricci Tersenghi, “The most uniform distribution of points on the sphere,” *arXiv:2407.01503*, 2024.
- [84] N. W. Johnson, “Convex polyhedra with regular faces,” *Canad. J. Math.*, vol. 18, pp. 169–200, 1966.
- [85] Edison Research, *Share of Ear®*, 2024.
- [86] Y. Wycisk, K. Sander, B. Manca, R. Kopiez, and F. Platz, “The Headphone and Loudspeaker Test – Part II: A comprehensive method for playback device screening in Internet experiments,” *Behavior Research Methods*, vol. 56, pp. 362–378, 2024.
- [87] UC Berkeley Haas School of Business *et al.*, “Study shows the persuasive power headphones have on listeners,” 2022.
- [88] Dolby Laboratories, “Dolby Atmos Home Theater Installation Guidelines,” 2022. Available: <https://www.dolby.com> (dolby.com in Bing). Accessed: May 16, 2025.

- [89] NTi Audio, “DS3 Dodecahedron Speaker,” NTi Audio AG, Schaan, Liechtenstein, 2024. Available: <https://www.nti-audio.com> (nti-audio.com in Bing). Accessed: May 16, 2025.
- [90] Hottinger Brüel & Kjær, “OmniPower Sound Source Type 4292-L,” HBK, Nærum, Denmark, 2024. Available: <https://www.bksv.com> (bksv.com in Bing). Accessed: May 16, 2025.
- [91] DTU Electro, “ODEON Omni Source,” Technical University of Denmark, Lyngby, Denmark, 2024. Available: <https://www.odeon.dk> (odeon.dk in Bing). Accessed: May 16, 2025.
- [92] 3DXTech, “CarbonX ASA+CF Carbon Fiber Reinforced ASA Filament — Technical Data Sheet,” Grand Rapids, MI, USA, 2024. Available: <https://www.3dxtech.com> (3dxtech.com in Bing). Accessed: May 16, 2025.
- [93] Tang Band Speaker Co., Ltd., “T1-2025SC 3/4" Miniature Balanced Mode Radiator,” 2024. Available: <https://www.tb-speaker.com> (tb-speaker.com in Bing). Accessed: May 16, 2025.
- [94] ISO, *ISO 3382-1:2009 – Acoustics — Measurement of room acoustic parameters — Part 1: Performance spaces*, 2009.
- [95] ISO, *ISO 140-1:1978 – Acoustics — Measurement of sound insulation in buildings and of building elements — Part 1: Requirements for laboratories*, 1978.
- [96] Analog Devices, “MAX98357A: Digital Input Class-D Audio Power Amplifier — Datasheet,” 2024. Available: <https://www.analog.com> (analog.com in Bing). Accessed: May 16, 2025.
- [97] fchk, “How many audio channels are supported by the CM5?” *Raspberry Pi Forums*, May 2025. Available: <https://forums.raspberrypi.com> (raspberrypi.com in Bing). Accessed: May 16, 2025.
- [98] Texas Instruments, “TIDA-00223: Automotive Audio I²S Over Coax Class-D Amplifier Reference Design,” 2024. Available: <https://www.ti.com> (ti.com in Bing). Accessed: May 16, 2025.
- [99] R. Bauer *et al.*, “Housing influence on multi-band directional MEMS microphones inspired by *Ormia ochracea*,” in *Proc. IEEE Sensors Conference*, 2016.
- [100] High Sight, “High Sight Cablecam System,” 2024. Available: <https://www.highsight.no> (highsight.no in Bing). Accessed: May 16, 2025.

- [101] G. Götz, A. Martinez Ornelas, S. Schlecht, and V. Pulkki, “Autonomous robot twin system for room acoustic measurements,” *J. Audio Eng. Soc.*, vol. 69, no. 4, pp. 261–272, 2021.
- [102] J. Roginska, J. L. Braasch, and H. M. Behler, “3D sound spatialization with game engines: The virtual acoustics performance of a game engine and a middleware for interactive audio design,” in *Proc. AES Convention 147*, New York, NY, USA, Oct. 2019.
- [103] M. Vorländer and S. Weinzierl, *Auralization: Fundamentals of Acoustics, Modelling, Simulation, Algorithms and Acoustic Virtual Reality*. Cham, Switzerland: Springer, 2020.
- [104] F. G. Katz, “Spatial audio for virtual environments,” *J. Audio Eng. Soc.*, vol. 64, no. 7/8, pp. 546–560, 2016.
- [105] J. Larsson, D. Västfjäll, and M. Kleiner, “Perceived presence and auditory cues: The importance of realistic reverberation in virtual environments,” in *Proc. AES 22nd Int. Conf. Virtual, Synthetic and Entertainment Audio*, 2002.
- [106] CATT, *CATT Acoustic v9*, Gothenburg, Sweden, 2024. Available: <https://www.catt.se> (catt.se in Bing). Accessed: May 16, 2025.
- [107] DataKustik GmbH, *CadnaA – Computer Aided Noise Abatement*, Gilching, Germany, 2024. Available: <https://www.datakustik.com> (datakustik.com in Bing). Accessed: May 16, 2025.
- [108] AFMG Technologies GmbH, *EASE – Enhanced Acoustic Simulator for Engineers*, Berlin, Germany, 2024. Available: <https://www.afmg.eu> (afmg.eu in Bing). Accessed: May 16, 2025.
- [109] Odeon A/S, *Odeon Room Acoustics Software*, DTU Electro, Lyngby, Denmark, 2024. Available: <https://odeon.dk> (odeon.dk in Bing). Accessed: May 16, 2025.
- [110] SoundPLAN GmbH, *SoundPLANnoise – Environmental Noise Modeling Software*, Backnang, Germany, 2024. Available: <https://www.soundplan.eu> (soundplan.eu in Bing). Accessed: May 16, 2025.
- [111] Treble Technologies, *Treble Acoustic Simulation Platform*, Reykjavík, Iceland, 2024. Available: <https://www.treble.tech> (treble.tech in Bing). Accessed: May 16, 2025.
- [112] Unity Technologies, “3D Software for Architecture, Engineering & Construction,” 2024. Available: <https://unity.com> (unity.com in Bing). Accessed: May 16, 2025.

- [113] S. Hill and J. Schulte, “Lights, camera, interaction! The real-time technology of ILM StageCraft,” in *DigiPro '23: The Digital Production Symposium*, Los Angeles, CA, USA, Aug. 2023, doi: 10.1145/3603521.3604290.
- [114] M. Peteva, “Methodological creation of HDRI from 360-degree camera – Case study,” in *Computer Science and Education in Computer Science (CSECS 2022)*, Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 450, pp. 84–91, Springer, 2022.
- [115] S. Baile, “Presence and telepresence: Conceptualizations in the literature,” Thesis, 2019.
- [116] M. P. Puckette and K. Hagan, “Remnant: Exploring presence and absence through acoustic disturbances in space,” in *Proc. Int. Comput. Music Conf. (ICMC)*, 2019.
- [117] M. P. Cartmell, J. E. Faller, N. A. Lockerbie, and E. Handous, “On the modelling and testing of a laboratory-scale Foucault pendulum as a precursor for the design of a high-performance measurement instrument,” *Proc. Math. Phys. Eng. Sci.*, vol. 476, no. 2238, pp. 1–23, Jun. 2020, doi: 10.1098/rspa.2019.0680.
- [118] Rémi, “Foucault pendulum, Panthéon, Paris, France,” *Wikimedia Commons*, 2011. Available: https://commons.wikimedia.org/wiki/File:Panth%C3%A9on_Pendule_de_Foucault2.JPG (commons.wikimedia.org in Bing). License: CC BY-SA 3.0. Accessed: May 16, 2025.

APPENDICES

A. Stereo decoder attenuation matrix

Table 2: Stereo decoder attenuation matrix

	L:	R:
<i>Cft:</i>	0.076385	0.076385
<i>Lst:</i>	0.085231	0.03459
<i>Rst:</i>	0.03459	0.085231
<i>Crt:</i>	0.036296	0.036296
<i>Cftm:</i>	0.144496	0.144496
<i>Lstm:</i>	0.204814	0.032629
<i>Rstm:</i>	0.032629	0.204814
<i>Crtm:</i>	0.034953	0.034953
<i>Lfbm:</i>	1.0	0.052066
<i>Rfbm:</i>	0.052066	1.0
<i>Lrbm:</i>	0.061259	0.028949
<i>Rrbm:</i>	0.028949	0.061259
<i>Lfb:</i>	0.109455	0.046988
<i>Rfb:</i>	0.046988	0.109455
<i>Lrb:</i>	0.051651	0.031739
<i>Rrb:</i>	0.031739	0.051651

B. Binauralization matrix

Table 4: Binauralization matrix

Ambisonic channel:	ILD (dB):		ITD (ms):		4 kHz high-shelf (dB):		LPF (hz):		Notch (Hz):	
	Left:	Right:	Left:	Right:	Left:	Right:	Left:	Right:	Left:	Right:
<i>Cft:</i>	-	-	-	-	-	-	7100	7100	8200	8200
<i>Lst:</i>	-	-10.0	-	0.60	+1.2	-	-	7100	-	8200
<i>Rst:</i>	-10.0	-	0.60	-	-	+1.2	7100	-	8200	-
<i>Crt:</i>	-	-	-	-	-	-	7100	7100	8200	8200
<i>Cftm:</i>	-	-	-	-	-	-	7700	7700	7400	7400
<i>Lstm:</i>	-	-10.0	-	0.60	+1.2	-	-	7700	-	7400
<i>Rstm:</i>	-10.0	-	0.60	-	-	+1.2	7700	-	7400	-
<i>Crtm:</i>	-	-	-	-	-	-	7700	7700	7400	7400
<i>Lfbm:</i>	-	-7.1	-	0.42	+0.9	-	-	8300	-	6600
<i>Rfbm:</i>	-7.1	-	0.42	-	-	+0.9	8300	-	6600	-
<i>Lrbm:</i>	-	-7.1	-	0.42	+0.9	-	-	8300	-	6600
<i>Rrbm:</i>	-7.1	-	0.42	-	-	+0.9	8300	-	6600	-
<i>Lfb:</i>	-	-7.1	-	0.42	+0.9	-	-	8900	-	5800
<i>Rfb:</i>	-7.1	-	0.42	-	-	+0.9	8900	-	5800	-
<i>Lrb:</i>	-	-7.1	-	0.42	+0.9	-	-	8900	-	5800
<i>Rrb:</i>	-7.1	-	0.42	-	-	+0.9	8900	-	5800	-

C. Quad decoder attenuation matrix

Table 5: Quad decoder attenuation matrix

	L:	R:	Lr:	Rr:
<i>Cfi:</i>	0.076385	0.076385	0.036296	0.036296
<i>Lst:</i>	0.085231	0.03459	0.085231	0.03459
<i>Rst:</i>	0.03459	0.085231	0.03459	0.085231
<i>Crt:</i>	0.036296	0.036296	0.076385	0.076385
<i>Cfim:</i>	0.144496	0.144496	0.034953	0.034953
<i>Lstm:</i>	0.204814	0.032629	0.204814	0.032629
<i>Rstm:</i>	0.032629	0.204814	0.032629	0.204814
<i>Crtm:</i>	0.034953	0.034953	0.144496	0.144496
<i>Lfbm:</i>	1.0	0.052066	0.061259	0.028949
<i>Rfbm:</i>	0.052066	1.0	0.028949	0.061259
<i>Lrbm:</i>	0.061259	0.028949	1.0	0.052066
<i>Rrbm:</i>	0.028949	0.061259	0.052066	1.0
<i>Lfb:</i>	0.109455	0.046988	0.051651	0.031739
<i>Rfb:</i>	0.046988	0.109455	0.031739	0.051651
<i>Lrb:</i>	0.051651	0.031739	0.109455	0.046988
<i>Rrb:</i>	0.031739	0.051651	0.046988	0.109455

D. 7.1.4 decoder attenuation matrix

Table 6: 7.1.4 decoder attenuation matrix

	L:	C:	R:	Ls:	Rs:	Lr:	Rr:	Lh:	Rh:	Lhr:	Rhr:
<i>Cfi:</i>	0.037437	0.045141	0.037437	0.030886	0.030886	0.024787	0.024787	0.351066	0.351066	0.080076	0.080076
<i>Lst:</i>	0.040097	0.030886	0.023744	0.045141	0.023473	0.040097	0.023744	0.351066	0.080076	0.351066	0.080076
<i>Rst:</i>	0.023744	0.030886	0.040097	0.032	0.045141	0.023744	0.040097	0.080076	0.351066	0.080076	0.351066
<i>Crt:</i>	0.024787	0.023473	0.024787	0.030886	0.030886	0.037437	0.037437	0.080076	0.080076	0.351066	0.351066
<i>Cfm:</i>	0.071782	0.130407	0.071782	0.043469	0.043469	0.029087	0.029087	0.212824	0.212824	0.052197	0.052197
<i>Lstm:</i>	0.08871	0.043469	0.026999	0.130407	0.026081	0.08871	0.026999	0.212824	0.052197	0.212824	0.052197
<i>Rstm:</i>	0.026999	0.043469	0.08871	0.026081	0.130407	0.026999	0.071782	0.052197	0.212824	0.052197	0.212824
<i>Crm:</i>	0.029087	0.026081	0.029087	0.043469	0.043469	0.071782	0.071782	0.052197	0.052197	0.212824	0.212824
<i>Lfbm:</i>	1.0	0.187125	0.052032	0.187125	0.036992	0.064685	0.029693	0.117366	0.053348	0.053348	0.03452
<i>Rfbm:</i>	0.052032	0.187125	1.0	0.036992	0.187125	0.029693	0.064685	0.053348	0.117366	0.03452	0.053348
<i>Lrbm:</i>	0.064685	0.036992	0.029693	0.187125	0.036992	1.0	0.052032	0.053348	0.03452	0.117366	0.053348
<i>Rrbm:</i>	0.029693	0.036992	0.064685	0.036992	0.187125	0.052032	1.0	0.03452	0.053348	0.053348	0.117366
<i>Lfb:</i>	0.457407	0.212824	0.069773	0.212824	0.052197	0.08456	0.041716	0.046947	0.035566	0.035566	0.028626
<i>Rfb:</i>	0.069773	0.212824	0.457407	0.052197	0.212824	0.041716	0.08456	0.035566	0.046947	0.028626	0.035566
<i>Lrb:</i>	0.08456	0.052197	0.041716	0.212824	0.052197	0.457407	0.069773	0.035566	0.028626	0.046947	0.035566
<i>Rrb:</i>	0.041716	0.052197	0.08456	0.052197	0.212824	0.069773	0.457407	0.028626	0.035566	0.035566	0.046947

E. STAIR spectrograms

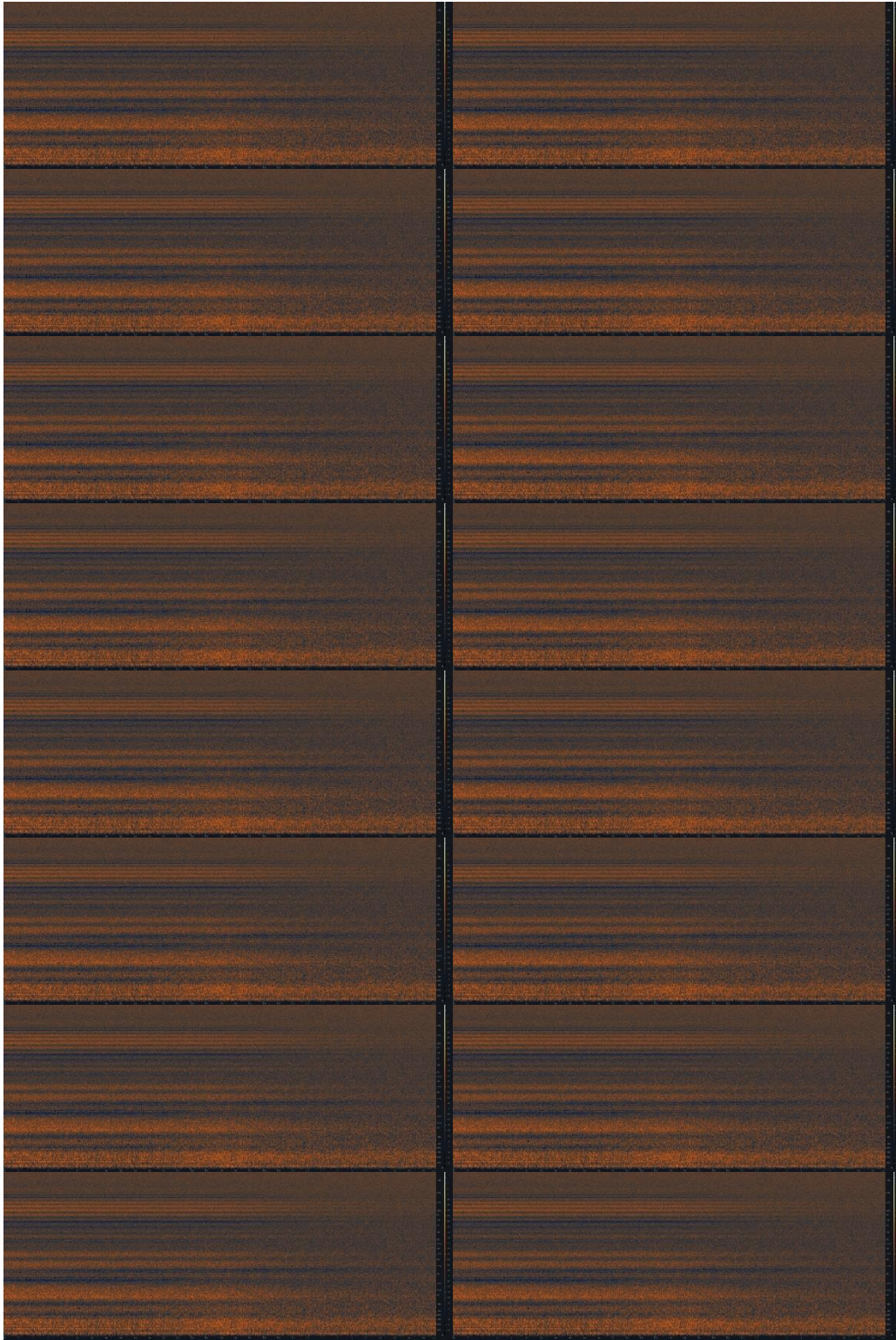


Figure 28: SyntheticSTAIR spectrograms; channels 1_Cft, ..., 16_Rrb in order from left to right and then down

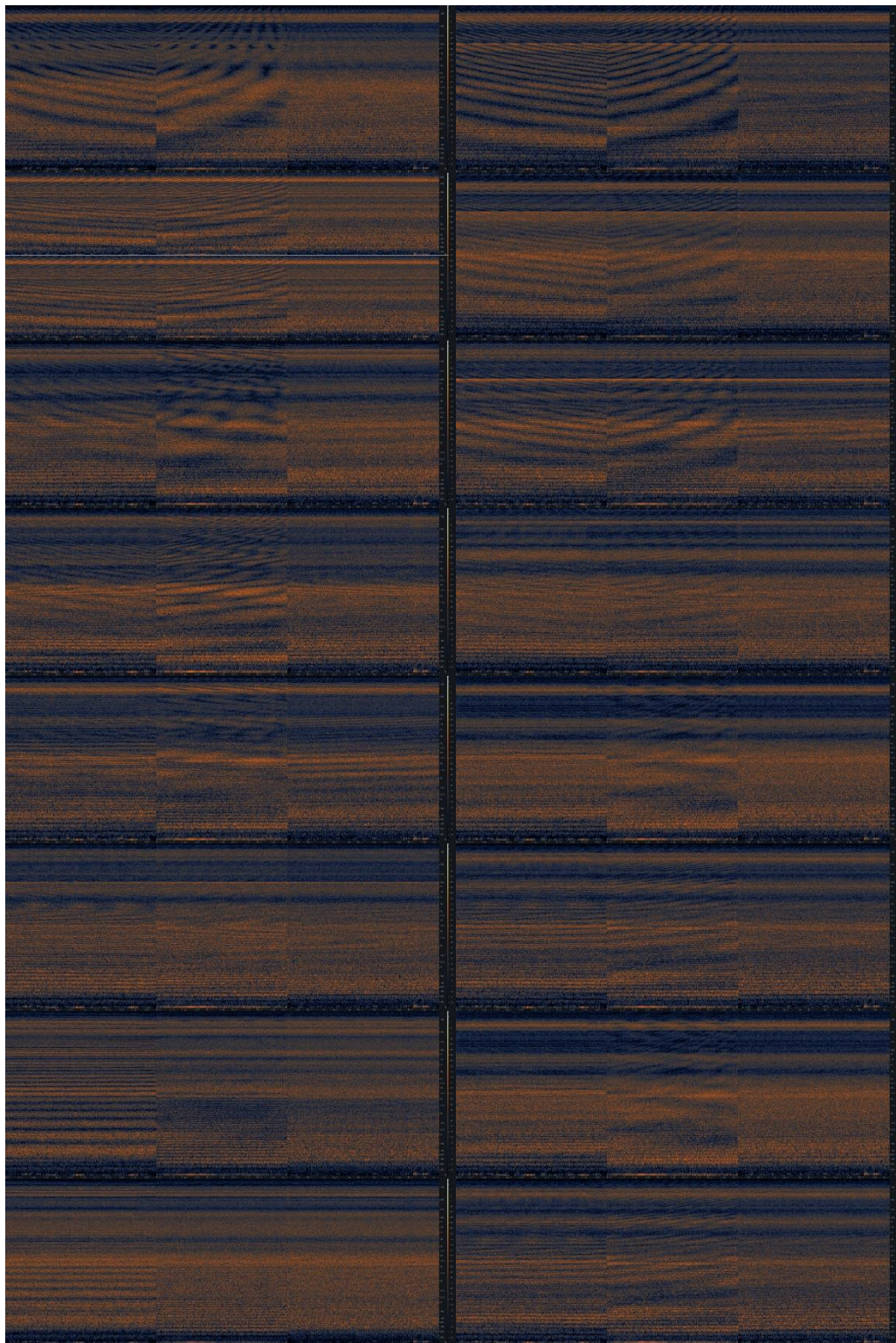


Figure 29: WaysSTAIR spectrograms; channels 1_Cft, ..., 16_Rrb in order from left to right and then down

ProQuest Number: 32701500

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by
ProQuest LLC a part of Clarivate (2026).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license
or other rights statement, as indicated in the copyright statement or in the metadata
associated with this work. Unless otherwise specified in the copyright statement
or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA